

C++从零到高级

语法、资源管理、泛型、并发、性能与工程实践

前言

很多人学 C++ 的痛苦不在语法本身，而在语法背后的执行模型没有连起来：变量到底放在哪里，引用为什么会悬空，构造函数什么时候运行，模板报错为什么像墙一样长，标准库容器为什么有时快、有时慢。只背语法表，很快就会写出能编译但不可维护的程序；只看项目，又会被概念淹没。

这本书按另一条路写：每章先从一个具体小问题开始，再把代码、内存、类型、接口和工程约束一起讲清楚。它不是语法速查表，也不是只讲高级技巧的炫技书。它的目标是让读者从第一段程序走到能写可测试、可维护、可解释性能的现代 C++ 代码。

书中代码默认使用 C++20。示例会优先使用标准库、RAII、值语义、明确所有权和小接口；类成员函数中的成员访问会写出 `this->`，这是为了在教学阶段把“成员”和“局部变量”区分清楚。运行时代码尽量避免递归，树和图的处理会用显式栈、队列或工作表说明，因为工程代码里递归深度常常不是一个可以随手忽略的细节。

核心思想

学 C++ 不是记住更多语法，而是建立三条主线：第一，值和对象如何存在；第二，资源如何被拥有和释放；第三，接口如何把复杂性限制在可测试的边界内。

如何阅读本书

如果你刚开始学 C++，可以按顺序读第一到第七章。不要急着跳到模板、并发和性能；如果你还不能解释一个对象何时构造、何时析构、谁拥有资源，那么高级语法只会放大错误。

如果你已经会写基本程序，可以重点读第五章到第十三章。读的时候建议做三件事：第一，手敲每章的核心代码；第二，故意改坏一个边界条件，看编译器、测试或运行时如何反馈；第三，把本章的接口改成自己的小工具，而不是只复制书里的函数。

每章会反复出现几个固定栏目：

- **问题入口**：先给一个真实的小需求。
- **朴素写法**：先写能想到的直接方案。
- **失败原因**：解释它为什么会错、慢或难维护。
- **现代写法**：用标准库和语言机制修正。
- **边界检查**：列出容易遗漏的输入、生命周期和复杂度。

常见误区

不要把书中代码当成唯一答案。真正要学的是代码背后的不变量：哪些数据必须一直成立，哪些资源只有一个拥有者，哪些函数只观察不修改，哪些错误必须显式返回或抛出。

目录

第一部分	从第一段程序到可解释的执行	
第 1 章	先建立执行模型	2
第 2 章	第一段可维护程序	9
第 3 章	开发环境、编译器诊断和学习实验	17
第二部分	语言基础：值、控制流、函数和抽象	
第 4 章	值、类型和初始化	23
第 5 章	控制流、函数和错误路径	29
第 6 章	类、封装和不变量	36
第 7 章	引用、指针和非拥有视图	42
第 8 章	表达式、作用域和常量	48
第 9 章	函数重载、命名空间和头文件边界	54
第三部分	现代 Cpp 的核心：资源、对象和容器	
第 10 章	生命周期、所有权和 RAII	61
第 11 章	容器、迭代器和标准库思维	69
第 12 章	错误处理和接口设计	74
第 13 章	复制、移动和对象设计	80
第 14 章	字符串、文件和流式处理	85
第 15 章	对象内存、对齐和数据布局	91
第四部分	泛型、标准库和可组合设计	
第 16 章	模板、concept 和泛型设计	97
第 17 章	标准算法、ranges 和可组合代码	104
第 18 章	lambda、函数对象和可调用设计	110
第 19 章	optional、variant、expected 风格和标准库词汇类型	119
第 20 章	编译期计算、type traits 和约束边界	125
第五部分	工程化：构建、测试、调试和性能	
第 21 章	构建、测试和调试	131
第 22 章	性能、内存和测量	135
第 23 章	异常安全、代码质量和工具链	141
第 24 章	测试驱动的重构	147
第 25 章	CMake 依赖、安装边界和 CI	151
第六部分	高级专题：并发、异步和大型代码组织	
第 26 章	并发、线程和同步	156
第 27 章	大型代码组织和演进	163
第 28 章	继承、多态和类型擦除	167
第 29 章	异步边界、取消和背压	174
第 30 章	原子操作和内存模型入门	179
第 31 章	协程入门：异步流程不是线程	184

第七部分 综合项目：把语言能力落到工程

第 32 章	项目一：命令行词频统计器	190
第 33 章	项目二：实现一个 LRU 缓存	196
第 34 章	项目三：配置加载器	202
第 35 章	项目四：一个小型任务队列	210
第 36 章	项目五：异步日志器	216
第 37 章	项目六：命令行文件索引器	224
第 38 章	项目七：插件式命令行工具	230
附录 A	学习检查清单	237
附录 B	标准库能力地图	239
附录 C	练习路线图	241
术语表		243

第零部分

从第一段程序到可解释的执行

第 1 章

先建立执行模型

1.1 从一个错误的猜测开始

很多入门材料会直接讲 `int`、`if`、`for`、`class`。这些概念当然重要，但如果一开始不回答“程序运行时到底发生了什么”，后面的指针、引用、移动语义和 RAII 都会像孤立规则。我们先看一个最小程序：

Listing 1.1 一个看似普通的程序

```
1 #include <iostream>
2 #include <string>
3
4 int main()
5 {
6     std::string name = "Ada";
7     int score = 95;
8     std::cout << name << " " << score << '\n';
9     return 0;
10 }
```

这段程序能输出 `Ada 95`。如果只从语法角度看，它包含头文件、变量、输出和返回值。但从执行模型看，它还包含更多信息：`name` 是一个对象，它管理一段字符资源；`score` 是一个整数值；`std::cout` 是标准库提供的输出流；程序从 `main` 开始执行，函数结束时局部对象按逆序销毁。

心智模型

C++ 程序可以先用四个问题观察：对象在哪里存在，谁拥有资源，什么时候构造和析构，错误如何被发现和传播。语法只是表达这些事实的工具。

1.2 先把第一段程序里的名词讲清

入门阶段最容易被一堆符号劝退。上面那段程序里，每个看似固定的写法都有明确责任：

写法	它是什么	为什么需要
<code>\#include <iostream></code>	预处理指令，要求编译前引入标准输入输出声明	没有它，编译器通常不知道 <code>std::cout</code> 的类型和可用操作
<code>\#include <string></code>	引入标准字符串类型声明	<code>std::string</code> 不是语言关键字，而是标准库类型
<code>std::</code>	标准库名字所在的命名空间前缀	避免用户自己的 <code>string</code> 、 <code>cout</code> 与标准库名字冲突
<code>int main()</code>	程序入口函数	操作系统启动程序后，C++ 运行时调用这里
<code>std::string name</code>	创建一个字符串对象，并用 <code>name</code> 访问它	字符内容由对象管理，离开作用域时释放相关资源
<code>std::cout << ...</code>	把数据写到标准输出流	<code><<</code> 会根据右侧类型选择合适的输出操作
<code>'\\n'</code>	换行字符	它只是一个字符，不是字符串，也不是刷新所有缓冲的命令

`return 0`从 `main` 返回状态码

按惯例 0 表示程序正常结束，非 0 常表示错误

这张表不要求一次背熟，但要建立一个习惯：遇到新写法，先问它属于语言、标准库、构建工具还是操作系统接口。`int`、`return` 和函数定义属于语言核心；`std::string`、`std::cout` 属于标准库；`#include` 属于预处理和编译输入组织；终端上看到的输出属于运行时外部效果。把层次分开，后面学习头文件、链接错误、命名空间和对象生命周期才不会混成一团。

1.3 编译不是运行

初学者常把“能编译”理解成“程序正确”。这不够。编译器主要检查语法、类型和一部分静态规则；运行时才会遇到输入、文件、内存、时间和并发调度。下面的程序能编译，但它不可靠：

Listing 1.2 能编译但会越界的程序

```
1 #include <iostream>
2 #include <vector>
3
4 int main()
5 {
6     std::vector<int> scores{90, 85, 77};
7     std::cout << scores[3] << '\n';
8 }
```

`scores[3]` 访问第四个元素，但数组只有三个元素。标准库的 `operator[]` 通常不会做边界检查，程序可能输出杂乱值，也可能崩溃。改成 `scores.at(3)` 后，标准库会在越界时抛出异常。这里不是说所有代码都必须用 `at`，而是要理解：接口选择影响错误出现的位置。

常见误区

能编译只说明程序通过了语言规则检查，不说明数据范围、资源生命周期和业务不变量正确。写 C++ 时必须同时关心编译期和运行期。

1.4 从源码到可执行文件

一个常见构建链条是：预处理、编译、汇编、链接。预处理处理 `##include` 和宏；编译把翻译单元转换成目标代码；链接把多个目标文件和库合成可执行文件。先记住这个模型，后面遇到“头文件重复定义”“链接找不到符号”“模板为什么要放头文件里”时就不会只靠猜。

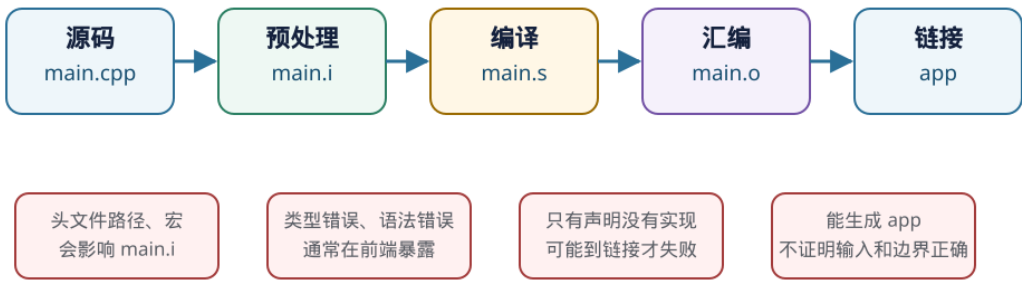
Listing 1.3 手动观察构建步骤

```
1 g++ -std=c++20 -E main.cpp -o main.i
2 g++ -std=c++20 -S main.cpp -o main.s
3 g++ -std=c++20 -c main.cpp -o main.o
4 g++ main.o -o app
```

第一步生成的 `main.i` 会很大，因为头文件内容被展开进来。第三步生成 `main.o`，它还不是最终程序。第四步链接时，链接器需要找到 `std::cout`、字符串构造函数等符号的实现。把四步拆开看，能定位很多新手错误：

从 main.cpp 到可执行文件

每个产物只能证明一部分事实，不能把“能链接”误当成“运行一定正确”。



读构建日志时先问：当前文件在哪一层？工具读入了什么？产物能证明什么，不能证明什么？

图示：源码、预处理产物、汇编、目标文件和可执行文件各自只能证明一部分事实。

产物	工具读什么	产物说明什么	不能说明什么
main.i	源文件和头文件	预处理后编译器实际看到的文本	类型和链接一定正确
main.s	预处理后的翻译单元	编译器已经能生成汇编形态	最终库符号已经找到
main.o	单个翻译单元	当前文件语法和类型基本通过，留下目标代码和符号表	整个程序已经完整
app	目标文件和库	链接器找到了需要的实现，形成可执行文件	运行时输入和业务逻辑一定正确

例如只写了 `int add(int,int);` 而没有函数体，`-c` 生成 `main.o` 可能成功，因为编译器只需要声明来检查调用；最后生成 `app` 会失败，因为链接器需要真正的实现。反过来，程序能链接成功，也不代表 `scores[3]` 这种运行期越界会被提前发现。

1.5 对象、值和名字

变量名不是对象本身，而是访问对象的名字。下面的代码里，`x` 和 `y` 是两个不同对象：

Listing 1.4 复制会产生新对象

```
1 int x = 10;
2 int y = x;
3 y = 20;
```

执行后 `x` 仍然是 `10`，`y` 变成 `20`。如果换成引用，情况就不同：

Listing 1.5 引用是已有对象的别名

```
1 int x = 10;
2 int& alias = x;
3 alias = 20;
```

这时 `x` 也变成 `20`。学习 C++ 的第一个分水岭就在这里：你必须看清一行代码是在创建新对象、观察旧对象，还是通过别名修改旧对象。

核心理念

名字、对象和值要分开理解。名字是访问入口，对象有生命周期，值是对象当前保存的信息。引用不创建新对象，它让一个已有对象有了另一个访问入口。

1.6 本章边界检查

读完本章后，你应该能回答这些问题：

- `std::string name = "Ada"` 创建了什么对象？
- 为什么 `scores[3]` 能编译却不可靠？
- 编译错误和链接错误有什么区别？
- 复制和引用在对象层面有什么不同？

后面的章节会不断回到这些问题。只要执行模型稳，语法规则就不再是散点。

1.7 把一行程序拆成执行步骤

为了让这个模型不只停留在口号上，我们把第一段程序里的两行拆开看：

Listing 1.6 两行代码里的对象变化

```
1 std::string name = "Ada";
2 std::cout << name << '\n';
```

第一行不是“变量等于字符串”这么简单。更准确的过程是：编译器看到左边类型是 `std::string`，右边是字符串字面量；于是构造一个 `std::string` 对象，让这个对象保存字符内容，并把名字 `name` 绑定到这个对象。第二行也不是把对象复制到屏幕上，而是调用输出流的插入操作，把 `name` 当前表示的字符序列写到标准输出。

如果把 `name` 改成整数，输出语句仍然长得差不多：

Listing 1.7 同一个输出符号背后是不同操作

```
1 int score = 95;
2 std::cout << score << '\n';
```

这说明 `<<` 不是一个固定的机器动作，而是一组根据类型选择的操作。初学时不用急着理解“重载”的全部规则，但要先记住：C++ 代码表面相似，不代表执行细节相同。类型决定能做什么，也决定怎么做。

1.8 三类错误要分开定位

写程序时常见三类错误。第一类是编译错误，例如少分号、类型不匹配、调用不存在的函数。第二类是链接错误，例如声明了函数却没有提供实现。第三类是运行错误，例如越界、除零、文件不存在、输入格式不对。

Listing 1.8 声明了函数但没有实现会导致链接错误

```
1 int add(int lhs, int rhs);
2
3 int main()
4 {
5     return add(1, 2);
6 }
```

这段代码能通过语法检查，因为编译器知道 `add` 的参数和返回类型。但链接时找不到函数体，所以会失败。许多初学者看到一屏英文报错就慌，其实只要先分类，定位就清楚很多：编译错看当前源文件和类型，链接错看函数实现和目标文件，运行错看输入、数据范围和生命周期。

1.9 一个最小实验流程

建议从第一章开始就固定一个实验流程。把下面代码保存为 `main.cpp`：

Listing 1.9 最小实验程序

```
1 #include <iostream>
2
3 int main()
4 {
5     int value = 41;
6     ++value;
7     std::cout << value << '\n';
8 }
```

用下面命令构建并运行：

Listing 1.10 手动编译运行

```
1 g++ -std=c++20 -Wall -Wextra -pedantic main.cpp -o app
2 ./app
```

`-Wall -Wextra -pedantic` 会打开更多警告。警告不是噪声，它经常指出“能编译但值得怀疑”的代码。以后每写一个小程序，都先做到三件事：能用固定命令构建，能说明输出为什么是这个值，能解释如果输入为空或越界会怎样。

习题与作业

把 `scores[3]` 改成 `scores.at(3)`，重新运行并观察错误出现在哪里。再把 `std::string name = "Ada"` 改成 `auto name = "Ada"`，思考这时 `name` 的类型是不是 `std::string`。这两个小实验分别训练运行期边界和类型推导。

1.10 把程序当成一条证据链

从这一章开始，本书采用一个固定习惯：每看到一段 C++ 代码，都把它拆成可验证的证据链。证据链至少有六个节点：源文件写了什么，编译器看到了什么类型，链接器需要哪些符号，运行时创建了哪些对象，作用域结束时销毁了什么，外部世界观察到了什么输出或副作用。

仍然用最小程序举例：

Listing 1.11 证据链样本程序

```
1 #include <iostream>
2 #include <string>
3
4 int main()
5 {
6     std::string name = "Ada";
7     int score = 95;
8     std::cout << name << " " << score << '\n';
9 }
```

如果只说“它输出一个名字和分数”，信息太少。更细的读法是：源文件通过 `##include`

`<string>` 引入字符串类型的声明；编译器确认 `name` 是一个 `std::string` 对象，`score` 是一个 `int` 对象；输出表达式会根据左操作数 `std::cout` 和右操作数类型选择合适的插入操作；函数返回前，局部对象按构造的逆序析构。对这个程序来说，析构似乎没有戏剧性，但等对象拥有文件、锁、内存或线程时，析构顺序就是正确性的核心证据。

阶段	要问的问题	可以看到的证据
源码	名字和类型是否表达意图	变量声明、函数签名、头文件
编译	类型规则是否成立	编译错误、警告、生成目标文件
链接	所有声明是否有实现	未定义引用、重复定义、库顺序
运行	输入和状态是否满足假设	输出、返回码、日志、异常
销毁	资源是否按路径释放	析构函数、作用域、RAII 对象
维护	改动是否能被测试挡住	单元测试、脚本、构建命令

这个表不是形式。以后遇到“为什么头文件里不能随便定义全局变量”“为什么返回引用会悬空”“为什么 `vector` 扩容后指针失效”“为什么线程退出会卡住”，都可以把问题放回这条链上定位。

1.11 一次真实的人门事故

假设你写了一个成绩统计程序，样例输入能跑：

Listing 1.12 看起来正常的输入

```
1 90 80 100
```

输出也像那么回事：

Listing 1.13 看起来正常的输出

```
1 count: 3
2 average: 90
```

但是用户传了空文件，程序崩溃。新手很容易把问题归结为“空文件特殊”，然后在 `main` 里加一堆补丁。更好的做法是沿证据链追问：

- 源码里是否有访问第一个元素的语句？
- 编译器能否知道容器运行时为空？
- 函数签名有没有表达“可能没有统计结果”？
- 输出阶段是否把“没有结果”和“平均值为零”混成一回事？

答案通常是：编译器只知道 `scores[0]` 语法成立，不知道运行时一定有元素。于是问题不在“忘记写一个 `if`”，而在接口没有表达空输入。正确修复应当让统计函数返回“有结果或无结果”的类型，例如 `std::optional<ScoreSummary>`。这就是从事故推到设计，而不是从事故推到补丁。

1.12 实验：让编译器多说话

入门阶段不要害怕命令行。用下面四个小实验建立直觉：

Listing 1.14 观察预处理和警告

```
1 g++ -std=c++20 -Wall -Wextra -pedantic -c main.cpp -o main.o
2 g++ -std=c++20 -E main.cpp -o main.i
3 g++ -std=c++20 -S main.cpp -o main.s
4 g++ main.o -o app
```

`main.i` 能让你看到头文件展开后的体积，因此理解“包含头文件不是复制粘贴那么简单”。`main.s` 能让你知道高级语法最终会变成低层指令，虽然现在不要求读懂每一行。单独生成 `main.o`

能把编译和链接分开：如果 `-c` 成功而最后链接失败，问题多半不在当前函数体语法，而在符号实现或库连接。

再做一个故意出错的版本：

Listing 1.15 故意制造警告

```
1 #include <iostream>
2
3 int main()
4 {
5     int value = 3.14;
6     std::cout << value << '\n';
7 }
```

很多编译器会警告浮点到整数的转换丢失信息。警告出现时，不要先想“怎么关掉”。先问：这是不是建模错误？如果你要的是人数，整数合理；如果你要的是平均值，截断就可能是 bug。高质量 C++ 学习要把警告当成早期证据，而不是当成噪声。

1.13 本章验收：能解释一行代码的前后果

本章真正的验收不是背出“编译、链接、运行”三个词，而是能解释一行代码的前后果。看到下面这行：

Listing 1.16 一行代码的前后果

```
1 auto summary = summarize_scores(scores);
```

你至少应该能追问：`auto` 推导出的类型是什么；`scores` 被复制、引用还是作为视图观察；函数可能失败吗；失败如何表达；返回的对象由谁拥有；这一行之后哪些状态发生了变化。如果这些问题都能回答，语法才真正落到了执行模型上。

第 2 章

第一段可维护程序

2.1 问题入口：统计一组成绩

第一段程序不要只写 `Hello, world`。我们从小需求开始：读入若干成绩，输出人数、总分、平均分、最高分和最低分。这个需求足够小，但已经覆盖输入、容器、循环、函数和错误处理。最直接的写法可能是把所有逻辑塞进 `main`：

Listing 2.1 所有逻辑挤在 `main` 里

```
1 #include <iostream>
2 #include <vector>
3
4 int main()
5 {
6     std::vector<int> scores;
7     int value = 0;
8     while (std::cin >> value) {
9         scores.push_back(value);
10    }
11
12    int sum = 0;
13    int min_score = scores[0];
14    int max_score = scores[0];
15    for (int score : scores) {
16        sum += score;
17        if (score < min_score) {
18            min_score = score;
19        }
20        if (score > max_score) {
21            max_score = score;
22        }
23    }
24
25    std::cout << scores.size() << '\n';
26    std::cout << sum / scores.size() << '\n';
27    std::cout << min_score << " " << max_score << '\n';
28 }
```

它能在普通输入下运行，但有三个问题。第一，空输入时 `scores[0]` 越界。第二，平均分用了整数除法，95 和 96 的平均值会被截断。第三，读取、统计和输出混在一起，后面不好测试。

2.2 把数据结果变成类型

先设计一个结果类型。统计结果不是五个散落的变量，而是一个概念：`ScoreSummary`。

Listing 2.2 用结构体表达统计结果

```
1 #include <cstdint>
2
```



```

3 struct ScoreSummary {
4     std::size_t count = 0;
5     int sum = 0;
6     int min_score = 0;
7     int max_score = 0;
8     double average = 0.0;
9 };

```

这个结构体没有行为，只是承载结果。它的默认值也有意义：当没有成绩时，`count` 是 `0`，平均值是 `0.0`。但是否允许空输入，不应该靠默认值偷偷决定，而应该由统计函数明确处理。

2.3 函数边界：只观察，不拥有

统计函数只需要观察一段成绩，不需要复制它。用 `std::span<const int>` 可以表达“我看一段连续整数，但不拥有它”。

Listing 2.3 统计函数使用 `span` 观察数据

```

1 #include <algorithm>
2 #include <numeric>
3 #include <optional>
4 #include <span>
5 #include <vector>
6
7 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores)
8 {
9     if (scores.empty()) {
10         return std::nullopt;
11     }
12
13     const auto [min_it, max_it] = std::minmax_element(scores.begin(), scores.end());
14     const int sum = std::accumulate(scores.begin(), scores.end(), 0);
15
16     ScoreSummary summary;
17     summary.count = scores.size();
18     summary.sum = sum;
19     summary.min_score = *min_it;
20     summary.max_score = *max_it;
21     summary.average = static_cast<double>(sum) / static_cast<double>(scores.size());
22     return summary;
23 }

```

`std::optional` 表达“可能没有结果”。这比返回一个看似正常但其实无意义的 `ScoreSummary` 更诚实。调用者必须处理空输入。

2.4 输入和输出留在边缘

现在 `main` 只做三件事：读入、调用核心逻辑、输出结果。

Listing 2.4 把核心逻辑从 `main` 中拆出来

```

1 #include <iostream>
2
3 std::vector<int> read_scores()
4 {

```



```

5     std::vector<int> scores;
6     int value = 0;
7     while (std::cin >> value) {
8         scores.push_back(value);
9     }
10    return scores;
11 }
12
13 int main()
14 {
15     const std::vector<int> scores = read_scores();
16     const std::optional<ScoreSummary> summary = summarize_scores(scores);
17     if (!summary.has_value()) {
18         std::cerr << "no scores\n";
19         return 1;
20     }
21
22     std::cout << "count: " << summary->count << '\n';
23     std::cout << "sum: " << summary->sum << '\n';
24     std::cout << "average: " << summary->average << '\n';
25     std::cout << "range: " << summary->min_score
26                 << "..." << summary->max_score << '\n';
27 }

```

这段代码的重点不是更长，而是责任更清楚。读输入的函数可以单独替换，统计函数可以用固定数组测试，输出格式可以改而不碰统计逻辑。

核心思想

好程序不是从“少写几行”开始，而是从“每段代码只负责一件事”开始。核心逻辑尽量不直接依赖终端、文件和全局状态。

2.5 本章边界检查

这个小程序至少要检查四类输入：

- 空输入：应返回 `std::nullopt`。
- 单个成绩：最小值和最大值相同。
- 多个成绩：平均分不能被整数除法截断。
- 非法文本：当前读入逻辑会在失败处停止，这是否符合需求要由调用场景决定。

从第一段程序开始，就把“能跑”和“可解释、可测试、可维护”分开。后者才是工程里的 C++。

2.6 一步一步推导统计函数

为什么统计函数要先处理空输入？从最小反例看最清楚。输入为空时，根本不存在最小值、最大值和平均值。如果代码为了省事把它们设成 0，调用者就会误以为真的统计到了一个值为 0 的成绩。于是函数的返回类型不能只是 `ScoreSummary`，还要能表达“没有结果”。

再看单元素输入：

Listing 2.5 单元素时每个字段都能推导出来

```

1 const std::vector<int> scores{91};
2 // count = 1
3 // sum = 91
4 // min_score = 91

```

```
5 // max_score = 91
6 // average = 91.0
```

单元素不是特殊麻烦，而是最好的基准。只要它成立，多元素就可以理解成“把同一套规则重复应用”。扫描第一个元素时，最小值和最大值都等于它；扫描后续元素时，只在发现更小或更大时更新边界。标准库的 `std::minmax_element` 把这个扫描过程封装好了，但你仍然要知道它在做什么。

2.7 完整文件版本

前面的代码片段可以合成一个可运行文件。入门阶段不要只看片段，至少要能把它们合起来编译：

先看文件顶部这些 `\#include`。它们不是装饰，也不是“多写总没错”。每个头文件都给后面的代码提供一个具体名字：

头文件	提供的主要名字	本程序为什么用它
<code><algorithm></code>	<code>std::minmax_element</code>	一次扫描找到最小和最大元素
<code><iostream></code>	<code>std::cin</code> 、 <code>std::cout</code> 、 <code>std::cerr</code>	从标准输入读取，向标准输出和错误输出写文本
<code><numeric></code>	<code>std::accumulate</code>	把一段整数累加成分
<code><optional></code>	<code>std::optional</code> 、 <code>std::nullopt</code>	表达空输入时没有统计结果
<code></code>	<code>std::span</code>	让统计函数观察一段连续整数，而不拥有或复制它
<code><vector></code>	<code>std::vector</code>	保存读入的成绩列表

如果漏掉某个头文件，编译器报的往往不是“你忘了 include”，而是“某个名字不存在”或“类型不完整”。读错误时要回到这张表：代码使用了哪个标准库名字，它应该来自哪个头文件。

Listing 2.6 完整成绩统计程序

```
1 #include <algorithm>
2 #include <iostream>
3 #include <numeric>
4 #include <optional>
5 #include <span>
6 #include <vector>
7
8 struct ScoreSummary {
9     std::size_t count = 0;
10    int sum = 0;
11    int min_score = 0;
12    int max_score = 0;
13    double average = 0.0;
14 };
15
16 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores)
17 {
18     if (scores.empty()) {
19         return std::nullopt;
20     }
21
22     const auto [min_it, max_it] = std::minmax_element(scores.begin(), scores.end());
```

```

23     const int sum = std::accumulate(scores.begin(), scores.end(), 0);
24
25     return ScoreSummary{
26         .count = scores.size(),
27         .sum = sum,
28         .min_score = *min_it,
29         .max_score = *max_it,
30         .average = static_cast<double>(sum) / static_cast<double>(scores.size()),
31     };
32 }
33
34 std::vector<int> read_scores()
35 {
36     std::vector<int> scores;
37     int value = 0;
38     while (std::cin >> value) {
39         scores.push_back(value);
40     }
41     return scores;
42 }
43
44 int main()
45 {
46     const std::vector<int> scores = read_scores();
47     const std::optional<ScoreSummary> summary = summarize_scores(scores);
48     if (!summary.has_value()) {
49         std::cerr << "no scores\n";
50         return 1;
51     }
52
53     std::cout << "count: " << summary->count << '\n';
54     std::cout << "sum: " << summary->sum << '\n';
55     std::cout << "average: " << summary->average << '\n';
56     std::cout << "range: " << summary->min_score
57         << "..." << summary->max_score << '\n';
58 }

```

运行时可以这样喂输入：

Listing 2.7 用管道提供输入

```
1 echo "90 80 100" | ./score
```

如果没有任何输入，程序返回错误码 **1**。如果有输入，核心统计逻辑完全不关心输入来自键盘、文件还是测试数组。这就是把边界留在边缘的价值。

2.8 从测试反推接口

接口不是凭感觉定的。先写出想验证的行为：

Listing 2.8 从行为反推接口

```

1 const std::vector<int> empty;
2 assert(!summarize_scores(empty).has_value());
3

```

```

4 const std::vector<int> one{88};
5 const auto single = summarize_scores(one);
6 assert(single.has_value());
7 assert(single->min_score == 88);
8 assert(single->max_score == 88);
9 assert(single->average == 88.0);

```

这些测试逼着我们承认两个事实：空输入没有统计结果，非空输入必须保留小数平均值。于是 `std::optional<ScoreSummary>` 和 `double average` 都不是装饰，而是从需求和边界推出来的。

习题与作业

把成绩限制为 0 到 100。不要在统计函数里偷偷忽略非法值，而是设计一个读取或校验阶段，让非法输入能被明确报告。完成后再给空输入、负数、超过 100、单元素和普通多元素各写一个测试。

2.9 从需求合同推到代码形状

CPU 卷讲程序时会先建立“账本”：输入是什么，状态是什么，输出是什么，失败怎么记录。这个成绩统计程序也应当这样学。不要一上来写循环，先写合同：

项目	合同
输入	零个或多个整数成绩，允许空白分隔
合法范围	每个成绩必须在 0 到 100 之间
成功输出	人数、总分、最小值、最大值、平均值
空输入	没有统计结果，不能伪装成平均值 0
非法输入	明确报告第几个 token 或哪一行有问题
核心逻辑	不依赖终端，不直接读写文件，便于测试

合同写完后，代码形状就不是凭感觉决定的。读入阶段负责把文本变成整数并校验范围；统计阶段只接收已经合法的整数；输出阶段把结果变成人能读的文本。这样拆开后，任何一个阶段出错，都能知道该改哪里。

2.10 错误版本为什么危险

很多入门代码会这样处理非法输入：

Listing 2.9 静默跳过坏数据的危险版本

```

1 std::vector<int> read_scores()
2 {
3     std::vector<int> scores;
4     int value = 0;
5     while (std::cin >> value) {
6         if (value >= 0 && value <= 100) {
7             scores.push_back(value);
8         }
9     }
10    return scores;
11 }

```

它的问题不是语法，而是业务语义：如果输入是 `90 -1 100`，程序会悄悄统计 90 和 100。用户以为三条数据都参与了统计，结果却被吞掉一条。对于学习者来说，这种代码尤其坏，因为它把“数据不合法”变成了“看似成功”。更好的接口要能表达读取失败：

Listing 2.10 读取结果显式表达失败

```

1 struct ReadError {
2     std::string token;
3     std::string message;
4 };
5
6 struct ReadScoresResult {
7     std::vector<int> scores;
8     std::optional<ReadError> error;
9 };

```

有了这个类型，调用者必须面对两种情况：要么拿到合法成绩列表，要么拿到错误。入门阶段先用这种简单结果类型，后续可以换成 `std::expected` 或项目自己的错误模型。

2.11 把输入解析也做成可测试核心

为了测试方便，读取函数不要直接绑死 `std::cin`。它可以接收任意输入流：

Listing 2.11 从输入流读取并校验成绩

```

1 constexpr int SCORE_MIN = 0;
2 constexpr int SCORE_MAX = 100;
3
4 ReadScoresResult read_scores(std::istream& input)
5 {
6     ReadScoresResult result;
7     std::string token;
8     while (input >> token) {
9         const std::optional<int> parsed = parse_int_fast(token);
10        if (!parsed.has_value()) {
11            return {.error = ReadError{.token = token,
12                                       .message = "score must be an integer"}};
13        }
14        if (*parsed < SCORE_MIN || *parsed > SCORE_MAX) {
15            return {.error = ReadError{.token = token,
16                                       .message = "score must be 0..100"}};
17        }
18        result.scores.push_back(*parsed);
19    }
20    return result;
21 }

```

这个函数仍然简单，但它已经有工程味道：输入来源被抽象成 `std::istream&`，范围常量有名字，错误带着原始 token。现在测试非法文本时，不需要在终端手动输入：

Listing 2.12 用字符串流测试读取阶段

```

1 std::istringstream input{"90 abc 100"};
2 const ReadScoresResult result = read_scores(input);
3 assert(result.error.has_value());
4 assert(result.error->token == "abc");

```

这就是“从零”阶段最该建立的能力：把看似只能手动试的程序，拆成可以自动验证的函数。

2.12 从样例推到测试矩阵

一个案例讲细，不是只给一组样例。要把边界列成矩阵：

场景	输入	期望
空输入	空字符串	统计阶段返回无结果
单个成绩	88	最小、最大、平均都为 88
普通成绩	90 80 100	人数 3，平均 90.0
非法文本	90 abc	读取阶段报整数错误
范围过小	-1	读取阶段报范围错误
范围过大	101	读取阶段报范围错误
小数文本	99.5	如果合同只收整数，应报错

这个矩阵能反推实现。如果 99.5 被 `std::stoi` 解析成 99，说明数字解析没有检查完整消费；如果空输入输出平均值 0，说明统计接口把“无数据”和“真实为零”混淆了。

2.13 主程序最后才组装

当读取和统计都可测试后，`main` 才负责组装：

Listing 2.13 主程序只连接阶段

```

1 int run(std::istream& input, std::ostream& output, std::ostream& error)
2 {
3     const ReadScoresResult read = read_scores(input);
4     if (read.error.has_value()) {
5         error << read.error->message << ": " << read.error->token << '\n';
6         return 2;
7     }
8
9     const std::optional<ScoreSummary> summary = summarize_scores(read.scores);
10    if (!summary.has_value()) {
11        error << "no scores\n";
12        return 1;
13    }
14
15    output << "count: " << summary->count << '\n';
16    output << "sum: " << summary->sum << '\n';
17    output << "average: " << summary->average << '\n';
18    return 0;
19 }
```

这段代码的价值是边界清晰：`run` 可以在测试里传 `std::istringstream` 和 `std::ostringstream`，真正的 `main` 只需要把 `std::cin`、`std::cout`、`std::cerr` 传进去。以后换成文件输入、网络输入或图形界面，核心统计逻辑都不用重写。

习题与作业

把本程序改成“每行一个学生，格式为姓名和成绩”。要求错误消息带行号；统计仍然只统计合法成绩；姓名不能为空。先写测试矩阵，再改代码。不要让 `main` 直接承担解析细节。

第 3 章

开发环境、编译器诊断和学习实验

3.1 问题入口：同一段代码为什么有人能编译，有人不能

学习 C++ 时，经常遇到“我这里能编译，你那里不行”。原因可能不是代码本身，而是编译器版本、标准选项、标准库实现、构建类型和警告设置不同。下面这段代码使用了 C++20 的 `std::span`：

Listing 3.1 依赖 Cpp20 的代码

```
1 #include <span>
2 #include <vector>
3
4 int sum(std::span<const int> values)
5 {
6     int total = 0;
7     for (int value : values) {
8         total += value;
9     }
10    return total;
11 }
```

如果编译命令没有写 `-std=c++20`，编译器可能根本不认识 `std::span`。因此从第一天开始，构建命令就应该是学习记录的一部分，而不是“随便点一下运行”。

3.2 固定最小编译命令

单文件实验建议从下面命令开始：

Listing 3.2 推荐的单文件实验命令

```
1 g++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion main.cpp -o app
2 ./app
```

这条命令里的每个词都要能解释：

片段	作用	常见误解
<code>g++</code>	调用 GNU C++ 编译器驱动，负责编译并在需要时调用链接器	它不是 C++ 语言本身，只是一个工具实现
<code>-std=c++20</code>	固定语言标准版本，让 <code>std::span</code> 等 C++20 名字可用	不写时环境默认版本可能因机器而异
<code>-Wall -Wextra</code>	打开一组常见警告	<code>-Wall</code> 不是所有警告，只是常用集合
<code>-Wpedantic</code>	提醒非标准扩展或不够可移植的写法	能编译不代表所有编译器都接受
<code>-Wconversion</code>	提醒可能丢信息的隐式数值转换	不是所有转换都错，但必须有业务理由
<code>main.cpp</code>	输入源文件	多文件项目不能只编译一个源文件就期待所有实现都存在

-o app

指定输出程序名

没有它时输出名通常是 **a.out**

-std=c++20 固定语言版本。**-Wall -Wextra -Wpedantic** 打开常见警告。**-Wconversion** 会提示一些隐式转换风险。不要把警告当成“编译器太烦”。它们经常指出真实问题：

Listing 3.3 隐式转换可能丢信息

```
1 double average = 95.8;
2 int score = average;
```

如果业务上就是要截断，应该显式写出转换，并在变量名或附近逻辑中说明为什么可以截断。隐式发生的丢失信息，最容易藏 bug。

3.3 读错误信息的顺序

编译错误信息可能很长，尤其包含模板时。不要从第一行开始逐字翻译，先按这个顺序找：

1. 第一个出现你自己文件路径的位置。
2. 报错行附近的真正表达式。
3. 错误类别：类型不匹配、找不到名字、没有匹配函数、约束不满足。
4. 后续 note 里给出的候选函数或约束原因。

例如：

Listing 3.4 类型不匹配示例

```
1 std::vector<int> values{1, 2, 3};
2 std::string text = values;
```

编译器可能打印一串构造函数候选，但核心错误很简单：**std::string** 没有办法从 **std::vector<int>** 构造。先抓核心类别，再看候选细节。

3.4 Debug 和 Release 的区别

Debug 构建更适合调试，Release 构建更适合性能测量。不要用 Debug 时间判断算法快慢：

Listing 3.5 两种构建模式

```
1 g++ -std=c++20 -O0 -g main.cpp -o app-debug
2 g++ -std=c++20 -O2 -DNDEBUG main.cpp -o app-release
```

-O0 -g 保留调试信息，不做太多优化。**-O2** 会做优化。**-DNDEBUG** 会关闭 **assert**。这推出一个重要规则：**assert** 只能检查内部不变量，不能替代用户输入校验。因为 Release 下它可能根本不执行。

3.5 用小实验验证规则

学 C++ 不应该只背结论。每个规则都可以设计小实验。例如验证 **vector** 扩容导致指针失效：

Listing 3.6 观察 vector 扩容

```
1 std::vector<int> values;
2 values.reserve(1);
3 values.push_back(10);
4 const int* old_address = values.data();
5 values.push_back(20);
6 const int* new_address = values.data();
```



```
7 std::cout << (old_address == new_address) << '\n';
```

这段代码不保证一定输出 0，因为实现可以有不同扩容策略。但当输出 0 时，你就能直观看到原指针不再指向同一块存储。实验不是替代标准，而是帮助你把抽象规则和运行现象连起来。

3.6 最小项目脚手架

当单文件开始变长，就应拆成最小项目：

Listing 3.7 最小项目目录

```
1 demo/
2   CMakeLists.txt
3   include/demo/math.hpp
4   src/math.cpp
5   tests/math_tests.cpp
```

构建命令固定为：

Listing 3.8 固定项目构建命令

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
2 cmake --build build
3 ctest --test-dir build --output-on-failure
```

这三个命令让项目能被别人重复构建。能重复构建，问题才容易复现；能复现，才谈得上调试和修复。

习题与作业

建立一个 demo 项目，实现 `sum(std::span<const int>)`。先故意不写 `-std=c++20`，观察错误；再加上标准选项。然后加入一个会触发 `-Wconversion` 的例子，写下警告真正提醒了什么风险。

3.7 一次完整诊断演练

只讲“读第一条错误”还不够。我们做一个完整演练。源文件如下：

Listing 3.9 含有多类错误的实验程序

```
1 #include <iostream>
2 #include <vector>
3
4 int total(std::vector<int> values)
5 {
6     int sum = 0;
7     for (int value : values) {
8         sum += value;
9     }
10    return sum;
11 }
12
13 int main()
14 {
15     std::vector<double> values{1.2, 2.8};
16     std::cout << total(values) << '\n';
```

17 }

编译器会报类型不匹配: `total` 需要 `std::vector<int>`, 调用者给的是 `std::vector<double>`。新手常问: “里面都是数字, 为什么不能传?” 因为 `std::vector<int>` 和 `std::vector<double>` 是两个不同类型的容器, 不能自动把整组元素逐个转换。这个错误推动我们重新设计接口: 如果函数只求和整数, 就调用者必须给整数; 如果函数应该支持多种数字类型, 就要进入模板或 `ranges` 的设计, 而不是期待容器自动变形。

再看函数参数 `std::vector<int> values`。即使调用类型正确, 这里也会复制整个容器。只观察时应该写成:

Listing 3.10 诊断后修正接口

```
1 int total(std::span<const int> values)
2 {
3     int sum = 0;
4     for (int value : values) {
5         sum += value;
6     }
7     return sum;
8 }
```

同一次诊断里, 我们同时看到了类型合同和性能合同。高质量读错误信息, 不是把报错消掉就结束, 而是追问接口是否表达了真实意图。

3.8 链接错误演练

再故意制造一个链接错误:

Listing 3.11 只有声明没有定义

```
1 int multiply(int lhs, int rhs);
2
3 int main()
4 {
5     return multiply(6, 7);
6 }
```

这段代码能编译成目标文件, 因为编译器只需要知道 `multiply` 的声明就能检查调用。但链接时会失败, 因为最终程序需要函数体。诊断链条是:

1. 当前源文件语法和类型检查通过。
2. 目标文件里留下一个未解析符号。
3. 链接器找遍参与链接的目标文件和库, 找不到实现。
4. 报告未定义引用。

修复方法不是在调用点乱改, 而是提供实现并确保实现文件参与构建:

Listing 3.12 提供函数定义

```
1 int multiply(int lhs, int rhs)
2 {
3     return lhs * rhs;
4 }
```

当项目有多个源文件时, 链接错误经常来自 CMake 漏加源文件、库链接顺序不对、声明和定义签名不一致。看到链接错误, 先查 “实现在哪里、有没有被编进目标”。

3.9 警告分级处理

不是所有警告严重程度相同，但都要分类。可以按四类处理：

类别	例子	处理
正确性风险	未初始化变量、悬空引用、符号比较混乱	立即修复
信息丢失	窄化转换、浮点到整数	明确业务意图，必要时显式转换
可维护性	未使用变量、遮蔽名字	删除或重命名
平台差异	类型宽度、格式化占位不匹配	用标准类型和正确格式

如果某个警告确实是误报，也不要随手全局关闭。优先用更清楚的代码让警告消失；实在需要抑制，也应该局部、带原因。

3.10 最小复现能力

工程里遇到复杂错误时，要学会把问题缩到最小复现。最小复现不是把整项目发给别人，而是保留触发问题的最少文件、最少命令和最少输入。例如：

Listing 3.13 最小复现记录

```

1 compiler: g++ 13.2
2 command: g++ -std=c++20 -Wall -Wextra main.cpp -o app
3 input: empty file
4 expected: report no scores
5 actual: segmentation fault

```

这份记录能让别人快速进入同一现场。你自己调试时也一样：把不可控因素减到最少，错误才会暴露出真正形状。

3.11 工具链验收清单

进入后续章节前，你应该能独立完成：

- 写出固定编译命令并解释每个选项。
- 区分编译错误、链接错误、运行错误和警告。
- 根据第一条自己文件路径定位错误位置。
- 用 Debug 构建调试，用 Release 构建测性能。
- 把复杂错误缩成可重复的最小复现。

这些能力比会背某个语法点更基础。不会诊断，写得越多，问题堆得越快。

第零部分

语言基础：值、控制流、函数和抽象

第 4 章

值、类型和初始化

4.1 问题入口：金额为什么不能随便用 double

假设要写一个购物车，最直接的金额表示是 `double`：

Listing 4.1 金额直接用 `double` 的朴素写法

```
1 double price = 0.1;
2 double total = price + price + price;
3 if (total == 0.3) {
4     // ...
5 }
```

这段代码的问题不在 C++ 独有，而在二进制浮点无法精确表示很多十进制小数。 $0.1 + 0.1 + 0.1$ 未必和 0.3 完全相等。金额通常应该用最小单位的整数，例如“分”。

Listing 4.2 用整数分表示金额

```
1 #include <stdint>
2
3 struct Money {
4     std::int64_t cents = 0;
5 };
6
7 Money add(Money lhs, Money rhs)
8 {
9     return Money{.cents = lhs.cents + rhs.cents};
10 }
```

这里的重点是类型不是装饰。类型表达业务含义，也限制错误。一个普通 `int` 可能是年龄、下标、数量、分数或错误码；而 `Money` 明确告诉读者它表示金额。

4.2 初始化：少给错误留入口

C++ 有多种初始化形式。入门阶段建议优先使用大括号初始化，因为它能挡住一部分窄化转换。

Listing 4.3 大括号初始化拒绝窄化转换

```
1 int ok{42};
2 // int bad{3.14}; // 编译错误：double 到 int 会丢失信息
3 int truncated = 3.14; // 可以编译，但值变成 3
```

不要把这理解成“永远只用一种形式”。构造对象、调用工厂函数、使用标准库容器时会遇到不同写法。先建立原则：初始化应该让对象一出生就处在有效状态。

4.3 `const` 不是形式主义

`const` 的意义是把“不修改”写进接口。下面这个函数只观察价格列表，不应该复制，也不应该修改。

Listing 4.4 用 const 引用表达只读访问

```

1 #include <vector>
2
3 std::int64_t total_cents(const std::vector<Money>& items)
4 {
5     std::int64_t total = 0;
6     for (const Money& item : items) {
7         total += item.cents;
8     }
9     return total;
10 }

```

如果写成 `std::vector<Money> items`，每次调用都会复制整个数组。复制不一定错误，但这里没有必要。接口应该表达意图：观察就用 `const&` 或 `std::span<const T>`，拥有或需要本地副本时才按值传递。

4.4 auto 的正确用法

`auto` 可以减少重复类型，但不能掩盖所有权和复制。

Listing 4.5 range-for 中避免不必要复制

```

1 std::vector<std::string> names = {"Ada", "Bjarne", "Grace"};
2
3 for (const auto& name : names) {
4     // 只观察，不复制 string
5 }
6
7 for (auto name : names) {
8     // 每轮复制一个 string，除非确实需要本地副本
9 }

```

教学阶段要养成一个习惯：看到 `auto`，仍然要问它推导出的到底是值、引用还是指针。语法短不等于语义简单。

4.5 边界：整数溢出和符号混用

整数类型也有坑。`int` 通常是 32 位，累加大数据时可能溢出。`std::size_t` 是无符号类型，和 `int` 混用时可能产生意外比较。

Listing 4.6 避免有符号和无符号混乱

```

1 std::vector<int> values{1, 2, 3};
2 for (std::size_t i = 0; i < values.size(); ++i) {
3     // i 和 size() 类型一致
4 }

```

但如果下标要向左移动到 `-1` 表示结束，`std::size_t` 就不适合。类型选择不是背规则，而是看值域和操作。

常见误区

类型错误常常不是编译错误，而是建模错误。金额、长度、数量、索引、标识符和错误码不要长期混在同一种裸整数里。

4.6 从金额例子继续推到类型设计

只把金额改成整数分还不够。下面这段代码仍然可能把“分”和“件数”混起来：

Listing 4.7 裸整数仍然容易混淆含义

```
1 std::int64_t cents = 1299;
2 int quantity = 3;
3 std::int64_t wrong = cents + quantity; // 编译器不知道业务上不合理
```

把金额包装成 **Money** 后，错误会更难发生。再进一步，可以把常见操作写成函数：

Listing 4.8 让金额操作表达业务含义

```
1 struct Money {
2     std::int64_t cents = 0;
3 };
4
5 Money multiply(Money price, int quantity)
6 {
7     return Money{.cents = price.cents * quantity};
8 }
9
10 Money add(Money lhs, Money rhs)
11 {
12     return Money{.cents = lhs.cents + rhs.cents};
13 }
```

这不是为了让代码变繁琐，而是把业务边界交给类型系统。以后如果要禁止负金额、处理货币种类、做舍入策略，修改点会集中在 **Money** 附近，而不是散落在所有 **int** 计算里。

4.7 值、引用和视图的选择

函数参数最常见的三种形态是按值、按引用和视图。可以用一个用户列表作为判断样本：

Listing 4.9 三种参数形态表达不同意图

```
1 struct User {
2     int id = 0;
3     std::string name;
4 };
5
6 void rename(User& user, std::string new_name)
7 {
8     user.name = std::move(new_name);
9 }
10
11 void print_user(const User& user)
12 {
13     std::cout << user.id << " " << user.name << '\n';
14 }
15
16 std::size_t total_name_length(std::span<const User> users)
17 {
18     std::size_t total = 0;
19     for (const User& user : users) {
```

```

20     total += user.name.size();
21 }
22 return total;
23 }

```

`rename` 要修改已有对象，所以用非 `const` 引用。`print_user` 只观察一个对象，所以用 `const&`。`total_name_length` 观察一段连续对象，所以用 `std::span<const User>`。这三个接口的区别不是语法偏好，而是所有权和修改权限不同。

4.8 初始化和有效状态

初始化的目标是让对象一出生就可用。反例是先创建半成品，再靠后续赋值补齐：

Listing 4.10 半成品对象容易被提前使用

```

1 Money price;
2 // 中间几十行代码
3 price.cents = 1299;

```

更好的写法是直接给出有效值：

Listing 4.11 直接构造有效值

```

1 Money price{.cents = 1299};

```

如果值需要校验，就不要暴露裸字段，而是放到类的构造函数或工厂函数里。也就是说，初始化不是“把值塞进去”，而是“建立一个满足规则的对象”。

习题与作业

设计一个 `Percent` 类型表示折扣百分比，要求范围是 `0` 到 `100`。先写一个能被误用的裸 `int` 版本，再把它改成类型。重点观察：哪些错误从运行期判断提前到了构造边界。

4.9 金额案例要从业务事故讲起

只说“金额不要用 `double`”还不够。我们从一个实际账单事故推导：购物车有三件商品，每件 `0.10` 元，页面显示总价 `0.30`，结算服务却在比较优惠门槛时认为它不等于 `0.30`。这类 bug 最糟糕的地方是样子很正常，只有在等值比较、四舍五入或跨系统传输时才暴露。

错误版本如下：

Listing 4.12 金额浮点事故

```

1 double price = 0.10;
2 double total = price + price + price;
3 bool eligible = total == 0.30;

```

这段代码的推理链条是错的：业务世界里的 `0.10` 是十进制金额，机器里的 `double` 是二进制浮点近似值。近似值可以做科学计算，但不适合表达需要精确计账的最小货币单位。于是设计不应该从“怎么比较 `double`”开始，而应该从“账本的最小单位是什么”开始。人民币账单通常可以用“分”作为最小单位：

Listing 4.13 金额以最小单位入账

```

1 struct Money {
2     std::int64_t cents = 0;
3 };

```


这还只是第一步。真正的类型设计要回答：能不能为负，能不能乘数量，折扣如何舍入，是否允许不同币种相加。每多一个业务问题，就应该在类型边界上留下证据。

4.10 从裸整数到强类型

裸整数版本仍然容易把“分”和“件数”相加。我们可以把构造边界收紧：

Listing 4.14 用工厂函数建立金额

```
1 class Money {
2 public:
3     [[nodiscard]] static Money from_cents(std::int64_t cents)
4     {
5         return Money{cents};
6     }
7
8     [[nodiscard]] std::int64_t cents() const noexcept
9     {
10        return this->cents_;
11    }
12
13 private:
14     explicit Money(std::int64_t cents)
15         : cents_{cents}
16     {
17     }
18
19     std::int64_t cents_ = 0;
20 };
```

现在调用点必须写出 `Money::from_cents(1299)`。这看似多几个字符，实际上是在给读者和编译器提供业务标签。下一步再定义允许的运算：

Listing 4.15 只开放有意义的金额运算

```
1 Money add(Money lhs, Money rhs)
2 {
3     return Money::from_cents(lhs.cents() + rhs.cents());
4 }
5
6 Money multiply(Money price, int quantity)
7 {
8     return Money::from_cents(price.cents() * quantity);
9 }
```

注意这里没有定义 `Money + int`。如果业务上没有意义，就不要为了方便给错误开门。强类型不是“把字段藏起来”，而是让非法组合更难写出来。

4.11 溢出不是理论问题

把金额放进整数后，仍然要面对溢出。假设单笔金额用 `int` 表示分，一个订单批量导入了很多大额项目，求和可能超过 `int` 上限。更稳妥的做法是用足够宽的类型，并在边界处检查：

Listing 4.16 累加前先选择足够宽的类型

```
1 std::optional<Money> total_price(std::span<const Money> items)
```

```

2 {
3     std::int64_t total = 0;
4     for (Money item : items) {
5         if (item.cents() > std::numeric_limits<std::int64_t>::max() - total) {
6             return std::nullopt;
7         }
8         total += item.cents();
9     }
10    return Money::from_cents(total);
11 }

```

这个例子不是让你每次加法都写这么长，而是训练一种意识：类型选了整数，不代表所有风险消失。建模、范围、运算和错误表达必须一起设计。对于金额，溢出通常应该是明确错误，而不是绕回负数或得到一个看似正常的总价。

4.12 单位也应该进入类型

类型设计不仅适用于金额。下面是一个常见事故：函数要求毫秒，调用者传了秒。

Listing 4.17 时间单位混淆

```

1 void set_timeout_ms(int timeout);
2
3 set_timeout_ms(5); // 是 5 毫秒，还是调用者以为的 5 秒？

```

标准库已经给了更好的词汇：

Listing 4.18 用 chrono 表达时间单位

```

1 void set_timeout(std::chrono::milliseconds timeout);
2
3 set_timeout(std::chrono::seconds{5});

```

调用者写 `std::chrono::seconds{5}`，接口收 `std::chrono::milliseconds`，转换关系由类型系统表达。这里的教训和 Money 一样：如果一个数字带单位，就尽量不要让它长期裸奔。裸整数越多，代码审查时越难看出错在哪里。

4.13 初始化形式背后的安全边界

大括号初始化拒绝窄化转换，这不是语法癖好，而是安全边界。看两个版本：

Listing 4.19 两个初始化版本的差别

```

1 int count = 3.14;
2 // int strict_count{3.14};

```

第一个版本把 3.14 截成 3，代码仍然能跑。第二个版本直接在编译期拒绝。学习阶段应该喜欢这种“早失败”的形式，因为它把 bug 从运行期提前到编译期。不是所有转换都禁止，但每次需要窄化时，都应该写出显式转换，让读者知道你承担了信息丢失。

4.14 本章验收：为每个数字写出身份

看见一个数字变量时，至少问四个问题：它的单位是什么，它的范围是什么，它是否允许缺失，它和哪些数字能运算。成绩、端口、毫秒、字节数、金额、比例、数组下标，看起来都能用整数，但它们不是同一种东西。现代 C++ 的类型系统足够强，真正的训练是学会把业务身份交给类型，而不是把所有含义塞进变量名和注释。

第 5 章

控制流、函数和错误路径

5.1 问题入口：命令行参数解析

控制流不是只会写 `if` 和 `for`。真正的训练是把正常路径和错误路径写清楚。假设程序支持：

Listing 5.1 命令行需求

```
1 tool --input data.txt --limit 100
```

朴素写法会在 `main` 里一边循环一边判断：

Listing 5.2 混乱的参数解析

```
1 for (int i = 1; i < argc; ++i) {
2     std::string arg = argv[i];
3     if (arg == "--input") {
4         input = argv[++i];
5     } else if (arg == "--limit") {
6         limit = std::stoi(argv[++i]);
7     }
8 }
```

这段代码漏了很多边界：`-input` 后面没有值会越界，`-limit abc` 会抛异常，重复参数如何处理也不明确。

5.2 先定义结果

参数解析的结果应该是一个类型：

Listing 5.3 参数解析结果

```
1 #include <optional>
2 #include <string>
3
4 struct Options {
5     std::string input_path;
6     int limit = 0;
7 };
8
9 struct ParseError {
10     std::string message;
11 };
```

标准库直到 C++23 才有 `std::expected`。为了保持 C++20，本章用一个简单结构表达“成功或失败”：

Listing 5.4 C++20 下的简单结果类型

```
1 struct ParseResult {
2     std::optional<Options> options;
```

```

3     std::optional<ParseError> error;
4 };

```

真实项目可以使用成熟的 `expected` 实现；教材这里先把控制流讲清楚。

5.3 让循环只做推进

解析函数可以按下标推进。每次遇到需要值的选项，都先检查后面是否还有参数。

Listing 5.5 明确错误路径的参数解析

```

1 #include <cstdlib>
2 #include <string_view>
3 #include <vector>
4
5 ParseResult parse_options(std::span<char* const> args)
6 {
7     Options options;
8     bool has_input = false;
9     bool has_limit = false;
10
11     for (std::size_t i = 1; i < args.size(); ++i) {
12         const std::string_view arg = args[i];
13         if (arg == "--input") {
14             if (i + 1 >= args.size()) {
15                 return {.error = ParseError{"--input requires a value"}};
16             }
17             options.input_path = args[++i];
18             has_input = true;
19             continue;
20         }
21         if (arg == "--limit") {
22             if (i + 1 >= args.size()) {
23                 return {.error = ParseError{"--limit requires a value"}};
24             }
25             const std::string value = args[++i];
26             try {
27                 options.limit = std::stoi(value);
28             } catch (const std::exception&) {
29                 return {.error = ParseError{"--limit must be an integer"}};
30             }
31             has_limit = true;
32             continue;
33         }
34         return {.error = ParseError{"unknown option"}};
35     }
36
37     if (!has_input) {
38         return {.error = ParseError{"missing --input"}};
39     }
40     if (!has_limit) {
41         return {.error = ParseError{"missing --limit"}};
42     }
43     return {.options = options};

```

```
44 }
```

这段代码不追求最短。它追求每个错误路径都能解释。工程代码里，错误消息是接口的一部分。

5.4 函数大小和责任边界

当解析规则继续增多，可以把“取下一个值”抽成辅助函数，把“字符串转整数”抽成辅助函数。拆函数不是为了好看，而是为了隔离可测试的决策点。

Listing 5.6 把字符串转整数变成独立函数

```
1 std::optional<int> parse_int(std::string_view text)
2 {
3     try {
4         std::size_t consumed = 0;
5         const int value = std::stoi(std::string{text}, &consumed);
6         if (consumed != text.size()) {
7             return std::nullopt;
8         }
9         return value;
10    } catch (const std::exception&) {
11        return std::nullopt;
12    }
13 }
```

这里有一个小成本：`std::stoi` 接受 `std::string`，所以我们从 `string_view` 构造了字符串。性能敏感场景可以用 `std::from_chars`。入门阶段先把正确性和接口讲清楚。

5.5 本章边界检查

控制流代码要重点检查：

- 选项缺值。
- 未知选项。
- 数字格式错误。
- 重复选项是否允许。
- 默认值是否真实表达业务需求。

核心思想

控制流的高级能力不是写复杂分支，而是让每条路径都有清晰含义。正常路径短，错误路径明确，状态推进单调，函数边界可测试。

5.6 从缺值反例推到下标推进规则

参数解析最容易错在 `++i`。看这个输入：

Listing 5.7 缺少参数值的输入

```
1 tool --input
```

如果代码直接写 `input = argv[++i]`，`i` 会越过最后一个参数。这个反例推出第一条规则：任何消耗下一个参数的选项，都必须先检查 `i + 1 < args.size()`。再看另一个输入：

Listing 5.8 未知选项不能静默忽略

```
1 tool --input data.txt --limit 100
```

`-limit` 是拼写错误。如果解析器静默忽略未知选项，程序可能用默认限制继续跑，结果比直接失败更危险。这个反例推出第二条规则：未知选项应该成为错误，除非需求明确允许透传。

5.7 让数字解析没有半截成功

`std::stoi("12abc")` 会解析出 12，如果不检查消费长度，就会把坏输入当成好输入。前面 `parse_int` 里检查 `consumed != text.size()`，就是为了排除这种半截成功。

在 C++20 中也可以用 `std::from_chars`，它不抛异常，也不受本地化影响：

Listing 5.9 用字符转换解析整数

```
1 #include <charconv>
2
3 std::optional<int> parse_int_fast(std::string_view text)
4 {
5     int value = 0;
6     const char* begin = text.data();
7     const char* end = text.data() + text.size();
8     const auto [ptr, ec] = std::from_chars(begin, end, value);
9     if (ec != std::errc{} || ptr != end) {
10         return std::nullopt;
11     }
12     return value;
13 }
```

这个版本更适合底层解析。它的接口仍然返回 `std::optional<int>`，因为调用者关心的是“是否完整解析成功”，而不是底层用了异常还是错误码。

5.8 把主流程写成可读句子

解析函数写好后，`main` 不应该继续塞满细节。入口可以保持很薄：

Listing 5.10 入口只负责连接步骤

```
1 int run(std::span<char* const> args)
2 {
3     const ParseResult parsed = parse_options(args);
4     if (parsed.error.has_value()) {
5         std::cerr << parsed.error->message << '\n';
6         return 2;
7     }
8
9     return run_with_options(*parsed.options);
10 }
```

这里有一个隐含要求：`ParseResult` 不能同时既有 `options` 又有 `error`。更严谨的项目会用 `expected` 或自定义变体类型表达二选一。教材这里先用简单结构，是为了把控制流看清楚；真正工程里可以进一步收紧类型。

习题与作业

给参数解析器补上重复选项规则：`-limit 10 -limit 20` 是报错，还是后者覆盖前者？先写出你的规则，再修改 `has_limit` 分支，让测试覆盖这两种输入。

5.9 把参数解析看成状态机

命令行解析最容易写成一堆互相穿插的 `if`。CPU 卷处理复杂控制流时会先把状态列出来，这里也一样。对 `tool -input data.txt -limit 100` 来说，解析器的状态可以很小：

当前看到	需要检查	下一步
<code>-input</code>	后面是否还有参数	消耗下一个参数作为路径
<code>-limit</code>	后面是否还有参数，是否完整整数	消耗下一个参数作为限制数
未知选项	是否允许透传	本工具不允许，直接报错
普通文本	是否处在等待值状态	本工具不接受裸参数，报错
扫描结束	必填项是否齐全	缺什么报什么

有了这张表，循环怎么写就清楚了：下标只向右推进；每次消耗值都先检查边界；遇到错误立即返回；循环结束后统一检查必填项。控制流的质量不在于用了多少语法，而在于状态推进是否单调、失败位置是否可解释。

5.10 为什么要避免半截解析

数字解析有一个隐藏坑：`12abc` 不应该被当成 `12`。如果工具用于截断日志、限制扫描数量或控制删除范围，半截成功可能造成严重后果。下面的测试应当失败：

Listing 5.11 半截数字应当报错

```
1 assert(!parse_int_fast("12abc").has_value());
2 assert(!parse_int_fast("").has_value());
3 assert(parse_int_fast("12").value() == 12);
```

这个测试矩阵推导出实现必须同时检查两个条件：转换错误码为成功，返回指针正好到达字符串末尾。只检查错误码不够；只检查非空也不够。控制流里很多 bug 都来自这种“少检查一个证据字段”。

5.11 重复参数是一条业务规则

重复选项没有天然答案。不同工具可能选择“后者覆盖前者”“报错”“收集成列表”。教材里选择报错，是为了让用户更早发现命令写错：

Listing 5.12 重复选项样本

```
1 tool --input a.txt --input b.txt --limit 10
```

如果静默取后者，用户可能以为两个文件都被处理。报错版本可以这样写：

Listing 5.13 重复选项检查

```
1 if (arg == "--input") {
2     if (has_input) {
3         return {error = ParseError{"duplicate --input"}};
4     }
5     if (i + 1 >= args.size()) {
6         return {error = ParseError{"--input requires a value"}};
7     }
8     options.input_path = args[++i];
9     has_input = true;
10    continue;
```

```
11 }
```

注意这不是模板，而是从需求推导出来的规则。如果以后要支持多个输入文件，接口就不应该叫 `input_path`，而应该变成 `input_paths`，类型也应从 `std::string` 变成 `std::vector<std::string>`。状态变量和数据结构要跟业务合同一起变化。

5.12 错误消息也是用户界面

“unknown option” 对程序员可能够用，对用户不够。更有用的错误应该包含原始参数和期望格式：

Listing 5.14 带上下文的未知选项错误

```
1 return {.error = ParseError{
2     .message = "unknown option: " + std::string{arg}
3 }};
```

如果工具有很多选项，错误还可以补上 `try -help`。错误消息的标准是：用户看到后能不能立刻改输入。不要只写 `parse failed`，它没有定位价值。

5.13 从 run 函数建立测试边界

一个命令行工具至少有三层：解析参数、执行业务、连接终端。测试解析器时，不需要真的跑业务；测试业务时，不需要从 `argv` 解析；测试入口时，只验证错误码和输出流连接。

Listing 5.15 入口函数适合测试

```
1 int run(std::span<char* const> argv,
2         std::ostream& output,
3         std::ostream& error)
4 {
5     const ParseResult parsed = parse_options(argv);
6     if (parsed.error.has_value()) {
7         error << parsed.error->message << '\n';
8         return 2;
9     }
10
11     return execute(*parsed.options, output, error);
12 }
```

这段代码看起来比直接在 `main` 里写多了一层，但测试收益很大。你可以构造假参数数组，传入 `std::ostringstream`，断言返回码和错误文本。真正的 `main` 只负责把系统边界转接进来。

5.14 验收练习：为解析器画一张失败表

完成一个解析器后，不要只测成功命令。至少列出失败表：

输入	应报告
<code>tool</code>	缺少 <code>-input</code> 或 <code>-limit</code>
<code>tool -input</code>	<code>-input</code> 缺少值
<code>tool -limit abc</code>	限制必须是整数
<code>tool -limit 12abc</code>	限制必须完整解析
<code>tool -input a -input b</code>	重复输入参数
<code>tool -limt 10</code>	未知选项，提示拼写错误

能把失败表写出来，控制流才算有边界。否则程序只是碰巧处理了几个样例。

第 6 章

类、封装和不变量

6.1 问题入口：温度范围

类不是把几个变量包起来。类的价值在于维护不变量。不变量（invariant）指对象在任何对外可见状态下都必须成立的条件。假设要表示一个温度区间，要求最低温不能高于最高温。

朴素结构体写法如下：

Listing 6.1 没有保护不变量的结构体

```
1 struct TemperatureRange {
2     double low = 0.0;
3     double high = 0.0;
4 };
5
6 TemperatureRange range{.low = 30.0, .high = 10.0}; // 逻辑错误
```

编译器不知道业务规则，所以这段代码能编译。要把规则放进类型里。

6.2 构造函数建立有效状态

Listing 6.2 构造函数检查不变量

```
1 #include <stdexcept>
2
3 class TemperatureRange {
4 public:
5     TemperatureRange(double low, double high)
6         : low_{low}, high_{high}
7     {
8         if (high < low) {
9             throw std::invalid_argument{"high must not be lower than low"};
10        }
11    }
12
13    [[nodiscard]] double low() const noexcept
14    {
15        return this->low_;
16    }
17
18    [[nodiscard]] double high() const noexcept
19    {
20        return this->high_;
21    }
22
23    [[nodiscard]] bool contains(double value) const noexcept
24    {
25        return this->low_ <= value && value <= this->high_;
26    }
```

```

27
28 private:
29     double low_ = 0.0;
30     double high_ = 0.0;
31 };

```

数据成员是私有的，调用者不能绕过构造函数随意改。成员函数里使用 `this->`，目的是把“当前对象的成员”写清楚。

6.3 setter 不一定是封装

很多人把封装理解成“成员私有，再给每个成员写 getter 和 setter”。这只是形式。如果写出下面的接口，不变量又被破坏了：

Listing 6.3 危险的 setter

```

1 void set_low(double value)
2 {
3     this->low_ = value; // 可能让 low_ > high_
4 }

```

更好的接口应该表达业务动作。例如扩展范围：

Listing 6.4 用业务动作维护不变量

```

1 void expand_to_include(double value) noexcept
2 {
3     if (value < this->low_) {
4         this->low_ = value;
5     }
6     if (value > this->high_) {
7         this->high_ = value;
8     }
9 }

```

调用者不需要知道内部如何保存，只知道对象会把某个值纳入范围。

6.4 值语义优先

`TemperatureRange` 可以复制、赋值、放进 `std::vector`。这是值语义。值语义让对象像数字和字符串一样好用。

Listing 6.5 值语义让对象易组合

```

1 std::vector<TemperatureRange> ranges;
2 ranges.emplace_back(10.0, 20.0);
3 ranges.emplace_back(-5.0, 8.0);

```

不要过早把所有类都设计成继承体系。许多业务概念更适合小而明确的值对象。

6.5 本章边界检查

设计类时至少问：

- 对象什么时候有效？
- 哪些状态绝对不允许出现？

- 构造函数能否一次建立有效状态？
- 对外接口是业务动作，还是裸字段修改？
- 对象是否应该支持复制和移动？

核心思想

类的核心不是语法里的 `class`，而是把数据和规则放在同一个边界内。好的类让错误状态难以构造，让正确用法自然出现。

6.6 从坏对象推导构造边界

继续看温度区间。为什么不能让调用者先默认构造，再分别设置上下界？因为中间状态可能已经被观察到：

Listing 6.6 分步设置制造临时坏状态

```
1 TemperatureRange range;
2 range.set_high(10.0);
3 range.set_low(30.0); // 如果允许，就破坏 low <= high
4 if (range.contains(20.0)) {
5     // 这里的判断已经没有可信含义
6 }
```

这个反例推出一条类设计规则：如果一个对象必须同时拥有几项数据才有效，就让构造函数一次收齐这些数据，并在构造边界检查。不应该把“不完整但暂时别用”的对象暴露给普通调用者。

6.7 成员函数应该维护对象语义

假设业务需要把两个温度区间合并。不要把内部字段拿出来让调用者自己算，而是提供表达业务的函数：

Listing 6.7 合并两个有效区间

```
1 TemperatureRange merge(TemperatureRange lhs, TemperatureRange rhs)
2 {
3     const double low = std::min(lhs.low(), rhs.low());
4     const double high = std::max(lhs.high(), rhs.high());
5     return TemperatureRange{low, high};
6 }
```

因为 `lhs` 和 `rhs` 都已经是有有效对象，`min(low)` 和 `max(high)` 构造出来的新对象也应当有效。这里的推理链条很重要：类不变性让函数可以建立在更少的防御性检查上。

6.8 什么时候用 `struct`，什么时候用 `class`

C++里 `struct` 和 `class` 的主要语法差异是默认访问权限；真正的设计差异来自意图。只承载数据、没有复杂不变量的聚合，适合 `struct`：

Listing 6.8 简单结果适合 `struct`

```
1 struct ScoreSummary {
2     std::size_t count = 0;
3     int sum = 0;
4     double average = 0.0;
5 };
```

需要保护规则、隐藏表示、控制修改路径的概念，适合 `class`。这不是硬性宗教，而是让读者一看到类型形态，就能大致知道它的使用方式。

6.9 类的测试不是只测 `getter`

测试 `TemperatureRange` 时，不要只检查 `low()` 和 `high()` 返回原值。更重要的是不变量：

Listing 6.9 围绕不变量测试类

```
1 const TemperatureRange normal{10.0, 20.0};
2 assert(normal.contains(10.0));
3 assert(normal.contains(15.0));
4 assert(!normal.contains(21.0));
5
6 bool rejected = false;
7 try {
8     const TemperatureRange invalid{30.0, 10.0};
9 } catch (const std::invalid_argument&) {
10     rejected = true;
11 }
12 assert(rejected);
```

如果一个类的测试只是在验证字段保存和取出，说明这个类可能没有承担真正职责；它也许只是一个 `struct`，或者需要重新思考业务动作。

习题与作业

实现一个 `Date` 类，要求月份在 1 到 12，日期必须符合月份天数。先不处理闰年，再把闰年规则加进去。练习重点不是背日期算法，而是把“不允许出现的状态”挡在构造函数外。

6.10 `Date` 案例：从非法状态推导类

温度区间只有两个字段，规则也简单。日期更接近真实业务：年、月、日三个字段一起决定对象是否有效。朴素结构体很容易制造坏对象：

Listing 6.10 日期坏对象

```
1 struct DateData {
2     int year = 0;
3     int month = 0;
4     int day = 0;
5 };
6
7 DateData impossible{.year = 2026, .month = 2, .day = 31};
```

编译器不知道二月没有三十一日。于是日期不能长期只靠裸 `struct` 表达。先写规则函数：

Listing 6.11 日期规则函数

```
1 constexpr bool is_leap_year(int year) noexcept
2 {
3     return (year % 400 == 0) ||
4            (year % 4 == 0 && year % 100 != 0);
5 }
6
```

```

7 constexpr int days_in_month(int year, int month) noexcept
8 {
9     constexpr int DAYS_IN_MONTH[] = {
10         0, 31, 28, 31, 30, 31, 30,
11         31, 31, 30, 31, 30, 31,
12     };
13     if (month == 2 && is_leap_year(year)) {
14         return 29;
15     }
16     return DAYS_IN_MONTH[month];
17 }

```

规则函数单独存在，类负责调用它建立有效状态：

Listing 6.12 Date 构造边界

```

1 class Date {
2 public:
3     Date(int year, int month, int day)
4         : year_{year}, month_{month}, day_{day}
5     {
6         if (month < 1 || month > 12) {
7             throw std::invalid_argument{"month must be 1..12"};
8         }
9         if (day < 1 || day > days_in_month(year, month)) {
10             throw std::invalid_argument{"day is invalid for month"};
11         }
12     }
13
14     [[nodiscard]] int year() const noexcept { return this->year_; }
15     [[nodiscard]] int month() const noexcept { return this->month_; }
16     [[nodiscard]] int day() const noexcept { return this->day_; }
17
18 private:
19     int year_ = 0;
20     int month_ = 1;
21     int day_ = 1;
22 };

```

这个类没有 setter。不是因为 setter 永远不好，而是因为单独改月份或日期可能瞬间破坏不变量。更合适的接口是业务动作，例如 `next_day()` 或 `add_days()`，由类内部维护规则。

6.11 类背后的普通对象模型

类不是魔法。可以把它理解成“数据字段加一组只能通过对象调用的函数”。下面的成员函数：

Listing 6.13 成员函数

```

1 [[nodiscard]] bool contains(double value) const noexcept
2 {
3     return this->low_ <= value && value <= this->high_;
4 }

```

近似等价于一个接收当前对象地址的普通函数：

Listing 6.14 成员函数的近似展开

```

1 bool TemperatureRange_contains(const TemperatureRange* self,
2                                double value) noexcept
3 {
4     return self->low_ <= value && value <= self->high_;
5 }

```

真实 `private` 成员不能被外部普通函数访问，这里只是帮助理解：`this` 就是当前对象的入口。构造函数则是在对象创建时运行的一段初始化代码，用来把字段放到有效状态。析构函数是在对象离开生命周期时运行的清理代码。用这个普通代码模型看类，封装和生命周期会更具体。

6.12 不变量让调用者少做防御

如果 `Date` 保证自己永远有效，那么格式化函数就不需要每次重新检查月份范围：

Listing 6.15 建立在不变量上的函数

```

1 std::string format_date(const Date& date)
2 {
3     return std::format("{:04}-{:02}-{:02}",
4                        date.year(),
5                        date.month(),
6                        date.day());
7 }

```

这个函数可以信任 `date.month()` 在 `1..12`，因为坏对象无法构造出来。不变量的价值就在这里：检查集中在边界，内部逻辑不必到处防御。

6.13 测试 `Date` 的规则矩阵

日期测试要覆盖规则，而不是只测 `getter`：

场景	输入	期望
普通日期	2026, 6, 27	构造成功
月份过小	2026, 0, 1	拒绝
月份过大	2026, 13, 1	拒绝
普通二月	2026, 2, 29	拒绝
闰年二月	2024, 2, 29	构造成功
日期为零	2026, 1, 0	拒绝

有了这个矩阵，类的职责就很清楚：任何公开可见的 `Date` 对象都必须是真实日历日期。

第 7 章

引用、指针和非拥有视图

7.1 问题入口：函数到底改没改原对象

很多 C++ 初学者第一次写函数时，会遇到一个反直觉问题：

Listing 7.1 按值传参不会修改原对象

```
1 void add_bonus(int score)
2 {
3     score += 5;
4 }
5
6 int main()
7 {
8     int value = 90;
9     add_bonus(value);
10    std::cout << value << '\n'; // 仍然是 90
11 }
```

这段代码没有错。函数参数 `score` 是一个新对象，它从 `value` 复制得到。修改 `score` 只会修改副本。这个反例推出第一条规则：看到函数参数时，先判断它是在复制对象、引用对象，还是通过指针间接访问对象。

如果函数确实要修改调用者传入的对象，可以用引用：

Listing 7.2 非 `const` 引用表达可修改

```
1 void add_bonus(int& score)
2 {
3     score += 5;
4 }
```

`int&` 不是一个新的整数对象，而是已有整数对象的别名。调用 `add_bonus(value)` 后，函数内部的 `score` 和外部的 `value` 指向同一个对象。

7.2 引用的核心：必须绑定到有效对象

引用一旦创建，就必须绑定到某个对象。它不能“空着”，也不能随便改绑到另一个对象：

Listing 7.3 引用绑定后就是别名

```
1 int first = 1;
2 int second = 2;
3 int& ref = first;
4 ref = second; // 不是改绑，而是把 first 改成 2
```

执行后 `first` 变成 2，`ref` 仍然是 `first` 的别名。这个细节很重要，因为它解释了为什么引用适合表达“这个参数一定存在”。如果对象可能不存在，引用就不是合适表达。

7.3 指针表达可空和可重新指向

指针保存对象地址，可以为空，也可以重新指向别的对象：

Listing 7.4 指针可以表示可选对象

```
1 void print_score(const int* score)
2 {
3     if (score == nullptr) {
4         std::cout << "missing\n";
5         return;
6     }
7     std::cout << *score << '\n';
8 }
```

`*score` 表示访问指针指向的对象。访问前必须确认它不是 `nullptr`。这推出第二条规则：接口如果用裸指针表达观察关系，就必须清楚说明它是否允许为空。允许为空时，函数内部要检查；不允许为空时，优先考虑引用。

7.4 不要返回局部对象的地址

指针和引用都不拥有对象，所以最常见错误是指向已经销毁的局部变量：

Listing 7.5 返回局部变量地址是错误

```
1 const int* bad_pointer()
2 {
3     int value = 42;
4     return &value; // 函数结束后 value 已经销毁
5 }
```

函数返回后，`value` 的生命周期结束。调用者拿到的地址已经悬空。正确做法通常是返回值：

Listing 7.6 返回值让调用者拥有结果

```
1 int make_value()
2 {
3     return 42;
4 }
```

现代 C++ 返回值并不意味着低效。编译器会进行返回值优化，必要时也可以移动对象。为了省一次可能不存在的复制而返回悬空引用，是非常不划算的。

7.5 span：一段连续数据的非拥有视图

当函数需要观察数组、`vector` 或 `array` 的一段连续元素时，`std::span` 比裸指针加长度更清楚：

Listing 7.7 用 `span` 表达连续区间

```
1 int sum_scores(std::span<const int> scores)
2 {
3     int total = 0;
4     for (int score : scores) {
5         total += score;
6     }
7     return total;
8 }
```

```
8 }
```

`std::span<const int>` 表达三件事：它不拥有元素；它只读；它知道长度。调用者可以传 `std::vector<int>`、`std::array<int, N>` 或普通数组。函数内部不用同时维护 `int* data` 和 `std::size_t size` 两个参数，也减少了长度不匹配的风险。

7.6 string_view：字符串内容的非拥有视图

`std::string_view` 类似 `span<const char>`，但专门表示字符串内容。它适合只观察文本：

Listing 7.8 字符串视图适合只读参数

```
1 bool starts_with(std::string_view text, std::string_view prefix)
2 {
3     return text.size() >= prefix.size() &&
4           text.substr(0, prefix.size()) == prefix;
5 }
```

它不分配内存，也不要求调用者一定传 `std::string`。字符串字面量、`std::string` 和子串都能传入。但它不拥有内容，所以不能长期保存一个指向临时字符串的 `string_view`：

Listing 7.9 保存临时字符串视图会悬空

```
1 std::string_view bad_view()
2 {
3     std::string text = "temporary";
4     return std::string_view{text}; // text 离开函数后销毁
5 }
```

非拥有视图的好处是轻，代价是生命周期要求更严格。只要你把视图保存到函数调用之后，就必须能证明底层对象活得更久。

7.7 接口选择表

意图	推荐形态	说明
复制一个便宜值	<code>int value</code>	标量和小对象可以按值
只观察一个大对象	<code>const T&</code>	不复制，不修改
修改一个必定存在的对象	<code>T&</code>	调用点能看出会修改
观察可能不存在的对象	<code>const T*</code>	<code>nullptr</code> 表示没有
接管独占所有权	<code>std::unique_ptr<T></code>	调用点需要 <code>std::move</code>
观察连续数据	<code>std::span<const T></code>	不拥有，带长度
观察文本	<code>std::string_view</code>	不拥有字符内容

7.8 完整案例：校验用户输入

假设要校验用户名：不能为空，只能包含字母、数字和下划线，长度不能超过 32。这个函数只观察文本，不保存文本，所以用 `std::string_view`：

Listing 7.10 用户名校验

```
1 #include <cctype>
2 #include <string_view>
3
4 constexpr std::size_t USERNAME_MAX_LENGTH = 32;
```

```

5
6 bool is_valid_username(std::string_view name)
7 {
8     if (name.empty() || name.size() > USERNAME_MAX_LENGTH) {
9         return false;
10    }
11    for (unsigned char ch : name) {
12        if (std::isalnum(ch) || ch == '_') {
13            continue;
14        }
15        return false;
16    }
17    return true;
18 }

```

这里 `USERNAME_MAX_LENGTH` 用常量命名，而不是把 32 散在代码里。循环里用 `unsigned char` 是因为 `std::isalnum` 对普通 `char` 的行为不安全。一个小函数也能体现类型、生命周期、常量和边界处理。

习题与作业

实现 `trim(std::string_view text)`，返回去掉首尾空白后的 `std::string_view`。要求不分配新字符串。写测试覆盖空字符串、全空白、无空白、只有左空白、只有右空白。然后回答：返回的视图依赖谁的生命周期？

7.9 生命周期事故账本

引用、指针和视图之所以难，不是因为符号多，而是因为它们把“访问入口”和“对象生命”分开了。CPU 卷会为并发事件列事件链；这里也给生命周期列账本。看这个函数：

Listing 7.11 返回悬空字符串视图

```

1 std::string_view make_label()
2 {
3     std::string text = "score";
4     return std::string_view{text};
5 }

```

事件链如下：

时刻	发生了什么	证据
进入函数	创建局部 <code>text</code>	<code>std::string</code> 拥有字符缓冲区
返回语句	创建 <code>string_view</code>	视图只保存地址和长度，不拥有字符
函数结束	<code>text</code> 析构	字符缓冲区被释放
调用者使用返回 值	视图仍保存旧地址	地址已经不再指向有效字符串对象

这个表比一句“不要返回局部变量视图”更重要。它告诉你错误发生在哪个边界：返回值本身还活着，但它借看的底层对象死了。以后判断 `std::span`、`std::string_view`、裸指针、引用时，都按这个账本走。

7.10 接口身份：观察、修改、拥有、可缺失

同一个 `User` 对象可以有四种完全不同的接口：

Listing 7.12 同一个对象的四重接口身份

```

1 void print_user(const User& user);
2 void rename_user(User& user, std::string new_name);
3 void store_user(std::unique_ptr<User> user);
4 void maybe_print_user(const User* user);

```

`print_user` 要求对象必定存在，只观察；`rename_user` 要求对象必定存在，并会修改；`store_user` 接管所有权；`maybe_print_user` 允许没有对象。函数签名就是调用合同。不要把这四种关系都写成 `User*`，否则调用者只能靠注释猜：能不能为空？会不会删除？会不会保存？

一个常见坏接口如下：

Listing 7.13 含义不清的裸指针接口

```

1 void process(User* user);

```

它至少有四种解释：可能为空、必定不为空、只观察、会修改、甚至可能保存指针到以后用。这样的接口会把风险推给调用者。改接口时先说清楚身份，再选参数形态。

7.11 视图不能越过拥有者的改变

`std::span` 和 `std::string_view` 还会被“底层容器改变”影响。看这个例子：

Listing 7.14 `vector` 扩容后 `span` 可能失效

```

1 std::vector<int> values{1, 2, 3};
2 std::span<int> view{values};
3 values.push_back(4); // 可能扩容
4 view[0] = 10;        // 如果扩容发生，view 可能悬空

```

问题不在 `span` 自己。它只是忠实保存了原来那段连续内存的地址和长度。真正改变的是 `vector`：尾部追加时如果容量不足，`vector` 会申请新内存，把元素搬过去，释放旧内存。旧地址不再属于当前元素，视图就失效了。

所以视图适合短时间传参，不适合长期保存。尤其不要在类成员里保存 `std::span` 或 `std::string_view`，除非类的文档和构造方式能严格保证底层对象活得更久，并且不会发生破坏地址稳定性的修改。

7.12 完整案例：不分配的 trim

前面的练习现在展开讲。需求是去掉字符串首尾空白，不分配新字符串，返回原文本中的一段视图。先写合同：

- 输入视图只被观察，不被修改。
- 返回视图借用同一段底层字符。
- 空字符串和全空白字符串都返回空视图。
- 调用者必须保证原字符串在返回视图使用期间仍然有效。

实现如下：

Listing 7.15 返回子串视图的 trim

```

1 bool is_space(unsigned char ch)
2 {
3     return std::isspace(ch) != 0;
4 }
5

```

```

6 std::string_view trim(std::string_view text)
7 {
8     std::size_t first = 0;
9     while (first < text.size() &&
10           is_space(static_cast<unsigned char>(text[first]))) {
11         ++first;
12     }
13
14     std::size_t last = text.size();
15     while (last > first &&
16           is_space(static_cast<unsigned char>(text[last - 1]))) {
17         --last;
18     }
19
20     return text.substr(first, last - first);
21 }

```

为什么这里不用 `std::string` 返回？因为需求明确“不分配”，且返回内容是输入的一段。为什么 `is_space` 接收 `unsigned char`？因为字符分类函数对负的 `char` 值不安全。一个小工具函数，仍然能体现生命周期、类型和边界。

7.13 trim 的反例测试

这个函数的测试不应该只有 “`abc`”。至少要覆盖：

Listing 7.16 trim 的边界测试

```

1 assert(trim("") == "");
2 assert(trim(" ") == "");
3 assert(trim("abc") == "abc");
4 assert(trim(" abc") == "abc");
5 assert(trim("abc ") == "abc");
6 assert(trim(" abc def ") == "abc def");

```

再写一个不能做的调用：

Listing 7.17 不要保存临时字符串的 trim 结果

```

1 std::string_view bad = trim(std::string{" Ada "});
2 // 这一行之后临时 string 已销毁，bad 悬空

```

看到这个反例，就能理解为什么“函数返回 `string_view`”不是永远好。它只在调用者能管理底层生命周期时好。性能和安全从来不是分开的两本账。

第 8 章

表达式、作用域和常量

8.1 问题入口：一行表达式为什么读不懂

初学者经常写出能编译但难读的表达式：

Listing 8.1 压缩过度的表达式

```
1 if (i++ < values.size() && values[i] > limit || force) {  
2     process(values[i]);  
3 }
```

这行代码的问题不只是运算符优先级。它同时修改 `i`、访问容器、判断开关，还混合了 `&&` 和 `||`。读者必须在脑子里模拟执行顺序。更严重的是，`i++` 后再访问 `values[i]`，访问的是递增后的下标，可能不是作者想要的。

第一条规则：表达式不是越短越好。只要一行表达式同时做“修改状态”和“判断业务”，就值得拆开。

8.2 副作用要显式

副作用指表达式除了计算值，还改变了某些状态。自增、赋值、函数调用都可能有副作用。把副作用拆出来：

Listing 8.2 拆开状态推进和判断

```
1 const std::size_t current = i;  
2 ++i;  
3  
4 const bool in_range = current < values.size();  
5 const bool accepted = in_range && values[current] > limit;  
6 if (accepted || force) {  
7     process(values[current]);  
8 }
```

这个版本更长，但每一步都能命名。命名不是废话，它把中间含义固定下来，也让调试器里能直接观察。

8.3 作用域限制变量生命

变量作用域越大，读者越难判断它在哪里被修改。先看反例：

Listing 8.3 变量作用域过大

```
1 int result = 0;  
2 // 很多代码  
3 for (int value : values) {  
4     result += value;  
5 }  
6 // 又有很多代码使用 result
```

如果 `result` 只用于求和后立即返回，就让它待在小作用域里：

Listing 8.4 把变量限制在使用附近

```

1 int sum_values(std::span<const int> values)
2 {
3     int result = 0;
4     for (int value : values) {
5         result += value;
6     }
7     return result;
8 }

```

函数本身也是作用域工具。把独立计算抽成函数，变量生命周期自然缩短，错误传播范围也缩小。

8.4 const 和 constexpr

`const` 表示对象初始化后不能通过这个名字修改：

Listing 8.5 `const` 表达不再修改

```

1 const int max_retry_count = 3;

```

`constexpr` 表示值可以在编译期求出：

Listing 8.6 编译期常量

```

1 constexpr int PORT_MIN = 1;
2 constexpr int PORT_MAX = 65535;

```

两者不是同义词。运行时读到的配置值可以是 `const`，但不是 `constexpr`：

Listing 8.7 运行时 `const`

```

1 const int port = read_port_from_config();

```

它不能被修改，但它的值要运行时才知道。理解这个区别，后面学习数组大小、模板参数和编译期计算会轻松很多。

8.5 enum class 防止常量混用

裸整数常量容易混：

Listing 8.8 裸整数状态不清楚

```

1 constexpr int STATE_OPEN = 1;
2 constexpr int COLOR_RED = 1;
3
4 int state = COLOR_RED; // 编译器无法阻止

```

用 `enum class`：

Listing 8.9 强类型枚举

```

1 enum class ConnectionState {
2     closed,
3     connecting,
4     open,

```

```

5 };
6
7 enum class Color {
8     red,
9     green,
10 };

```

`ConnectionState` 和 `Color` 是不同类型，不能随便互相赋值。类型系统替你挡住一类低级错误。

8.6 switch 要处理完整状态

枚举常配合 `switch`：

Listing 8.10 处理连接状态

```

1 std::string_view describe(ConnectionState state)
2 {
3     switch (state) {
4         case ConnectionState::closed:
5             return "closed";
6         case ConnectionState::connecting:
7             return "connecting";
8         case ConnectionState::open:
9             return "open";
10    }
11    return "unknown";
12 }

```

这里保留最后的 `return` 是为了满足编译器路径分析。开启警告后，如果以后给枚举增加新状态却忘了处理，编译器通常会提醒。不要随便写 `default` 把所有遗漏吞掉；对封闭枚举，显式列出每个状态更利于维护。

8.7 完整案例：重试策略

实现一个重试策略：最多重试三次，只对临时错误重试。

Listing 8.11 重试策略

```

1 enum class ErrorKind {
2     none,
3     temporary_failure,
4     permanent_failure,
5 };
6
7 constexpr int RETRY_MAX_ATTEMPTS = 3;
8
9 bool should_retry(ErrorKind error, int attempts)
10 {
11     if (error != ErrorKind::temporary_failure) {
12         return false;
13     }
14     return attempts < RETRY_MAX_ATTEMPTS;
15 }

```


这个小函数看起来简单，但它体现了本章规则：错误类型用枚举表达，魔法数字变成常量，表达式拆成业务判断。测试也很清楚：无错误不重试，永久错误不重试，临时错误在次数内重试，达到上限不重试。

习题与作业

把一个复杂 `if` 条件拆成三个具名布尔变量。要求变量名表达业务含义，而不是 `a`、`b`、`flag1`。然后给每个条件组合写测试，确认拆分没有改变行为。

8.8 表达式要能被审查

复杂表达式最大的问题是审查困难。看一个文件过滤条件：

Listing 8.12 难以审查的文件过滤条件

```
1 if (!path.empty() && path.extension() == ".log" && size > 0 &&
2     size < max_size || include_empty && path.extension() == ".txt") {
3     process(path);
4 }
```

读者必须同时记住优先级：`&&` 高于 `||`。但业务上真正想表达的可能是：处理非空日志文件，或者在允许空文件时处理文本文件。把它拆成名字：

Listing 8.13 用名字表达业务条件

```
1 const bool is_log_file = path.extension() == ".log";
2 const bool is_text_file = path.extension() == ".txt";
3 const bool has_allowed_size = size > 0 && size < max_size;
4 const bool can_process_log = is_log_file && has_allowed_size;
5 const bool can_process_empty_text = include_empty && is_text_file;
6
7 if (!path.empty() && (can_process_log || can_process_empty_text)) {
8     process(path);
9 }
```

拆开后，审查者能逐个检查业务名是否对应实现。括号也让组合关系不依赖读者背优先级。高质量表达式不是最短，而是能被别人可靠审查。

8.9 短路求值不能隐藏副作用

`&&` 和 `||` 有短路求值：左边已经决定结果时，右边不会执行。这对空指针检查很有用：

Listing 8.14 短路保护指针解引用

```
1 if (user != nullptr && user->active) {
2     send_message(*user);
3 }
```

但不要把副作用藏在右侧：

Listing 8.15 短路隐藏副作用

```
1 if (ready || load_cache()) {
2     use_cache();
3 }
```

这里 `load_cache()` 是否执行取决于 `ready`。如果它只是查询还好，如果它会修改全局缓存或写日志，读者很难从条件表达式看出状态变化。更清楚的写法是先显式处理：

Listing 8.16 把副作用变成步骤

```
1 if (!ready) {
2     ready = load_cache();
3 }
4 if (ready) {
5     use_cache();
6 }
```

这就是本章的主线：会改变状态的动作应当像步骤，而不是藏在条件里。

8.10 常量命名要表达领域

常量不是把数字换成大写就结束。比较两个名字：

Listing 8.17 弱常量名

```
1 constexpr int MAX = 3;
```

Listing 8.18 带领域的常量名

```
1 constexpr int RETRY_MAX_ATTEMPTS = 3;
```

第二个名字告诉读者这是重试次数上限，不是数组长度、端口数量或线程数。大型项目里常量很多，名字必须带上下文。对于本书里的练习，也要从一开始养成这个习惯。

8.11 switch 的遗漏要让编译器发现

前面说不要随便写 `default`。再展开一点。假设枚举新增了状态：

Listing 8.19 新增状态

```
1 enum class ConnectionState {
2     closed,
3     connecting,
4     open,
5     failed,
6 };
```

如果原来的 `switch` 没有 `default`，并开启警告，编译器更容易提醒 `failed` 没处理。如果你写了 `default: return "unknown";`，新增状态可能悄悄走默认路径，测试没覆盖就漏掉。对封闭枚举，显式处理所有枚举值通常更好；对真正来自外部的不可信整数，才需要默认错误路径。

8.12 重试策略推导到测试

重试策略的测试可以直接从业务表推导：

错误	已尝试次数	上限	是否重试
<code>none</code>	0	3	否
<code>permanent_failure</code>	0	3	否
<code>temporary_failure</code>	0	3	是
<code>temporary_failure</code>	2	3	是

<code>temporary_failure</code>	3	3	否
--------------------------------	---	---	---

这个表比“测一下 `should_retry`”更具体。它让你看到边界在 `attempts < RETRY_MAX_ATTEMPTS`，不是 `<=`。很多 off-by-one 错误都能通过这种表提前发现。

8.13 本章验收：减少读者脑内执行

表达式、作用域和常量的共同目标，是减少读者在脑子里模拟程序的负担。变量靠近使用点，状态改变单独成句，业务条件有名字，数字有身份，枚举状态被穷尽处理。做到这些，代码才容易审查、调试和修改。

第 9 章

函数重载、命名空间和头文件边界

9.1 问题入口：同名函数为什么有时可以，有时不行

C++ 允许函数重载：同一个名字可以有不同参数列表。

Listing 9.1 函数重载

```
1 int area(int side)
2 {
3     return side * side;
4 }
5
6 int area(int width, int height)
7 {
8     return width * height;
9 }
```

调用 `area(5)` 时选择第一个，调用 `area(5, 3)` 时选择第二个。重载的价值是让同一个概念共享名字。但如果两个函数语义不同，就不要硬用同名。名字相同应该意味着读者可以用同一种心智模型理解它们。

9.2 重载解析从参数类型开始

看这个例子：

Listing 9.2 重载可能产生歧义

```
1 void print(long value);
2 void print(double value);
3
4 print(1);
```

`1` 是 `int`，既可以转换成 `long`，也可以转换成 `double`。编译器要根据转换等级选择候选；有时会报歧义。初学阶段不要设计过于相近的重载，尤其是整数、浮点、布尔和指针混在一起时。

9.3 默认参数和重载不要打架

下面的接口看起来方便，实际可能制造歧义：

Listing 9.3 默认参数和重载混用风险

```
1 void connect(std::string_view host, int port = 80);
2 void connect(std::string_view host);
```

调用 `connect("example.com")` 时，两者都像是可行候选。更好的做法是只保留一个清楚接口，或者用配置对象表达可选项：

Listing 9.4 用选项对象表达默认值

```
1 struct ConnectOptions {
2     std::string host;
```

```

3     int port = 80;
4     bool use_tls = false;
5 };
6
7 void connect(const ConnectOptions& options);

```

当参数越来越多时，选项对象比一串默认参数更可读，也更容易扩展。

9.4 命名空间控制名字范围

大型项目不能把所有名字放在全局命名空间。命名空间给名字加上归属：

Listing 9.5 命名空间组织模块

```

1 namespace score {
2
3 struct Summary {
4     std::size_t count = 0;
5     int sum = 0;
6 };
7
8 Summary summarize(std::span<const int> values);
9
10 } // namespace score

```

调用时写 `score::summarize(values)`，读者知道函数来自哪个模块。不要在头文件里写 `using namespace std;`，它会污染所有包含这个头文件的翻译单元，制造名字冲突。

9.5 头文件放声明，源文件放实现

普通非模板函数通常在头文件声明，在源文件实现：

Listing 9.6 summary.hpp

```

1 #pragma once
2
3 #include <span>
4
5 namespace score {
6
7 struct Summary {
8     std::size_t count = 0;
9     int sum = 0;
10 };
11
12 Summary summarize(std::span<const int> values);
13
14 } // namespace score

```

Listing 9.7 summary.cpp

```

1 #include <score/summary.hpp>
2
3 namespace score {
4

```

```

5 Summary summarize(std::span<const int> values)
6 {
7     Summary result;
8     result.count = values.size();
9     for (int value : values) {
10         result.sum += value;
11     }
12     return result;
13 }
14
15 } // namespace score

```

头文件是依赖传播边界。头文件包含越多，编译依赖越重。能在源文件包含的实现细节，就不要放进公开头文件。

9.6 include guard 和 pragma once

头文件可能被多次包含，需要防止重复定义。现代项目常用 `##pragma once`：

Listing 9.8 防止头文件重复包含

```

1 #pragma once
2
3 // declarations

```

传统写法是 include guard：

Listing 9.9 传统 include guard

```

1 #ifndef SCORE_SUMMARY_HPP
2 #define SCORE_SUMMARY_HPP
3
4 // declarations
5
6 #endif

```

两者目的相同。团队已有风格优先跟随。关键不是选哪个，而是所有头文件都要有保护。

9.7 模板为什么常放头文件

模板在使用点实例化。编译器看到 `average<int>` 的调用时，需要模板定义本身，而不只是声明。因此模板通常放在头文件：

Listing 9.10 模板定义放头文件

```

1 template <typename T>
2 T max_value(T lhs, T rhs)
3 {
4     return lhs < rhs ? rhs : lhs;
5 }

```

这也是模板代码会增加编译依赖的原因。模板越大，包含它的文件越多，编译时间越容易上升。泛型代码要更克制。

9.8 完整案例：拆一个小数学库

目录：

Listing 9.11 小库目录

```
1 mathlib/
2   include/mathlib/statistics.hpp
3   src/statistics.cpp
4   tests/statistics_tests.cpp
```

公开接口只暴露必要类型和函数：

Listing 9.12 statistics.hpp

```
1 #pragma once
2
3 #include <optional>
4 #include <span>
5
6 namespace mathlib {
7
8     struct AverageResult {
9         double value = 0.0;
10    };
11
12    std::optional<AverageResult> average(std::span<const int> values);
13
14 } // namespace mathlib
```

测试只包含公开头文件，不包含 `src/statistics.cpp`。如果测试必须包含源文件才能访问内部函数，说明边界可能还没设计好。

习题与作业

把成绩统计函数放进 `score` 命名空间，并拆成头文件、源文件、测试文件。要求头文件不包含 `iostream`，测试通过公开接口调用，不直接包含 `.cpp` 文件。

9.9 头文件是编译依赖合同

头文件里的每个 `##include` 都会传播给所有包含它的源文件。假设 `statistics.hpp` 为一个实现细节包含了 `<iostream>`，那么所有使用统计接口的文件都会间接依赖 I/O 头。小项目看不出来，大项目会变成编译时间和耦合问题。

判断一个头文件该包含什么，可以问：公开声明是否需要这个类型的完整定义？例如：

Listing 9.13 公开接口需要 `span` 和 `optional`

```
1 #include <optional>
2 #include <span>
3
4 std::optional<AverageResult> average(std::span<const int> values);
```

这里 `std::optional` 和 `std::span` 出现在函数签名里，头文件必须包含相应头。实现里才用到的 `numeric`、`algorithm`、`iostream` 应放到 `.cpp`。

9.10 前置声明不是万能省 include

有时可以用前置声明减少依赖：

Listing 9.14 类指针可以前置声明

```
1 class Database;
2
3 class UserRepository {
4 public:
5     explicit UserRepository(Database& database);
6
7 private:
8     Database& database_;
9 };
```

因为这里只保存引用，不需要知道 `Database` 的大小。但如果成员是值对象，就必须有完整定义：

Listing 9.15 值成员需要完整定义

```
1 class UserRepository {
2 private:
3     Database database_; // 头文件必须知道 Database 大小
4 };
```

前置声明是依赖管理工具，不是躲避 include 的魔法。用错会造成编译错误或更难懂的边界。

9.11 重载要避免语义漂移

同名重载应该共享同一概念。下面的接口就有语义漂移：

Listing 9.16 同名函数语义不一致

```
1 void open(std::filesystem::path path);
2 void open(int flags);
```

第一个像打开文件，第二个可能是打开功能开关。读者看到 `open(3)` 很难知道发生什么。更好的名字应当把概念分开：

Listing 9.17 不同概念用不同名字

```
1 void open_file(const std::filesystem::path& path);
2 void enable_features(FeatureFlags flags);
```

重载不是为了少起名，而是为了同一概念在不同输入形态下自然表达。面积函数的正方形和矩形可以共享 `area`；文件打开和功能开关不应该共享。

9.12 选项对象如何演进

连接函数如果一开始只有 `host` 和 `port`，用两个参数可以接受。但当参数增加到 TLS、超时、重试、代理时，选项对象更稳：

Listing 9.18 可演进的连接选项

```
1 struct ConnectOptions {
2     std::string host;
```



```

3   int port = 80;
4   bool use_tls = false;
5   std::chrono::milliseconds timeout{3000};
6   int retry_count = 0;
7 };

```

调用点更清楚：

Listing 9.19 具名字段调用

```

1 connect(ConnectOptions{
2     .host = "example.com",
3     .port = 443,
4     .use_tls = true,
5     .timeout = std::chrono::seconds{5},
6 });

```

这比 `connect("example.com", 443, true, 5000, 3)` 更不容易把参数顺序搞错。选项对象也方便以后增加字段，而不制造一堆相近重载。

9.13 拆库案例的构建边界

一个小数学库的 CMake 也应该体现边界：

Listing 9.20 小库目标和测试目标

```

1 add_library(mathlib src/statistics.cpp)
2 target_compile_features(mathlib PUBLIC cxx_std_20)
3 target_include_directories(mathlib PUBLIC include)
4
5 add_executable(statistics_tests tests/statistics_tests.cpp)
6 target_link_libraries(statistics_tests PRIVATE mathlib)
7 add_test(NAME statistics_tests COMMAND statistics_tests)

```

测试链接库目标，而不是直接包含源文件。这样测试使用的路径和普通用户一致：都通过公开头文件和链接库访问。如果内部函数需要测试，可以考虑把它提升为公开的可测试小模块，或者通过公开行为间接覆盖，而不是破坏边界。

9.14 本章验收：依赖方向能画出来

完成一个模块后，应该能画出依赖方向：`main` 依赖应用层，应用层依赖库接口，库实现依赖标准库和内部细节，测试依赖公开接口。头文件越干净，依赖图越清楚。依赖图画不出来时，通常说明头文件、命名空间或模块边界已经混在一起。

第零部分

现代 Cpp 的核心：资源、对象和容器

第 10 章

生命周期、所有权和 RAII

10.1 问题入口：文件忘记关闭

资源不只是内存。文件句柄、锁、网络连接、临时目录、线程都属于资源。C 风格写法常见模式是打开后手动关闭：

Listing 10.1 手动管理文件资源容易遗漏

```
1 #include <cstdio>
2
3 void write_message()
4 {
5     FILE* file = std::fopen("out.txt", "w");
6     if (file == nullptr) {
7         return;
8     }
9     std::fprintf(file, "hello\n");
10    std::fclose(file);
11 }
```

这段代码看起来没问题，但一旦中间出现多个返回路径或异常，`fclose` 就容易漏。现代 C++ 的核心习惯是 RAII：资源获取即初始化，对象析构即释放。

10.2 RAII 的基本形状

Listing 10.2 用 `ofstream` 自动管理文件

```
1 #include <fstream>
2
3 void write_message()
4 {
5     std::ofstream output{"out.txt"};
6     if (!output) {
7         return;
8     }
9     output << "hello\n";
10 }
```

`std::ofstream` 对象离开作用域时会关闭文件。你不需要在每个分支手写关闭逻辑。RAII 的关键不是某个类，而是一个原则：资源的生命周期绑定到对象生命周期。

核心理想

如果一个资源需要释放，就应该有一个对象负责释放。不要让调用者记住“用完后一定要调用 `cleanup`”。这种规则靠人记，迟早会漏。

10.3 `unique_ptr` 表达独占所有权

动态对象不是不能用，而是要表达谁拥有它。独占所有权用 `std::unique_ptr`。

Listing 10.3 独占智能指针表达唯一拥有者

```

1 #include <memory>
2 #include <string>
3
4 struct User {
5     std::string name;
6 };
7
8 std::unique_ptr<User> make_user(std::string name)
9 {
10     return std::make_unique<User>(User{.name = std::move(name)});
11 }

```

`unique_ptr` 不能复制，只能移动。这个限制非常有价值：它让所有权转移在代码里可见。

Listing 10.4 移动表示所有权转移

```

1 std::unique_ptr<User> first = make_user("Ada");
2 std::unique_ptr<User> second = std::move(first);
3 // first 现在不再拥有对象

```

如果一个函数只观察用户，不要接收 `unique_ptr`，而应该接收引用或指针：

Listing 10.5 观察对象不要夺取所有权

```

1 void print_user(const User& user);
2 void maybe_print_user(const User* user);

```

10.4 `shared_ptr` 不是默认选择

`std::shared_ptr` 表达共享所有权。共享所有权意味着对象最后由谁释放取决于所有引用计数的生命周期。这在图结构、缓存、异步回调中 useful，但也更难推理。

常见误区

不要因为“不知道谁拥有”就默认用 `shared_ptr`。这通常说明设计边界还没想清楚。优先用值、引用、`unique_ptr`；确实有共享生命周期时再用 `shared_ptr`。

10.5 移动语义解决的不是“更快复制”

移动语义经常被误解成一种性能魔法。它真正表达的是：一个对象内部拥有的资源可以转交给另一个对象，原对象进入仍然有效但内容不再依赖的状态。以字符串为例：

Listing 10.6 移动字符串

```

1 std::string source = "large payload";
2 std::string target = std::move(source);

```

移动后 `source` 仍然可以析构、赋新值、调用不要求具体内容的成员函数；但不要继续假设它还保存 `"large payload"`。移动不是复制，移动后的对象也不是被销毁的对象。

设计构造函数时，一个常用模式是“按值接收，然后移动进成员”。调用者传右值时可以移动；传左值时会复制一份，语义也清楚。

Listing 10.7 按值接收并移动进成员

```

1 class UserProfile {
2 public:
3     explicit UserProfile(std::string name)
4         : name_{std::move(name)}
5     {
6     }
7
8 private:
9     std::string name_;
10 };

```

这个模式适合构造函数或明确要保存副本的 API。不适合只观察参数的函数。只观察时仍然应该用 `std::string_view` 或 `const std::string&`。

10.6 悬空引用

生命周期错误里最危险的是悬空引用：引用或指针指向的对象已经销毁。

Listing 10.8 返回局部变量引用是错误

```

1 const std::string& bad_name()
2 {
3     std::string name = "Ada";
4     return name; // 错误：函数返回后 name 已销毁
5 }

```

正确做法通常是返回值。现代编译器会做返回值优化或移动，没必要为了“省一次复制”返回悬空引用。

Listing 10.9 返回值拥有自己的生命周期

```

1 std::string make_name()
2 {
3     return "Ada";
4 }

```

10.7 本章边界检查

看到资源管理代码时，问四个问题：

- 谁拥有资源？
- 资源什么时候释放？
- 出错或提前返回时是否释放？
- 接口是在观察资源，还是转移所有权？

这四个问题比背智能指针 API 更重要。

10.8 异常路径下 RAII 为什么成立

手动释放资源最怕“中间退出”。下面这个版本只要第二步失败，就必须记得关闭文件：

Listing 10.10 多出口函数里的手动清理压力

```

1 void write_two_lines_c_style()
2 {

```

```

3 FILE* file = std::fopen("out.txt", "w");
4 if (file == nullptr) {
5     return;
6 }
7 if (std::fprintf(file, "first\n") < 0) {
8     std::fclose(file);
9     return;
10 }
11 if (std::fprintf(file, "second\n") < 0) {
12     std::fclose(file);
13     return;
14 }
15 std::fclose(file);
16 }

```

这个例子里每个失败分支都要重复 `fclose`。分支越多，遗漏概率越高。RAII 把重复规则变成对象析构规则：

Listing 10.11 RAII 让退出路径统一

```

1 void write_two_lines()
2 {
3     std::ofstream output{"out.txt"};
4     if (!output) {
5         return;
6     }
7     output << "first\n";
8     if (!output) {
9         return;
10    }
11    output << "second\n";
12 }

```

无论函数从哪个 `return` 离开，`output` 都会析构。异常路径也一样：栈展开时已经构造成功的局部对象会按逆序析构。这个性质是现代 C++ 资源管理的地基。

10.9 所有权转移要在接口上可见

观察下面两个函数签名：

Listing 10.12 签名暴露所有权语义

```

1 void inspect_user(const User& user);
2 void store_user(std::unique_ptr<User> user);

```

`inspect_user` 告诉调用者：函数只观察对象，调用结束后对象仍归调用者所有。`store_user` 告诉调用者：函数会接管对象所有权，调用后原来的 `unique_ptr` 不再拥有对象。调用点必须写出 `std::move`：

Listing 10.13 调用点显式转移所有权

```

1 auto user = make_user("Ada");
2 inspect_user(*user);
3 store_user(std::move(user));

```

这个 `std::move` 不是把内存“立刻搬走”的命令，它是一个把左值转换成可移动表达式的标记。真正的移动由目标类型的移动构造或移动赋值完成。写 `std::move` 的价值是让读代码的人看见所有权边界。

10.10 移动后的对象该怎么用

移动后的对象仍然有效，但值通常不再值得假设。最安全的后续操作是销毁、重新赋值或调用明确允许的查询：

Listing 10.14 移动后重新赋值再使用

```
1 std::string name = "Ada";
2 std::string stored = std::move(name);
3
4 name = "Grace";
5 std::cout << name << '\n';
```

不要写依赖移动后内容的代码：

Listing 10.15 不要依赖移动后的旧内容

```
1 std::string name = "Ada";
2 std::string stored = std::move(name);
3 // 不要假设 name.empty() 一定为 true
```

标准只保证 `name` 处在可析构、可赋值的有效状态，不保证它具体等于什么。这个规则在容器、字符串、智能指针里都要记住。

10.11 `shared_ptr` 的典型陷阱：循环引用

共享所有权最大的陷阱是循环引用：

Listing 10.16 共享指针循环引用会阻止释放

```
1 struct Node {
2     std::shared_ptr<Node> next;
3     std::shared_ptr<Node> prev;
4 };
5
6 auto first = std::make_shared<Node>();
7 auto second = std::make_shared<Node>();
8 first->next = second;
9 second->prev = first;
```

即使外部的 `first` 和 `second` 离开作用域，两个节点仍然互相持有对方，引用计数不能归零。反向边如果只是观察关系，应该用 `std::weak_ptr`：

Listing 10.17 弱指针表达非拥有反向引用

```
1 struct Node {
2     std::shared_ptr<Node> next;
3     std::weak_ptr<Node> prev;
4 };
```

这再次说明：智能指针不是“高级指针替代品”，而是所有权词汇。选错词汇，生命周期仍然会错。

习题与作业

写一个 `Timer` 类，在构造时记录开始时间，在析构时打印耗时。然后把它放进一个有多个 `return` 分支的函数里，观察每条路径都会打印耗时。这个练习能帮助你把 RAII 从“文件关闭”推广到任意成对动作。

10.12 从提交失败推导事务式 RAII

文件关闭只是 RAII 的入门样本。更接近工程的场景是：写配置文件时不能把旧文件写坏。错误版本通常直接覆盖目标文件：

Listing 10.18 直接覆盖配置文件的风险

```
1 void save_config(const Config& config, const std::filesystem::path& path)
2 {
3     std::ofstream output{path};
4     output << render(config);
5 }
```

如果程序写到一半崩溃，目标文件可能只剩半截。更稳的策略是先写临时文件，成功后再重命名替换：

Listing 10.19 先写临时文件再提交

```
1 void save_config_atomically(const Config& config,
2                             const std::filesystem::path& path)
3 {
4     const std::filesystem::path temporary = path.string() + ".tmp";
5     {
6         std::ofstream output{temporary};
7         if (!output) {
8             throw std::runtime_error{"cannot open temporary config"};
9         }
10        output << render(config);
11        if (!output) {
12            throw std::runtime_error{"cannot write temporary config"};
13        }
14    }
15    std::filesystem::rename(temporary, path);
16 }
```

这里大括号不是装饰。它让 `ofstream` 在重命名前析构，确保文件先关闭。否则在某些平台上，打开的文件可能影响替换操作。RAII 的推理链条变成：临时文件对象负责写入期间的文件句柄；作用域结束释放句柄；提交步骤只在完整写入之后发生。

10.13 回滚动作也能交给对象

如果提交前出错，临时文件应该清理。可以写一个小的作用域守卫：

Listing 10.20 作用域守卫负责失败清理

```
1 class TemporaryFileCleanup {
2 public:
3     explicit TemporaryFileCleanup(std::filesystem::path path)
4         : path_{std::move(path)}
5     {
```



```

6     }
7
8     void release() noexcept
9     {
10         this->active_ = false;
11     }
12
13     ~TemporaryFileCleanup()
14     {
15         if (this->active_) {
16             std::error_code ignored;
17             std::filesystem::remove(this->path_, ignored);
18         }
19     }
20
21 private:
22     std::filesystem::path path_;
23     bool active_ = true;
24 };

```

使用方式如下：

Listing 10.21 提交成功后取消清理

```

1 const std::filesystem::path temporary = path.string() + ".tmp";
2 TemporaryFileCleanup cleanup{temporary};
3 write_full_file(temporary, render(config));
4 std::filesystem::rename(temporary, path);
5 cleanup.release();

```

析构函数里故意吞掉删除错误，因为析构函数不应随意抛异常。真正需要报告的错误应发生在显式步骤里，例如打开失败、写入失败、重命名失败。RAII 对象负责“无论路径怎么退出都尽量收尾”，不是负责隐藏核心业务错误。

10.14 析构顺序也是设计工具

局部对象按构造逆序析构。这个规则可以用来安排资源依赖：

Listing 10.22 依赖对象要晚构造早析构

```

1 void write_with_lock()
2 {
3     std::ofstream output{"out.txt"};
4     std::lock_guard<std::mutex> lock{global_mutex};
5     output << "message\n";
6 }

```

这段代码里 `lock` 比 `output` 后构造，所以先析构。是否合理要看需求：如果你希望整个文件写入和关闭都在锁内，应调整作用域；如果只需要保护写入语句，当前写法可能足够。关键是不要把析构顺序当成偶然。它是 C++ 资源管理的一部分。

10.15 所有权图：谁负责释放

判断智能指针时，可以画一张很小的所有权图：

关系	推荐表达	解释
对象属于当前作用域	直接值对象	离开作用域自动析构
对象唯一属于某个所有者	<code>std::unique_ptr</code>	所有权移动可见
对象被多个异步任务共同延寿	<code>std::shared_ptr</code>	最后一个所有者释放
对象只是被观察 反向引用不延寿	引用、指针、视图 <code>std::weak_ptr</code>	不负责释放 避免循环所有权

如果画不出谁拥有谁，就不要急着选 `shared_ptr`。很多共享指针滥用，本质是所有权图没想清楚。

10.16 本章验收：每个资源都要有退出路径

看到资源代码时，把所有退出路径列出来：正常返回、提前返回、异常、对象移动、容器扩容、线程取消。每条路径都要能回答资源如何释放。RAII 的价值就是让这些路径共享同一套析构规则，而不是在每个分支复制清理代码。

第 11 章

容器、迭代器和标准库思维

11.1 问题入口：去重并保持顺序

给定一组用户 ID，要求去掉重复项，但保留第一次出现的顺序。例如：

Listing 11.1 输入输出示例

```
1 input  = {3, 5, 3, 7, 5, 9};
2 output = {3, 5, 7, 9};
```

朴素写法是每次检查结果数组里是否已有该元素：

Listing 11.2 朴素去重

```
1 std::vector<int> unique_ids;
2 for (int id : ids) {
3     bool seen = false;
4     for (int existing : unique_ids) {
5         if (existing == id) {
6             seen = true;
7             break;
8         }
9     }
10    if (!seen) {
11        unique_ids.push_back(id);
12    }
13 }
```

这段代码正确，但复杂度是平方级。输入很大时，会反复扫描历史结果。

11.2 组合容器表达需求

需求有两个部分：判断是否出现过，保留顺序。哈希集合适合判断出现，向量适合保留顺序。

Listing 11.3 哈希集合加顺序数组

```
1 #include <unordered_set>
2 #include <vector>
3
4 std::vector<int> deduplicate_keep_order(std::span<const int> ids)
5 {
6     std::unordered_set<int> seen;
7     std::vector<int> result;
8     result.reserve(ids.size());
9
10    for (int id : ids) {
11        if (seen.insert(id).second) {
12            result.push_back(id);
13        }
14    }
```

```

15     return result;
16 }

```

`seen.insert(id).second` 表示这次是否真的插入了新元素。这个写法比先 `contains` 再 `insert` 少一次查找。

11.3 vector 不是数组替身

`std::vector` 是连续存储的动态数组。连续存储带来两个结果：遍历快，尾部追加通常快；但中间插入和删除需要移动后面的元素。

Listing 11.4 删除元素时使用 `erase-remove`

```

1 std::vector<int> values{1, 2, 3, 2, 4};
2 const auto new_end = std::remove(values.begin(), values.end(), 2);
3 values.erase(new_end, values.end());

```

`std::remove` 不会真的缩短容器，它只是把保留元素移动到前面并返回新的逻辑结尾。真正删除尾部残余的是 `erase`。

11.4 迭代器失效

容器操作可能让迭代器、引用和指针失效。比如 `vector` 扩容后，原来指向元素的指针可能全都失效。

Listing 11.5 尾部追加可能让指针失效

```

1 std::vector<int> values;
2 values.push_back(1);
3 int* first = &values[0];
4 values.push_back(2); // 可能扩容
5 // first 可能已经悬空

```

如果提前知道元素数量，用 `reserve` 可以减少扩容次数，但仍然不能把长期稳定地址建立在 `vector` 元素指针上。需要稳定节点地址时，考虑 `std::list`、`std::deque` 或自己管理对象存储，但要先确认真实需求。

11.5 选择容器的经验表

需求	常用容器	注意点
顺序存储、频繁遍历	<code>std::vector</code>	扩容会移动元素
两端插入删除	<code>std::deque</code>	不保证整体连续
按 key 查找	<code>std::unordered_map</code>	平均快，依赖哈希质量
有序遍历	<code>std::map</code>	节点结构，常数较大
唯一集合	<code>std::unordered_set</code>	自定义类型需要 hash

核心思想

标准库容器不是语法点，而是复杂度合同。选择容器前先说清楚：是否需要顺序、是否需要唯一、是否需要稳定地址、主要操作是什么。

11.6 从输入规模推导复杂度

朴素去重为什么慢？用一个具体输入推导。假设所有 ID 都不重复，且一共有 n 个。第一个元素比较 0 次，第二个比较 1 次，第三个比较 2 次，最后一个比较 $n - 1$ 次。总比较次数是：

$$0 + 1 + 2 + \cdots + (n - 1) = \frac{n(n - 1)}{2}$$

这就是平方级。输入 100 个时还不明显，输入 100000 个时就会非常慢。哈希集合版本把“是否见过”变成平均常数时间查询，于是整体接近线性。算法优化不是凭感觉说“哈希快”，而是看历史信息是否被反复扫描。

11.7 reserve 的作用和边界

在去重函数里写 `result.reserve(ids.size())`，不是为了改变复杂度，而是减少扩容。因为最多保留 `ids.size()` 个元素，提前预留这个容量合理。注意 `reserve` 只改变容量，不改变元素个数：

Listing 11.6 `reserve` 不会创建元素

```
1 std::vector<int> values;
2 values.reserve(10);
3 assert(values.size() == 0);
4 values.push_back(42);
5 assert(values.size() == 1);
```

如果写 `resize(10)`，就真的创建了十个元素。两者的区别必须分清：容量是能放多少，大小是已经有多少。

11.8 迭代器失效要看容器合同

不同容器的失效规则不同。入门阶段可以先抓住几个高频点：

- `vector` 扩容会让所有元素引用、指针、迭代器失效。
- `vector` 中间删除会让删除点之后的位置失效。
- `list` 移动节点通常不让其他节点迭代器失效。
- `unordered_map` rehash 可能让迭代器失效，但元素引用通常仍指向元素对象。

不要凭容器名字猜。每次把迭代器保存到后面用，都要问：中间有没有可能插入、删除、扩容或 rehash？LRU 缓存选择 `std::list`，正是因为它需要在移动节点时保持迭代器稳定。

11.9 容器选择案例：排行榜

假设要维护游戏排行榜，需求是按玩家 ID 更新分数，并输出前十名。一个容器很难同时满足所有操作。可以先用 `unordered_map<int, int>` 保存玩家到分数的映射；输出前十时再把条目放进 `vector` 排序：

Listing 11.7 排行榜的两阶段容器设计

```
1 using Scores = std::unordered_map<int, int>;
2
3 std::vector<std::pair<int, int>> top_scores(const Scores& scores)
4 {
5     std::vector<std::pair<int, int>> items(scores.begin(), scores.end());
6     std::ranges::sort(items, [] (const auto& lhs, const auto& rhs) {
7         if (lhs.second != rhs.second) {
8             return lhs.second > rhs.second;
9         }
10        return lhs.first < rhs.first;
11    });
12    if (items.size() > 10) {
13        items.resize(10);
```

```

14     }
15     return items;
16 }

```

如果更新非常频繁、查询前十也非常频繁，就可能需要额外维护有序结构。但第一版不要过早复杂化。先让主要操作和数据规模决定容器组合。

习题与作业

给 `deduplicate_keep_order` 写三组测试：全重复、全不重复、混合重复。再把 `unordered_set` 去掉，手写朴素版本，对比两者在 1000、10000、100000 个随机 ID 下的运行时间。

11.10 从需求字段推导容器组合

容器选择不能只背“vector 快、map 有序”。先把需求字段列出来。去重并保持顺序这个问题有两条互相独立的需求：第一，能快速判断一个 ID 是否已经出现；第二，输出按第一次出现的顺序排列。单个容器很难同时把这两条都表达得最好。

需求字段	合适结构	原因
快速判断出现过	<code>std::unordered_set</code>	平均常数时间查询和插入
保留第一次出现顺序	<code>std::vector</code>	追加顺序就是输出顺序
输出结果连续存放	<code>std::vector</code>	方便遍历、返回和测试

这样得到的解法不是“用了高级容器”，而是两个容器各自负责一个需求字段。以后做 LRU 缓存，会看到类似组合：哈希表负责按 key 找节点，链表负责维护最近使用顺序。

11.11 地址稳定不是小细节

如果代码保存了容器元素的地址，容器选择就进入另一个层次。看一个任务队列索引：

Listing 11.8 保存 vector 元素指针的风险

```

1 std::vector<Task> tasks;
2 tasks.push_back(Task{.id = 1});
3 Task* first = &tasks[0];
4 tasks.push_back(Task{.id = 2});
5 first->done = true; // 如果扩容发生，first 可能已经失效

```

这不是 `vector` 的缺点，而是它的合同：为了保持连续存储，扩容时可以搬走所有元素。你要的是遍历快和紧凑内存，就接受地址可能变化；你要的是节点地址稳定，就选择别的结构或保存索引而不是指针。

一个实用替代是保存下标：

Listing 11.9 保存下标而不是元素指针

```

1 std::size_t first_index = 0;
2 tasks.push_back(Task{.id = 2});
3 tasks[first_index].done = true;

```

下标也不是永远安全：如果删除或重排元素，它指向的逻辑任务可能变化。但它不会因为扩容变成悬空地址。容器合同要和修改操作一起看。

11.12 rehash 也会改变迭代环境

`unordered_map` 平均查找快，但插入很多元素时可能 rehash。rehash 会重新组织桶，迭代器会失效。假设你一边遍历一边插入：

Listing 11.10 遍历哈希表时插入要小心

```
1 for (auto it = counts.begin(); it != counts.end(); ++it) {
2     counts[new_key()] += 1; // 可能 rehash, 使 it 失效
3 }
```

这类代码危险，因为循环变量本身可能被插入操作破坏。常见修复是先收集要插入的内容，遍历结束后统一插入；或者在已知规模时提前 `reserve`，并确认仍不违反容器规则。不要凭“我这次测试没崩”判断安全。

11.13 erase 循环要让容器返回下一个位置

删除元素时，迭代器推进规则也要严格。错误版本：

Listing 11.11 删除后继续使用失效迭代器

```
1 for (auto it = values.begin(); it != values.end(); ++it) {
2     if (*it < 0) {
3         values.erase(it);
4     }
5 }
```

`erase` 后 `it` 已经失效，循环末尾的 `++it` 不可靠。正确版本使用返回值：

Listing 11.12 erase 返回下一个有效位置

```
1 for (auto it = values.begin(); it != values.end(); ) {
2     if (*it < 0) {
3         it = values.erase(it);
4     } else {
5         ++it;
6     }
7 }
```

如果删除条件简单，优先使用 `std::erase_if`：

Listing 11.13 用标准库表达删除条件

```
1 std::erase_if(values, [](int value) {
2     return value < 0;
3 });
```

标准库算法不是为了炫技，而是把容易写错的迭代器协议封装成一个清晰意图。

11.14 本章验收：写出容器账本

选择容器前写出四列：主要操作、顺序要求、地址稳定要求、规模。比如“十万个用户，按 ID 查找，偶尔输出按注册顺序排列”，很可能需要 `unordered_map` 加 `vector` 或在对象里保存注册序号。比如“几百个规则，按优先级顺序扫描”，简单 `vector` 可能比树和哈希都好。容器选择的高级感不来自名字，而来自你能说清楚每个合同。

第 12 章

错误处理和接口设计

12.1 问题入口：读取配置文件

假设要读取一个配置文件，里面只有一行端口号。可能发生的错误有：文件不存在、内容为空、不是数字、数字超出范围。朴素写法容易把错误吞掉：

Listing 12.1 吞掉错误的读取函数

```
1 int read_port()
2 {
3     std::ifstream input{"server.conf"};
4     int port = 0;
5     input >> port;
6     return port;
7 }
```

如果文件不存在，函数返回 0。但 0 是错误还是合法端口？调用者不知道。

12.2 错误要进入接口

先定义错误类型：

Listing 12.2 配置读取错误

```
1 enum class ConfigError {
2     file_not_found,
3     empty_file,
4     invalid_number,
5     out_of_range,
6 };
7
8 struct PortConfig {
9     int port = 0;
10 };
11
12 struct PortConfigResult {
13     std::optional<PortConfig> config;
14     std::optional<ConfigError> error;
15 };
```

然后实现读取逻辑：

Listing 12.3 显式返回错误

```
1 #include <fstream>
2 #include <optional>
3 #include <string>
4
5 constexpr int PORT_MIN = 1;
6 constexpr int PORT_MAX = 65535;
```



```

7
8 PortConfigResult read_port_config(const std::string& path)
9 {
10     std::ifstream input{path};
11     if (!input) {
12         return {error = ConfigError::file_not_found};
13     }
14
15     std::string text;
16     if (!(input >> text)) {
17         return {error = ConfigError::empty_file};
18     }
19
20     const std::optional<int> parsed = parse_int(text);
21     if (!parsed.has_value()) {
22         return {error = ConfigError::invalid_number};
23     }
24     if (*parsed < PORT_MIN || *parsed > PORT_MAX) {
25         return {error = ConfigError::out_of_range};
26     }
27     return {.config = PortConfig{.port = *parsed}};
28 }

```

这里没有说异常不好。异常适合无法就地处理、需要跨层传播的错误；显式结果适合调用者马上需要分支处理的错误。关键是不要让错误伪装成普通值。

12.3 接口越小，错误越少

接口设计的第一原则是少暴露。比如一个缓存类不应该同时负责读取文件、解析 JSON、网络刷新、过期策略和日志输出。先定义一个小接口：

Listing 12.4 小接口降低耦合

```

1 class PortProvider {
2 public:
3     explicit PortProvider(PortConfig config)
4         : config_{config}
5     {
6     }
7
8     [[nodiscard]] int port() const noexcept
9     {
10         return this->config_.port;
11     }
12
13 private:
14     PortConfig config_;
15 };

```

它只表达“我能提供端口”。至于端口来自文件、命令行还是测试固定值，不应该被这个类知道。

12.4 异常边界

异常不是洪水猛兽，但要有边界。底层函数可以抛出 `std::runtime_error`，但程序入口应该捕获并转换成可理解的错误消息。

Listing 12.5 程序入口处建立异常边界

```

1 int main()
2 {
3     try {
4         return run_application();
5     } catch (const std::exception& error) {
6         std::cerr << "fatal: " << error.what() << '\n';
7         return 1;
8     }
9 }

```

不要在每一层都捕获再抛出同样的异常。捕获必须有目的：恢复、补充上下文、转换错误类型或终止程序。

常见误区

最差的错误处理是“打印一下然后继续”。如果状态已经不可信，继续执行可能制造更隐蔽的数据损坏。

12.5 从端口号反例推导错误类型

为什么 `read_port` 返回 `0` 不够？因为我们至少有四种不同处理方式。文件不存在时，用户可能需要检查路径；空文件时，可能需要补配置；不是数字时，要指出格式错误；超出范围时，要告诉合法范围。一个 `0` 抹掉了这些差别。

因此错误类型不是为了形式完整，而是为了保留决策所需的信息。还可以给错误转换成人类可读消息：

Listing 12.6 错误枚举转消息

```

1 std::string_view describe(ConfigError error)
2 {
3     switch (error) {
4     case ConfigError::file_not_found:
5         return "config file not found";
6     case ConfigError::empty_file:
7         return "config file is empty";
8     case ConfigError::invalid_number:
9         return "port must be a number";
10    case ConfigError::out_of_range:
11        return "port must be between 1 and 65535";
12    }
13    return "unknown config error";
14 }

```

这个 `switch` 应该靠编译警告帮助维护。如果以后新增错误枚举，却忘了补消息，警告会提醒你。

12.6 错误处理分层

同一个错误在不同层有不同表达。底层读取函数返回结构化错误；命令行入口把错误转换成文本和退出码；测试直接断言错误枚举：

Listing 12.7 入口层转换错误

```

1 int run_server(std::string_view config_path)

```

```

2 {
3     const PortConfigResult result = read_port_config(std::string{config_path});
4     if (result.error.has_value()) {
5         std::cerr << describe(*result.error) << '\n';
6         return 2;
7     }
8     return start_server(result.config->port);
9 }

```

不要让底层函数直接 `std::cerr`。一旦底层打印，图形界面、服务进程和测试环境都被迫接受同一种输出方式。接口返回错误，调用者才有选择。

12.7 optional 适合什么，不适合什么

`std::optional<T>` 适合表达“可能没有值，但没有更多错误信息”。例如按 ID 查找用户：

Listing 12.8 查不到不是异常

```

1 std::optional<UserRecord> find_user(int id);

```

如果失败原因会影响处理，就不要只用 `optional`。读取配置失败显然有不同原因，所以需要错误类型。异常则适合跨多层传播、当前层无法恢复的错误，例如配置语义严重错误、数据库连接失败、程序无法继续启动。

12.8 接口大小与测试成本

比较两个接口：

Listing 12.9 过大的接口难测试

```

1 class ServerManager {
2 public:
3     void load_config();
4     void open_log();
5     void connect_database();
6     void start_threads();
7     void run();
8 };

```

这个类每个测试都可能需要文件、日志、数据库和线程。把端口读取拆成纯函数后，测试只需要临时文件或字符串解析。接口越小，构造测试环境越容易；测试越容易，错误路径越可能被覆盖。

习题与作业

把 `read_port_config` 拆成两个函数：一个从 `std::string_view` 解析端口文本，一个负责读文件。要求文本解析函数不做任何 I/O，并覆盖空字符串、字母、0、65536、合法端口五个测试。

12.9 Result 背后的状态机

错误接口也可以用普通代码形状理解。一个“值或错误”的结果，本质是两个状态：

Listing 12.10 Result 状态

```

1 Success(PortConfig)
2 Failure(ConfigError)

```

如果用两个 `optional` 字段表达，状态空间会变成四种：

有 config	有 error	含义
否	否	无意义状态
是	否	成功
否	是	失败
是	是	矛盾状态

所以工程里更严谨的写法是让类型只能处在成功或失败之一：

Listing 12.11 variant 实现二选一 Result

```
1 template <typename T, typename E>
2 class Result {
3 public:
4     static Result success(T value)
5     {
6         return Result{std::variant<T, E>{std::move(value)}};
7     }
8
9     static Result failure(E error)
10    {
11        return Result{std::variant<T, E>{std::move(error)}};
12    }
13
14    [[nodiscard]] bool has_value() const noexcept
15    {
16        return std::holds_alternative<T>(this->state_);
17    }
18
19 private:
20     explicit Result(std::variant<T, E> state)
21         : state_{std::move(state)}
22     {
23     }
24
25     std::variant<T, E> state_;
26 };
```

这段代码比两个 `optional` 麻烦，但它消除了无意义状态。教学早期可以用简单结构降低门槛；一旦接口要长期维护，就应该让类型替你挡住矛盾状态。

12.10 从异常到错误码的边界

异常和显式错误不是敌人。一个常见分层是：

- 解析一段用户文本：返回结构化错误，调用者要展示具体原因。
- 打开一个必须存在的内部资源：失败时抛异常，当前层无法恢复。
- 程序入口：捕获异常，转换成日志和退出码。

也就是说，选择错误机制前先问“谁能处理”。如果当前调用者能根据错误类型做分支，返回 `Result` 清楚；如果只能向上层传播，异常更直接；如果错误只是“查不到”，`optional` 足够。

12.11 parse_port_text 的完整推导

把端口解析从文件读取里拆出来：

Listing 12.12 只解析文本的端口函数

```
1 Result<PortConfig, ConfigError> parse_port_text(std::string_view text)
2 {
3     text = trim(text);
4     if (text.empty()) {
5         return Result<PortConfig, ConfigError>::failure(ConfigError::empty_file);
6     }
7
8     const std::optional<int> parsed = parse_int_fast(text);
9     if (!parsed.has_value()) {
10        return Result<PortConfig, ConfigError>::failure(ConfigError::invalid_number);
11    }
12    if (*parsed < PORT_MIN || *parsed > PORT_MAX) {
13        return Result<PortConfig, ConfigError>::failure(ConfigError::out_of_range);
14    }
15    return Result<PortConfig, ConfigError>::success(PortConfig{.port = *parsed});
16 }
```

这个函数不打开文件、不打印错误、不启动服务。它只把一段文本变成端口配置或错误。因此测试可以完全不碰文件系统：

Listing 12.13 端口文本测试样本

```
1 assert(!parse_port_text("").has_value());
2 assert(!parse_port_text("abc").has_value());
3 assert(!parse_port_text("0").has_value());
4 assert(!parse_port_text("65536").has_value());
5 assert(parse_port_text("8080").has_value());
```

这就是错误处理和接口设计的关系：接口越小，错误越具体，测试越便宜。

12.12 错误上下文不要丢

如果文件层调用 `parse_port_text`，错误还应补充路径：

Listing 12.14 文件层补充上下文

```
1 struct PortFileError {
2     ConfigError kind = ConfigError::invalid_number;
3     std::filesystem::path path;
4 };
```

底层解析不知道路径，文件层知道路径。不要让底层为了错误消息去依赖文件系统；也不要再在文件层把解析错误简化成 `false`。每层补充自己知道的上下文，错误信息才完整。

第 13 章

复制、移动和对象设计

13.1 问题入口：为什么有些对象不能复制

先看一个直觉上危险的类：

Listing 13.1 手写资源类的复制风险

```
1 class FileHandle {
2 public:
3     explicit FileHandle(FILE* file)
4         : file_{file}
5     {
6     }
7
8     ~FileHandle()
9     {
10         if (this->file_ != nullptr) {
11             std::fclose(this->file_);
12         }
13     }
14
15 private:
16     FILE* file_ = nullptr;
17 };
```

如果这个类允许默认复制，会发生什么？两个 `FileHandle` 对象会保存同一个 `FILE*`。它们析构时都会调用 `fclose`，同一个资源被释放两次。这就是复制语义和资源所有权冲突。

这个反例推出第一条规则：拥有独占资源的对象通常不应该复制，除非你能定义“复制资源”到底是什么意思。

13.2 明确禁止复制

禁止复制要写在类型接口上：

Listing 13.2 禁止复制独占资源对象

```
1 class FileHandle {
2 public:
3     explicit FileHandle(FILE* file)
4         : file_{file}
5     {
6     }
7
8     FileHandle(const FileHandle&) = delete;
9     FileHandle& operator=(const FileHandle&) = delete;
10
11     ~FileHandle()
12     {
13         if (this->file_ != nullptr) {
```

```

14         std::fclose(this->file_);
15     }
16 }
17
18 private:
19     FILE* file_ = nullptr;
20 };

```

现在调用者不能误复制。编译错误比运行时双重释放更早、更清楚。禁止复制不是“限制用户”，而是类型诚实表达所有权规则。

13.3 移动构造：把资源交给新对象

独占资源虽然不能复制，但可以移动。移动的关键是：新对象接管资源，旧对象不再负责释放。

Listing 13.3 资源类的移动构造

```

1 class FileHandle {
2 public:
3     explicit FileHandle(FILE* file)
4         : file_{file}
5     {
6     }
7
8     FileHandle(const FileHandle&) = delete;
9     FileHandle& operator=(const FileHandle&) = delete;
10
11     FileHandle(FileHandle&& other) noexcept
12         : file_{other.file_}
13     {
14         other.file_ = nullptr;
15     }
16
17     FileHandle& operator=(FileHandle&& other) noexcept
18     {
19         if (this == &other) {
20             return *this;
21         }
22         this->close();
23         this->file_ = other.file_;
24         other.file_ = nullptr;
25         return *this;
26     }
27
28     ~FileHandle()
29     {
30         this->close();
31     }
32
33 private:
34     void close() noexcept
35     {
36         if (this->file_ != nullptr) {
37             std::fclose(this->file_);

```

```

38         this->file_ = nullptr;
39     }
40 }
41
42 FILE* file_ = nullptr;
43 };

```

移动赋值里先释放当前资源，再接管对方资源。最后把对方置空，防止对方析构时再次释放。这个过程必须异常安全，所以移动构造和移动赋值通常标成 **noexcept**。标准容器在扩容时也会利用这个信息。

13.4 零法则：优先让成员自己管理资源

上面的例子是为了理解规则。真实代码里，优先使用标准库 RAII 类型，让类不需要手写析构、复制和移动：

Listing 13.4 用标准库成员获得零法则

```

1 class ReportWriter {
2 public:
3     explicit ReportWriter(std::filesystem::path path)
4         : path_{std::move(path)}
5     {
6     }
7
8     void write(std::string_view text) const
9     {
10         std::ofstream output{this->path_};
11         if (!output) {
12             throw std::runtime_error{"cannot open report"};
13         }
14         output << text;
15     }
16
17 private:
18     std::filesystem::path path_;
19 };

```

这个类没有手写析构、复制构造、移动构造。它的成员 `std::filesystem::path` 自己知道如何复制和移动。只要不直接管理裸资源，就能让编译器生成正确的特殊成员函数。这叫零法则：能不手写特殊成员，就不手写。

13.5 值对象应该容易复制

不是所有对象都要禁止复制。像 `Money`、`TemperatureRange`、`ScoreSummary` 这类值对象，复制语义自然、成本可控，应该像普通值一样使用：

Listing 13.5 值对象可以自然复制

```

1 TemperatureRange morning{10.0, 18.0};
2 TemperatureRange backup = morning;
3 backup.expand_to_include(20.0);

```

修改 `backup` 不应该影响 `morning`。如果一个看起来像值的对象复制后还共享内部状态，读者会很难推理。设计值对象时，要么让副本独立，要么明确改成共享所有权模型。

13.6 按值接收再移动的适用边界

构造函数保存字符串时，常见写法是按值接收：

Listing 13.6 按值接收适合保存副本

```
1 class User {
2 public:
3     explicit User(std::string name)
4         : name_{std::move(name)}
5     {
6     }
7
8 private:
9     std::string name_;
10 };
```

调用者传临时字符串时，参数可以移动；传已有字符串时，会复制一份再移动进成员。这个语义很清楚：对象要保存自己的名字。反过来，如果函数只打印名字，就不要按值接收：

Listing 13.7 只观察时用字符串视图

```
1 void print_user_name(std::string_view name)
2 {
3     std::cout << name << '\n';
4 }
```

按值接收不是性能技巧，而是所有权语义。保存副本时合理，只观察时不合理。

13.7 自赋值和自移动

复制赋值和移动赋值都可能遇到自己赋给自己。复制赋值常见写法可以用 copy-and-swap，但入门阶段先理解原则：不要在还没拿到新资源之前破坏旧资源。移动赋值则至少要处理 `this == &other`，否则可能把自己资源关闭后再接管空资源。

常见误区

如果你发现自己在多个类里手写析构、复制和移动，先停下来问：是不是应该用 `std::unique_ptr`、`std::vector`、`std::string`、`std::fstream` 等标准库成员承担资源管理？手写特殊成员函数越多，越容易出生命周期错误。

13.8 完整案例：不可复制的临时目录

临时目录对象拥有一个目录路径，析构时删除目录。它不应该复制，因为两个对象删除同一目录很危险；它可以移动，因为所有权可以转移：

Listing 13.8 临时目录所有权类型

```
1 class TemporaryDirectory {
2 public:
3     explicit TemporaryDirectory(std::filesystem::path path)
4         : path_{std::move(path)}
5     {
6         std::filesystem::create_directories(this->path_);
7     }
8
9     TemporaryDirectory(const TemporaryDirectory&) = delete;
```

```

10 TemporaryDirectory& operator=(const TemporaryDirectory&) = delete;
11
12 TemporaryDirectory(TemporaryDirectory&& other) noexcept
13     : path_{std::move(other.path_)}, active_{other.active_}
14 {
15     other.active_ = false;
16 }
17
18 TemporaryDirectory& operator=(TemporaryDirectory&& other) noexcept
19 {
20     if (this == &other) {
21         return *this;
22     }
23     this->cleanup();
24     this->path_ = std::move(other.path_);
25     this->active_ = other.active_;
26     other.active_ = false;
27     return *this;
28 }
29
30 ~TemporaryDirectory()
31 {
32     this->cleanup();
33 }
34
35 [[nodiscard]] const std::filesystem::path& path() const noexcept
36 {
37     return this->path_;
38 }
39
40 private:
41     void cleanup() noexcept
42     {
43         if (!this->active_) {
44             return;
45         }
46         std::error_code ignored;
47         std::filesystem::remove_all(this->path_, ignored);
48         this->active_ = false;
49     }
50
51     std::filesystem::path path_;
52     bool active_ = true;
53 };

```

这里删除目录使用 `std::error_code`，因为析构函数不应该抛异常。这个例子把本章几条线合在一起：独占资源禁止复制，允许移动，析构释放资源，析构失败不能让异常逃出。

习题与作业

设计一个 `SocketHandle`，假设关闭函数是 `close_socket(int fd)`。要求禁止复制、允许移动、析构关闭。再写出移动后对象的状态不变量：它是否还拥有有效 `fd`？析构时会不会重复关闭？

第 14 章

字符串、文件和流式处理

14.1 问题入口：读取一整个文件是否总是好主意

很多教程会直接把文件全部读进字符串：

Listing 14.1 一次性读取文件的思路

```
1 std::string read_all(std::istream& input)
2 {
3     std::ostringstream buffer;
4     buffer << input.rdbuf();
5     return buffer.str();
6 }
```

这段代码简洁，也适合小文件。但如果文件很大，内存占用会随着文件大小增长；如果只需要逐行处理，一次性读取就没有必要。这个反例推出第一条规则：I/O 设计要先问数据规模和处理方式。需要全局分析时可以读入内存；可以逐步处理时优先流式处理。

14.2 string、string_view 和字符编码边界

`std::string` 拥有一段字节序列。它不自动理解中文字符、Unicode 字符簇或文件编码。下面的 `size()` 返回字节数，不是用户感知的字符数：

Listing 14.2 `string` 的 `size` 是字节数量

```
1 std::string text = "Cpp";
2 std::cout << text.size() << '\n';
```

在本书示例里，我们主要处理 ASCII 命令行、配置和英文单词。遇到中文分词、Unicode 大小写转换、规范化比较，就应该使用专门库，而不是把 `std::tolower` 当成万能工具。

14.3 逐行读取的基本形态

逐行处理日志或配置时，用 `std::getline`：

Listing 14.3 逐行读取文件

```
1 std::vector<std::string> read_lines(const std::filesystem::path& path)
2 {
3     std::ifstream input{path};
4     if (!input) {
5         throw std::runtime_error{"cannot open file"};
6     }
7
8     std::vector<std::string> lines;
9     std::string line;
10    while (std::getline(input, line)) {
11        lines.push_back(line);
12    }
13    if (input.bad()) {
```

```

14     throw std::runtime_error{"failed while reading file"};
15 }
16 return lines;
17 }

```

循环结束不一定是错误。读到 EOF 会让循环停止，这是正常情况。`bad()` 表示底层 I/O 错误，才需要报告。不要把所有流状态都混成“读取失败”。

14.4 解析 key=value 配置

假设置配置文件每行是 `key=value`，空行和 `##` 开头的行忽略。先写一个解析单行的纯函数：

Listing 14.4 解析一行配置

```

1 struct ConfigEntry {
2     std::string key;
3     std::string value;
4 };
5
6 std::optional<ConfigEntry> parse_config_line(std::string_view line)
7 {
8     line = trim(line);
9     if (line.empty() || line.front() == '#') {
10         return std::nullopt;
11     }
12
13     const std::size_t pos = line.find('=');
14     if (pos == std::string_view::npos) {
15         throw std::runtime_error{"missing '=' in config line"};
16     }
17
18     std::string_view key = trim(line.substr(0, pos));
19     std::string_view value = trim(line.substr(pos + 1));
20     if (key.empty()) {
21         throw std::runtime_error{"empty config key"};
22     }
23     return ConfigEntry{std::string{key}, std::string{value}};
24 }

```

这里返回 `std::optional` 是因为空行和注释行不是错误，只是没有条目。缺少等号或 key 为空才是格式错误。这个区别来自需求，而不是语法。

14.5 为什么 trim 返回视图

`trim` 可以不分配新字符串，只返回原文本的子视图：

Listing 14.5 返回子视图的 trim

```

1 std::string_view trim(std::string_view text)
2 {
3     while (!text.empty() && std::isspace(static_cast<unsigned char>(text.front()))) {
4         text.remove_prefix(1);
5     }
6     while (!text.empty() && std::isspace(static_cast<unsigned char>(text.back()))) {
7         text.remove_suffix(1);
8     }
9 }

```

```

8     }
9     return text;
10 }

```

`remove_prefix` 和 `remove_suffix` 只移动视图边界, 不修改原字符串。返回的 `string_view` 仍然依赖原始 `line` 的生命周期。因此在 `parse_config_line` 里, 如果要把结果保存到容器, 就必须复制成 `std::string`。

14.6 输出格式也要测试

输出不是随手 `cout`。如果一个函数负责生成文本, 优先让它写入 `std::ostream&`, 这样测试可以用 `std::ostringstream`:

Listing 14.6 输出到任意流

```

1 void write_entries(std::ostream& output, std::span<const ConfigEntry> entries)
2 {
3     for (const ConfigEntry& entry : entries) {
4         output << entry.key << "=" << entry.value << '\n';
5     }
6 }

```

测试时不需要真实文件:

Listing 14.7 用字符串流测试输出

```

1 std::ostringstream output;
2 std::vector<ConfigEntry> entries{{"port", "8080"}, {"host", "localhost"}};
3 write_entries(output, entries);
4 assert(output.str() == "port=8080\nhost=localhost\n");

```

这就是流抽象的价值: 同一段代码可以写到终端、文件、内存字符串或网络缓冲, 只要它们提供相同的流接口。

14.7 错误报告要带上下文

解析配置时, 只说“格式错误”不够。调用者需要知道哪一行错:

Listing 14.8 读取配置时补充行号

```

1 std::vector<ConfigEntry> read_config_entries(const std::filesystem::path& path)
2 {
3     std::ifstream input{path};
4     if (!input) {
5         throw std::runtime_error{"cannot open config file"};
6     }
7
8     std::vector<ConfigEntry> entries;
9     std::string line;
10    int line_number = 0;
11    while (std::getline(input, line)) {
12        ++line_number;
13        try {
14            std::optional<ConfigEntry> entry = parse_config_line(line);
15            if (entry.has_value()) {

```

```

16         entries.push_back(std::move(*entry));
17     }
18     } catch (const std::exception& error) {
19         throw std::runtime_error{"line " + std::to_string(line_number) +
20                                 ": " + error.what()};
21     }
22 }
23 return entries;
24 }

```

这里捕获异常不是为了吞掉错误，而是补充上下文后继续向上抛。错误处理必须有目的：恢复、转换、补上下文或终止。

习题与作业

扩展配置解析：支持同一个 key 重复时后者覆盖前者，并记录一条警告。要求解析单行、合并条目、输出警告分别是不同函数。写测试覆盖注释、空行、缺等号、空 key、重复 key。

14.8 流状态要分清 EOF 和错误

`std::getline` 循环结束有多种原因。最常见是正常读到 EOF，这不是错误。如果底层设备出错，才是 I/O 错误。可以把读取循环写成固定形态：

Listing 14.9 读取结束后检查 bad 状态

```

1 std::string line;
2 while (std::getline(input, line)) {
3     handle_line(line);
4 }
5
6 if (input.bad()) {
7     throw std::runtime_error{"I/O error while reading"};
8 }

```

不要在循环结束后简单说“读取失败”。空文件、正常 EOF、格式错误、磁盘错误是四种不同情况。错误分类越细，调用者越容易修复。

14.9 大文件处理要控制内存上界

如果只统计日志行数，不需要把所有行放进 `vector`：

Listing 14.10 流式统计行数

```

1 std::size_t count_lines(std::istream& input)
2 {
3     std::size_t count = 0;
4     std::string line;
5     while (std::getline(input, line)) {
6         ++count;
7     }
8     if (input.bad()) {
9         throw std::runtime_error{"failed while counting lines"};
10    }
11    return count;
12 }

```

这个函数的内存上界接近一行文本，而不是整个文件。流式处理的代价是不能随意回看历史数据；如果后续算法需要全局排序或随机访问，就必须收集。设计 I/O 时先问：处理是否能单向完成？是否需要全局视图？内存上限是多少？

14.10 解析单行要保留原始文本

配置错误最好带原始行。比如缺少等号：

Listing 14.11 错误配置行

```
1 port 8080
```

错误消息可以写成：

Listing 14.12 带原始行的错误

```
1 line 12: missing '=': port 8080
```

为此，错误类型可以保存行号和片段：

Listing 14.13 带上下文的行错误

```
1 struct LineError {
2     int line = 0;
3     std::string text;
4     std::string message;
5 };
```

不要只返回 `false`。布尔值说明不了用户该改哪里。文件处理程序的可用性，很大程度取决于错误信息是否能定位。

14.11 编码边界要写进需求

本章示例主要处理 ASCII 风格的配置和命令。如果需求变成“读取用户提交的中文文本，按用户感知字符截断到二十个字”，`std::string::size()` 就不够了。它返回字节数，不是 Unicode 字符数，也不是字形簇数量。正确做法是引入专门文本处理库，或者明确系统只按字节处理。

这不是逃避问题，而是工程边界。标准 `std::string` 是字节容器；文本国际化是更大的主题。项目需求必须写清楚编码假设，否则后面会出现截断乱码、大小写转换错误、排序顺序不符合用户语言等问题。

14.12 文件写入也要有失败检查

很多代码只检查打开，不检查写入过程：

Listing 14.14 只检查打开不够

```
1 std::ofstream output{path};
2 if (!output) {
3     throw std::runtime_error{"cannot open output"};
4 }
5 output << data;
```

磁盘满、权限变化、网络文件系统异常都可能让写入失败。更稳的写法：

Listing 14.15 写入后检查流状态

```
1 std::ofstream output{path};
2 if (!output) {
3     throw std::runtime_error{"cannot open output"};
4 }
5 output << data;
6 if (!output) {
7     throw std::runtime_error{"failed to write output"};
8 }
```

如果写的是重要配置或索引文件，还应结合 RAII 章节的临时文件提交策略，避免失败时留下半截文件。

14.13 本章验收：I/O 函数可替换输入输出

一个文件处理函数如果直接依赖真实文件和终端，就很难测试。优先把核心逻辑写成接收 `std::istream&`、`std::ostream&` 或 `std::span<const std::string>` 的函数；只有最外层负责打开文件。这样同一套逻辑可以用字符串流测试，也可以在真实命令里使用。I/O 边界越薄，程序越容易验证。

第 15 章

对象内存、对齐和数据布局

15.1 问题入口：同样的字段为什么对象大小不同

看两个结构体：

Listing 15.1 字段顺序影响对象大小

```
1 struct A {  
2     char flag;  
3     int count;  
4     char tag;  
5 };  
6  
7 struct B {  
8     int count;  
9     char flag;  
10    char tag;  
11 };
```

它们保存的信息一样，但 `sizeof(A)` 和 `sizeof(B)` 可能不同。原因是对齐。许多平台要求 `int` 放在特定地址边界上，编译器会在字段之间插入填充字节。这个例子说明：对象不是抽象字段列表，它最终要落到内存布局上。

15.2 对齐不是优化细节

对齐让 CPU 更高效地访问数据，有些平台甚至要求未对齐访问会失败。可以观察：

Listing 15.2 观察大小和对齐

```
1 std::cout << sizeof(A) << " " << alignof(A) << '\n';  
2 std::cout << sizeof(B) << " " << alignof(B) << '\n';
```

不要为了省几个字节盲目重排所有结构体。字段顺序也影响可读性和 ABI。只有当对象数量巨大、内存确实成为瓶颈时，才把布局优化作为明确任务。

15.3 结构体布局的账本推导

把结构体布局想成编译器在填一张地址账本。假设某个平台上 `char` 对齐为 1，`int` 对齐为 4。结构体 A 的字段顺序是 `char`、`int`、`char`：

Listing 15.3 按字段顺序推导布局

```
1 struct A {  
2     char flag;    // offset 0  
3     int count;    // must start at an address divisible by 4  
4     char tag;     // after count  
5 };
```

推导过程如下：

1. `flag` 放在偏移 0，占 1 字节。

2. 下一个字段 `count` 要求偏移能被 4 整除。当前偏移是 1，所以编译器补 3 个填充字节，把 `count` 放到偏移 4。
 3. `count` 占 4 字节，结束后偏移到 8。
 4. `tag` 放在偏移 8，占 1 字节。
 5. 整个对象的大小通常还要补齐到最大对齐 4 的倍数，所以末尾再补 3 个字节。
- 这就是为什么 A 常见大小是 12。再看 B：

Listing 15.4 把大对齐字段提前

```
1 struct B {
2     int count;    // offset 0
3     char flag;    // offset 4
4     char tag;     // offset 5
5 };
```

`count` 从偏移 0 开始，天然满足 4 字节对齐；两个 `char` 接在后面，最后只需要把整体大小补到 8。字段没有变，内存账本变了。这个例子推出来的规律是：大对齐字段夹在小字段中间时，容易制造中间填充；把大字段放在一起，常能减少空洞。

但是这个规律不是“永远重排字段”。如果结构体对外暴露为二进制 ABI、写入文件格式、被别的语言绑定，字段顺序就是接口的一部分。优化布局前先问：这个类型是不是内部实现细节？对象数量是否足够多？有没有测量证明内存或缓存是瓶颈？

15.4 栈、堆和对象生命周期

“栈上对象”和“堆上对象”不是对象类型，而是存储位置和生命周期管理方式不同：

Listing 15.5 自动存储期对象

```
1 void run()
2 {
3     std::vector<int> values{1, 2, 3};
4 }
```

`values` 这个 `vector` 对象本身是局部对象，离开作用域析构。但它内部元素通常存放在动态分配的内存里，由 `vector` 管理。不要简单说“`vector` 在栈上所以数据也在栈上”。对象本体和它拥有的资源要分开看。

15.5 `vector` 背后的普通对象形状

`std::vector<int>` 可以近似理解成一个很小的对象，里面保存三类信息：元素起始位置、当前结束位置、容量结束位置。真实标准库实现细节不要求完全长这样，但心智模型很有用：

Listing 15.6 `vector` 的近似本体

```
1 class IntVectorShape {
2 public:
3     int* begin = nullptr;
4     int* end = nullptr;
5     int* capacity_end = nullptr;
6 };
```

因此这段代码里有两块东西：

Listing 15.7 对象本体和元素资源分离

```

1 void run()
2 {
3     std::vector<int> values{1, 2, 3};
4 }

```

`values` 这个本体在当前函数的自动存储期里；元素 1、2、3 通常在 `vector` 申请的动态存储里。离开作用域时，先调用 `values` 的析构函数，再由析构函数释放元素资源。把它还原成普通代码，大概是：

Listing 15.8 RAII 的普通代码形状

```

1 void run_shape()
2 {
3     IntVectorShape values = allocate_and_fill_ints({1, 2, 3});
4     // 使用 values
5     destroy_ints(values);
6 }

```

真实 `std::vector` 把 `destroy_ints` 放进析构函数，所以调用者不用手写清理。这就是 RAII 的价值：资源清理跟对象生命周期绑定，而不是散落在每条返回路径和异常路径上。

15.6 new 和 delete 为什么不该到处写

手写 `new` 和 `delete` 容易在异常路径泄漏：

Listing 15.9 手写 delete 容易遗漏

```

1 User* user = new User{"Ada"};
2 do_something_that_may_throw();
3 delete user;

```

如果中间函数抛异常，`delete` 不会执行。用 `std::unique_ptr`：

Listing 15.10 独占指针自动释放

```

1 auto user = std::make_unique<User>(User{"Ada"});
2 do_something_that_may_throw();

```

离开作用域时，`unique_ptr` 析构释放对象。现代 C++ 不是不能动态分配，而是动态资源应该由对象管理。

15.7 连续布局和指针追逐

性能章节提到过局部性。这里用内存结构再看一次。数组结构：

Listing 15.11 连续点数组

```

1 std::vector<Point> points;
2 for (const Point& point : points) {
3     total += point.x;
4 }

```

链式结构：

Listing 15.12 节点指针追逐

```

1 for (const Node* node = head; node != nullptr; node = node->next) {
2     total += node->value;
3 }

```

两者都是线性遍历，但连续数组更容易被 CPU 预取。链表每一步都要读取下一个地址，节点可能分散在内存各处。选择数据结构时，不能只看大 O，也要看数据如何摆放。

15.8 对象表示和未定义行为

C++ 允许你查看对象的字节表示：

Listing 15.13 查看对象字节

```

1 int value = 42;
2 const auto* bytes = reinterpret_cast<const unsigned char*>(&value);
3 for (std::size_t i = 0; i < sizeof(value); ++i) {
4     std::cout << static_cast<int>(bytes[i]) << '\n';
5 }

```

但这并不意味着可以随意把任意字节解释成任意对象。类型别名规则、对齐、对象生命周期都会影响是否合法。入门阶段记住保守规则：序列化和解析二进制格式时，不要直接把网络字节流 `reinterpret_cast` 成结构体；应该逐字段读写，明确大小端和字段长度。

15.9 为什么不能把协议头直接转成结构体

很多初学者会想：协议头不是正好有字段吗？那就定义结构体，然后强转：

Listing 15.14 把字节流强转成结构体的问题

```

1 struct WireHeader {
2     char magic[4];
3     std::uint16_t version;
4 };
5
6 const auto* header = reinterpret_cast<const WireHeader*>(bytes.data());

```

这个写法看起来省事，但问题有四层：

1. 长度不一定够。只有 `bytes.data()`，不能证明后面至少有 `sizeof(WireHeader)` 字节。
2. 对齐不一定满足。字节缓冲区起始地址可能不适合直接读取 `std::uint16_t`。
3. 填充字节不受协议控制。结构体里的字段之间可能有 padding，而协议通常按紧凑字节定义。
4. 字节序不明确。网络协议常用大端，本机可能是小端。

所以解析二进制协议时，安全模型不是“这段内存像结构体”，而是“我从第几个字节读几位，按协议规则组装成值”。这也是从底层推导出的工程规则：内存布局服务于本机对象，协议布局服务于跨机器交换，两者不能混为一谈。

15.10 完整案例：二进制头解析

假设协议头是 4 字节魔数加 2 字节版本。不要直接映射结构体：

Listing 15.15 逐字段解析协议头

```

1 struct Header {
2     std::array<unsigned char, 4> magic{};
3     std::uint16_t version = 0;

```

```
4 };
5
6 std::optional<Header> parse_header(std::span<const unsigned char> bytes)
7 {
8     if (bytes.size() < 6) {
9         return std::nullopt;
10    }
11    Header header;
12    std::copy_n(bytes.begin(), 4, header.magic.begin());
13    header.version = static_cast<std::uint16_t>(bytes[4]) << 8 |
14                    static_cast<std::uint16_t>(bytes[5]);
15    return header;
16 }
```

这里明确使用大端解析版本号。代码长一点，但跨平台行为清楚。二进制处理最怕“我机器上正好可以”。

习题与作业

写两个字段顺序不同的结构体，打印 `sizeof` 和 `alignof`。再设计一个解析 8 字节消息头的函数，要求不用 `reinterpret_cast`，逐字段解析并检查长度。

第零部分

泛型、标准库和可组合设计

第 16 章

模板、concept 和泛型设计

16.1 问题入口：同一段逻辑支持不同数字类型

假设要写一个求平均值函数。只支持 `std::vector<int>` 很窄：

Listing 16.1 只能处理 int vector

```
1 double average(const std::vector<int>& values)
2 {
3     int sum = 0;
4     for (int value : values) {
5         sum += value;
6     }
7     return static_cast<double>(sum) / static_cast<double>(values.size());
8 }
```

如果输入是 `double`、`long long` 或 `std::array<int, 4>`，这段函数就不能复用。模板让我们把“类型差异”变成参数。

16.2 函数模板

Listing 16.2 用 span 写数字平均值

```
1 #include <concepts>
2 #include <numeric>
3 #include <span>
4
5 template <std::floating_point Result = double, typename Number>
6 Result average(std::span<const Number> values)
7 {
8     if (values.empty()) {
9         return Result{};
10    }
11    const Number sum = std::accumulate(values.begin(), values.end(), Number{});
12    return static_cast<Result>(sum) / static_cast<Result>(values.size());
13 }
```

这个模板还有局限：`Number` 应该支持加法和默认构造。C++20 的 `concept` 可以把约束写出来。

16.3 用 concept 写约束

Listing 16.3 定义可累加数字概念

```
1 #include <concepts>
2
3 template <typename T>
4 concept AddableNumber = std::default_initializable<T> &&
5     requires(T lhs, T rhs) {
6         { lhs + rhs } -> std::convertible_to<T>;
7     }
```

```

7         };
8
9     template <std::floating_point Result = double, AddableNumber Number>
10    Result checked_average(std::span<const Number> values)
11    {
12        if (values.empty()) {
13            return Result{};
14        }
15        Number sum{};
16        for (const Number& value : values) {
17            sum = sum + value;
18        }
19        return static_cast<Result>(sum) / static_cast<Result>(values.size());
20    }

```

约束不是为了炫技，而是为了把错误提前、把报错变短、把接口意图写清楚。

16.4 模板不是复制粘贴

模板会在使用点实例化。不同类型会生成不同实例，编译时间和错误信息都可能变重。因此模板代码要更克制：不要把巨大业务逻辑塞进模板；能用普通函数处理的部分，放到普通函数里。

Listing 16.4 模板只负责薄薄的类型适配

```

1 double average_vector(const std::vector<int>& values)
2 {
3     return checked_average(std::span<const int>{values});
4 }

```

真实项目中，可以把模板放在接口层，把复杂实现放进非模板函数，减少编译膨胀。

16.5 模板报错怎么读

模板报错常常很长，因为编译器会把实例化链条全部打印出来。读这种报错不要从第一行硬啃到最后一行，而是按三步：

1. 找到你自己的文件路径和行号。
2. 找到第一个“不满足约束”或“没有匹配函数”的位置。
3. 回到模板接口，看传入类型缺少哪种能力。

例如把 `std::string` 传给只接受数值累加的模板，错误本质不是“编译器讨厌字符串”，而是字符串加法结果、默认值、除法转换等能力不符合平均值函数的合同。concept 的价值就是把这种合同前移，让错误更靠近调用点。

16.6 类模板：固定容量缓冲区

Listing 16.5 简单固定容量缓冲区

```

1 #include <array>
2 #include <optional>
3
4 template <typename T, std::size_t CAPACITY>
5 class RingBuffer {
6 public:
7     [[nodiscard]] bool empty() const noexcept
8     {

```



```

9      return this->size_ == 0;
10  }
11
12  [[nodiscard]] bool full() const noexcept
13  {
14      return this->size_ == CAPACITY;
15  }
16
17  bool push(const T& value)
18  {
19      if (this->full()) {
20          return false;
21      }
22      this->data_[this->tail_] = value;
23      this->tail_ = (this->tail_ + 1) % CAPACITY;
24      ++this->size_;
25      return true;
26  }
27
28  std::optional<T> pop()
29  {
30      if (this->empty()) {
31          return std::nullopt;
32      }
33      T value = this->data_[this->head_];
34      this->head_ = (this->head_ + 1) % CAPACITY;
35      --this->size_;
36      return value;
37  }
38
39 private:
40     std::array<T, CAPACITY> data_{};
41     std::size_t head_ = 0;
42     std::size_t tail_ = 0;
43     std::size_t size_ = 0;
44 };

```

这个例子展示了非类型模板参数 `CAPACITY`。容量成为类型的一部分，`RingBuffer<int, 8>` 和 `RingBuffer<int, 16>` 是不同类型。

核心思想

泛型设计的目标不是让一段代码接受所有类型，而是让它接受满足某组明确能力的类型。
concept 就是在接口上写出这组能力。

16.7 从重复函数推导模板

模板不是一上来就该用。先看重复出现在哪里：

Listing 16.6 重复的平均值函数

```

1 double average_ints(std::span<const int> values);
2 double average_doubles(std::span<const double> values);
3 double average_longs(std::span<const long long> values);

```

这三个函数的控制流程相同，只有元素类型不同。于是可以把元素类型提成模板参数。反过来，如果两个函数只是名字相似，但业务规则、错误处理、返回含义都不同，就不应该硬做模板。

16.8 concept 从失败调用里长出来

如果没有约束，下面的调用可能触发很长的模板报错：

Listing 16.7 错误类型不满足平均值语义

```
1 std::vector<std::string> names{"Ada", "Grace"};
2 const double result = checked_average(std::span<const std::string>{names});
```

字符串可以默认构造，也能相加，但不能自然地除以元素个数并转换成浮点平均值。这个反例说明 `AddableNumber` 还不够严格。可以把“可转换成结果类型”也写进约束：

Listing 16.8 更贴近平均值语义的约束

```
1 template <typename T, typename Result>
2 concept AveragableAs = std::default_initializable<T> &&
3     requires(T lhs, T rhs, std::size_t count) {
4         { lhs + rhs } -> std::convertible_to<T>;
5         { static_cast<Result>(lhs) / static_cast<Result>(count) }
6         -> std::convertible_to<Result>;
7     };
```

约束来自真实失败样本，而不是为了把概念写得复杂。每加一个约束，都应该能说出它挡住了哪类误用。

16.9 类模板里的容量边界

`RingBuffer<T, CAPACITY>` 还有一个遗漏：`CAPACITY` 为 0 时，取模会出问题。非类型模板参数也需要约束：

Listing 16.9 用 `requires` 排除零容量

```
1 template <typename T, std::size_t CAPACITY>
2     requires(CAPACITY > 0)
3 class RingBuffer {
4     // ...
5 };
```

这个约束把错误提前到编译期。如果用户写 `RingBuffer<int, 0>`，问题不是运行到“对零取模”才暴露，而是在类型实例化时就被拒绝。模板的一个优势就是把一部分不变量放到类型层。

16.10 模板实现如何避免膨胀

假设一个模板函数里有大段日志、文件处理和复杂业务逻辑。每个类型实例都会生成一份代码，编译也更慢。常见做法是把与类型无关的部分抽到普通函数：

Listing 16.10 模板层只做类型相关转换

```
1 void report_average_result(double value);
2
3 template <typename Number>
4 void report_average(std::span<const Number> values)
5 {
```

```

6   const double value = checked_average(values);
7   report_average_result(value);
8 }

```

这样模板只负责把不同数字类型汇总成 `double`，报告格式等无关逻辑只有一份。泛型代码越靠近基础设施，越要注意编译时间、错误信息和二进制体积。

习题与作业

给 `RingBuffer` 补两个测试：容量为 2 时连续 `push` 三次，第三次应失败；连续 `pop` 三次，第三次应返回 `std::nullopt`。再尝试声明 `RingBuffer<int, 0>`，确认编译期约束生效。

16.11 模板背后的实例化模型

模板也要用“普通代码形状”理解。下面这个函数模板：

Listing 16.11 函数模板

```

1 template <typename T>
2 T max_value(T lhs, T rhs)
3 {
4     return lhs < rhs ? rhs : lhs;
5 }

```

当你写：

Listing 16.12 两个调用点

```

1 int a = max_value(1, 2);
2 double b = max_value(1.5, 2.5);

```

可以近似理解为编译器生成了两个普通函数：

Listing 16.13 近似生成的普通函数

```

1 int max_value_int(int lhs, int rhs)
2 {
3     return lhs < rhs ? rhs : lhs;
4 }
5
6 double max_value_double(double lhs, double rhs)
7 {
8     return lhs < rhs ? rhs : lhs;
9 }

```

模板不是运行时拿着一个“任意类型”在跑。它是在编译期按具体类型生成代码。这解释了三个现象：模板定义通常要放头文件；模板错误常在使用点爆出；模板过大可能增加编译时间和二进制体积。

16.12 concept 背后是编译期门禁

给模板加 `concept`，可以理解为在生成具体函数前先检查类型能力：

Listing 16.14 有约束的模板

```

1 template <std::totally_ordered T>

```

```
2 T median(std::span<const T> values);
```

近似的普通语言描述是：只有当 `T` 支持完整排序关系时，编译器才允许实例化这个模板。它不是运行时 `if`，不会在程序启动后检查；它是编译期门禁。传入不能排序的类型，程序根本不生成对应函数。

16.13 从调用点推导模板参数

模板参数通常由调用点推导：

Listing 16.15 模板参数推导

```
1 auto value = max_value(1, 2); // T 推导为 int
```

但这句会失败：

Listing 16.16 两个实参推导出不同类型

```
1 auto value = max_value(1, 2.0);
```

第一个实参希望 `T` 是 `int`，第二个希望 `T` 是 `double`。编译器不会随便替你决定。你可以显式指定：

Listing 16.17 显式指定模板参数

```
1 auto value = max_value<double>(1, 2.0);
```

这时 `1` 会转换成 `double`。这个例子说明模板推导不是“自动变聪明”，它有严格规则。遇到模板报错时，先问：每个参数把模板参数推成了什么？

16.14 模板和普通函数如何配合

如果模板里有大量与类型无关的逻辑，可以把它移到普通函数。再给一个更完整的例子：格式化错误报告时，只有“把值转成字符串”与类型相关，其余拼接都无关。

Listing 16.18 非模板函数承接共同逻辑

```
1 std::string make_error_message(std::string_view field,
2                               std::string_view value_text)
3 {
4     std::string message;
5     message.reserve(field.size() + value_text.size() + 32);
6     message += "invalid ";
7     message += field;
8     message += ": ";
9     message += value_text;
10    return message;
11 }
12
13 template <typename T>
14 std::string make_typed_error(std::string_view field, const T& value)
15 {
16     return make_error_message(field, to_debug_string(value));
17 }
```

这样模板层很薄，实例化负担小，报错也更集中。泛型设计不是把所有东西都模板化，而是把真正依赖类型的部分模板化。

16.15 类模板展开也会生成不同类型

RingBuffer<int, 8> 和 RingBuffer<int, 16> 不是同一个类型。近似理解是：

Listing 16.19 不同容量生成不同类

```
1 class RingBufferInt8 {
2     std::array<int, 8> data_;
3     // ...
4 };
5
6 class RingBufferInt16 {
7     std::array<int, 16> data_;
8     // ...
9 };
```

这就是为什么容量能参与编译期检查，也解释了为什么不同容量的缓冲区不能直接互相赋值。容量不只是成员值，它已经进入类型身份。

16.16 模板错误诊断练习

当模板报错很长时，按这个模板账本定位：

问题	要看哪里	例子
推导失败	调用点实参类型	int 和 double 混传
约束失败	concept 条件	类型不能排序或不能相加
实例化失败	模板函数体	类型没有 size() 成员
链接失败	定义是否可见	模板定义只放在 .cpp

模板不是黑箱。把它还原成“按类型生成普通代码”的模型后，报错就能逐层拆开。

第 17 章

标准算法、ranges 和可组合代码

17.1 问题入口：筛选活跃用户

给定一组用户，筛选活跃用户 ID，并按 ID 排序。朴素写法通常是一堆循环：

Listing 17.1 手写循环筛选和排序

```
1 struct User {
2     int id = 0;
3     bool active = false;
4 };
5
6 std::vector<int> active_ids;
7 for (const User& user : users) {
8     if (user.active) {
9         active_ids.push_back(user.id);
10    }
11 }
12 std::sort(active_ids.begin(), active_ids.end());
```

这段代码没错。标准算法不是为了消灭所有循环，而是把常见意图变成可读的词汇。

17.2 算法表达意图

Listing 17.2 标准算法表达筛选

```
1 std::vector<User> active_users;
2 std::copy_if(users.begin(), users.end(), std::back_inserter(active_users),
3             [](const User& user) {
4                 return user.active;
5             });
```

如果最终只要 ID，还需要一次转换。C++20 ranges 可以写成管道式视图：

Listing 17.3 ranges 视图组合

```
1 #include <ranges>
2
3 auto active_id_view =
4     users
5     | std::views::filter([](const User& user) { return user.active; })
6     | std::views::transform([](const User& user) { return user.id; });
7
8 std::vector<int> ids(active_id_view.begin(), active_id_view.end());
9 std::ranges::sort(ids);
```

视图通常是惰性的：它描述计算，不立即生成新容器。最终构造 `ids` 时才真正遍历。

17.3 谓词和投影

`std::ranges::sort` 支持投影，可以直接按字段排序：

Listing 17.4 按字段排序

```
1 std::ranges::sort(users, std::less<>{}, &User::id);
```

这比手写比较器更短，也更不容易写错。比较器仍然有用，尤其当排序规则涉及多个字段。

17.4 避免悬空视图

`ranges` 的危险点是生命周期。视图不拥有底层数据，如果底层容器销毁，视图就悬空。

Listing 17.5 不要返回依赖局部容器的视图

```
1 auto bad_view()
2 {
3     std::vector<User> users = load_users();
4     return users | std::views::filter([](const User& user) {
5         return user.active;
6     });
7 }
```

正确做法是返回拥有结果的容器，或者让调用者传入数据并保证生命周期。

17.5 什么时候保留普通循环

普通循环仍然重要。涉及复杂状态机、多个输出、短路返回、详细错误上下文时，强行使用算法反而会降低可读性。比如解析协议时，一个明确的 `for` 或 `while` 循环通常比嵌套算法清楚。

核心思想

标准算法和 `ranges` 的价值是表达常见意图：查找、计数、转换、筛选、排序、归约。不是所有循环都要替换，但每个简单循环都值得问一句：这里是不是已有标准词汇？

17.6 从循环变量推导算法词汇

判断一个循环能不能换成标准算法，可以先问它在维护什么状态。下面的循环只维护一个布尔结果：

Listing 17.6 手写检查是否存在活跃用户

```
1 bool has_active = false;
2 for (const User& user : users) {
3     if (user.active) {
4         has_active = true;
5         break;
6     }
7 }
```

这正是 `std::ranges::any_of` 的语义：

Listing 17.7 标准算法表达是否存在

```
1 const bool has_active = std::ranges::any_of(users, [](const User& user) {
2     return user.active;
3 });
```

如果循环维护的是计数，用 `count_if`；维护的是查找位置，用 `find_if`；维护的是转换结果，用 `transform` 或视图。算法不是为了显得高级，而是让读者少读循环细节，直接看到意图。

17.7 视图是借来的计算

`ranges` 视图通常不拥有数据。可以把它理解成一张“计算计划单”：等你遍历它时，它才从原容器取元素、过滤、转换。这个特性节省中间容器，但也带来生命周期要求。

Listing 17.8 视图会反映底层容器变化

```
1 std::vector<User> users{{1, true}, {2, false}};
2 auto active = users | std::views::filter([](const User& user) {
3     return user.active;
4 });
5
6 users.push_back(User{3, true});
7 // 后续遍历 active 时，可能看到新加入的用户
```

这段代码是否安全，还要看 `push_back` 是否导致底层迭代器失效、视图对象如何保存范围。入门阶段先保守：视图不要跨越会修改底层容器结构的代码段长期保存。需要稳定结果时，立刻收集到 `vector`。

17.8 排序规则必须是严格弱序

比较器不是随便返回布尔值。排序要求比较关系稳定、可传递。反例：

Listing 17.9 错误比较器破坏排序语义

```
1 std::ranges::sort(users, [](const User& lhs, const User& rhs) {
2     return lhs.id <= rhs.id; // 错误：相等时也返回 true
3 });
```

当 `lhs.id == rhs.id` 时，`lhs < rhs` 和 `rhs < lhs` 都不应该成立。正确写法是：

Listing 17.10 相等时返回 false

```
1 std::ranges::sort(users, [](const User& lhs, const User& rhs) {
2     return lhs.id < rhs.id;
3 });
```

多字段排序也要按顺序推导：先比较主字段，主字段相等时再比较次字段。

17.9 完整案例：活跃用户报表

把本章内容合成一个小函数：输入用户列表，输出活跃用户 ID，按 ID 升序，不修改原列表。

Listing 17.11 活跃用户 ID 报表

```
1 std::vector<int> active_user_ids(std::span<const User> users)
2 {
3     auto ids_view = users
4         | std::views::filter([](const User& user) {
5             return user.active;
6         })
7         | std::views::transform([](const User& user) {
```



```

8         return user.id;
9     });
10
11     std::vector<int> ids(ids_view.begin(), ids_view.end());
12     std::ranges::sort(ids);
13     ids.erase(std::ranges::unique(ids).begin(), ids.end());
14     return ids;
15 }

```

这里先用视图表达筛选和转换，再收集成拥有结果的 `vector`，最后排序去重。返回值拥有自己的生命周期，不依赖传入的 `users`。

习题与作业

给 `active_user_ids` 加测试：空列表、没有活跃用户、重复 ID、已排序输入、逆序输入。然后用普通循环重写一版，对比哪一版更容易在代码里看出“筛选、转换、排序、去重”四个步骤。

17.10 从循环账本选择算法

判断一个循环是否适合换成算法，可以把循环账本写出来：

循环维护的东西	标准词汇	典型问题
是否存在某元素	<code>any_of</code>	谓词是否表达完整条件
所有元素是否满足	<code>all_of</code>	空区间时语义是否符合需求
找到第一个匹配	<code>find_if</code>	找不到时如何处理
把每个元素变成新值	<code>transform</code>	输出容器是否预留空间
只保留一部分元素	<code>copy_if</code> 或 视图过滤	是否需要拥有结果
把元素合成一个值	<code>accumulate</code> 或 <code>reduce</code>	操作是否满足结合律

这张表能避免两个极端：一端是所有简单循环都手写，读者要自己推意图；另一端是为了“现代”把复杂状态机硬塞进算法，反而难懂。算法的边界是：当循环账本正好对应一个标准词汇时，用它。

17.11 归约不是随便相加

`std::accumulate` 看起来简单，但初值类型会影响结果。错误版本：

Listing 17.12 初值类型导致截断或溢出

```

1 std::vector<double> values{0.1, 0.2, 0.3};
2 auto total = std::accumulate(values.begin(), values.end(), 0);

```

初值 `0` 是 `int`，归约结果会按整数累加路径推导，结果不是你想要的。应该写：

Listing 17.13 初值类型表达归约类型

```

1 auto total = std::accumulate(values.begin(), values.end(), 0.0);

```

对大整数也一样。如果元素是 `int`，但总和可能超过 `int`，初值应使用更宽类型。算法没有取消类型责任，它只是把循环形态变成标准接口。

17.12 ranges 管道的调试方法

`ranges` 管道写长后，新手会觉得像魔法。调试时不要盯着整条管道，按阶段拆：

Listing 17.14 把管道拆成命名阶段

```

1 auto active_users = users | std::views::filter([](const User& user) {
2     return user.active;
3 });
4
5 auto ids = active_users | std::views::transform([](const User& user) {
6     return user.id;
7 });
8
9 std::vector<int> result(ids.begin(), ids.end());

```

每个阶段都有一个问题：过滤条件是否正确，转换字段是否正确，最终是否需要拥有容器。拆开以后再合并成管道，代码就不是模板，而是从证据一步步压缩出来的表达。

17.13 稳定排序与业务语义

排序时要问：相同 key 的元素原顺序是否有意义？例如用户列表先按注册时间排列，现在要把活跃用户排在前面，但同为活跃的用户仍保留注册先后。普通 `sort` 不保证相等元素的原顺序，应该使用稳定排序：

Listing 17.15 稳定排序保留相等元素原相对顺序

```

1 std::ranges::stable_sort(users, [](const User& lhs, const User& rhs) {
2     return lhs.active && !rhs.active;
3 });

```

这个比较器只表达“活跃用户排在非活跃用户前”。两个都活跃或两个都不活跃时返回 `false`，稳定排序就保留原相对顺序。如果业务不关心相等元素顺序，普通排序可能更合适。选择算法也要写清业务语义。

17.14 完整案例：报表流水线和验收

假设报表需求变成：从用户列表中找出活跃且分数不低于阈值的用户，按分数降序、ID 升序输出前十个 ID。先不要写代码，先拆步骤：

1. 过滤活跃用户。
2. 过滤分数达到阈值的用户。
3. 收集成拥有结果，因为后面要排序和截断。
4. 按分数降序、ID 升序排序。
5. 截断到十个。
6. 转换成 ID 列表。

实现可以这样写：

Listing 17.16 活跃高分用户报表

```

1 std::vector<int> top_active_user_ids(std::span<const User> users,
2                                     int min_score)
3 {
4     auto candidates_view = users
5         | std::views::filter([](const User& user) {
6             return user.active;
7         })
8         | std::views::filter([min_score](const User& user) {
9             return user.score >= min_score;

```

```
10     });
11
12     std::vector<User> candidates(candidates_view.begin(), candidates_view.end());
13     std::ranges::sort(candidates, [](const User& lhs, const User& rhs) {
14         if (lhs.score != rhs.score) {
15             return lhs.score > rhs.score;
16         }
17         return lhs.id < rhs.id;
18     });
19
20     if (candidates.size() > 10) {
21         candidates.resize(10);
22     }
23
24     std::vector<int> ids;
25     ids.reserve(candidates.size());
26     std::ranges::transform(candidates, std::back_inserter(ids), &User::id);
27     return ids;
28 }
```

这里没有为了管道而管道。需要排序时，必须先拥有一份候选列表；需要返回 ID 时，再转换成结果。标准算法、ranges 和普通容器在同一个函数里协作，才是可维护的现代 C++。

第 18 章

lambda、函数对象和可调用设计

18.1 问题入口：排序规则从哪里来

给用户列表排序时，排序规则常常不是固定的。按 ID 升序、按姓名升序、按活跃状态优先，都是合理需求。最直接的写法是为每个规则写一个函数：

Listing 18.1 普通比较函数

```
1 bool by_id(const User& lhs, const User& rhs)
2 {
3     return lhs.id < rhs.id;
4 }
5
6 std::ranges::sort(users, by_id);
```

这个版本没问题。但如果排序规则只在一处使用，单独起一个函数名可能让代码跳来跳去。lambda 提供了就地定义小函数的方式：

Listing 18.2 lambda 就地表达比较规则

```
1 std::ranges::sort(users, [](const User& lhs, const User& rhs) {
2     return lhs.id < rhs.id;
3 });
```

lambda 的价值不是“写得短”，而是让局部规则贴近使用位置。读者看到排序调用时，就能看到排序规则。

18.2 捕获：lambda 如何拿到外部变量

假设要筛选分数大于阈值的学生。阈值来自外部变量：

Listing 18.3 按值捕获阈值

```
1 int threshold = 60;
2 auto passed = [threshold](const Student& student) {
3     return student.score >= threshold;
4 };
```

`[threshold]` 表示把当前 `threshold` 的值复制进 lambda 对象。之后外部变量变化，不影响这个 lambda 已经保存的值。按引用捕获则不同：

Listing 18.4 按引用捕获阈值

```
1 int threshold = 60;
2 auto passed = [&threshold](const Student& student) {
3     return student.score >= threshold;
4 };
5 threshold = 80;
```

这时 lambda 每次使用的是外部 `threshold` 当前值。按引用捕获更灵活,也更危险:如果 lambda 活得比被捕获变量更久,就会悬空。

18.3 捕获生命周期反例

下面的函数返回一个 lambda,但它引用了局部变量:

Listing 18.5 返回引用局部变量的 lambda 是错误的

```
1 auto make_bad_filter()
2 {
3     int threshold = 60;
4     return [&threshold](const Student& student) {
5         return student.score >= threshold;
6     };
7 }
```

函数返回后, `threshold` 已销毁。返回的 lambda 保存的是悬空引用。正确做法是按值捕获:

Listing 18.6 返回 lambda 时按值保存状态

```
1 auto make_score_filter(int threshold)
2 {
3     return [threshold](const Student& student) {
4         return student.score >= threshold;
5     };
6 }
```

这个反例推出捕获规则: lambda 不逃出当前作用域时,引用捕获可以很方便; lambda 要保存、返回、放进异步任务或跨线程使用时,优先按值捕获必要状态。

18.4 函数对象: 有名字的可调用类型

lambda 本质上会生成一个匿名函数对象。你也可以自己写函数对象:

Listing 18.7 带状态的函数对象

```
1 class ScoreAtLeast {
2 public:
3     explicit ScoreAtLeast(int threshold)
4         : threshold_{threshold}
5     {
6     }
7
8     [[nodiscard]] bool operator()(const Student& student) const noexcept
9     {
10         return student.score >= this->threshold_;
11     }
12
13 private:
14     int threshold_ = 0;
15 };
```

使用方式和 lambda 一样:

Listing 18.8 函数对象作为谓词

```

1 const auto passed = ScoreAtLeast{60};
2 const bool ok = passed(student);

```

什么时候写函数对象？当规则需要复用、需要命名、需要多个成员函数、或者测试这个规则本身时，具名类型比一个很长的 lambda 更清楚。

18.5 std::function 的边界

std::function 可以保存任意满足签名的可调用对象：

Listing 18.9 用 std::function 保存回调

```

1 using Filter = std::function<bool(const Student&)>;
2
3 std::vector<Student> filter_students(std::span<const Student> students, const Filter& ←
   ↪ filter)
4 {
5     std::vector<Student> result;
6     for (const Student& student : students) {
7         if (filter(student)) {
8             result.push_back(student);
9         }
10    }
11    return result;
12 }

```

std::function 的好处是接口稳定，可以在运行期保存不同类型的 callable。代价是类型擦除可能带来间接调用和分配。热路径上，如果规则类型在编译期已知，可以用模板：

Listing 18.10 模板接受任意谓词

```

1 template <typename Filter>
2 std::vector<Student> filter_students_fast(std::span<const Student> students, Filter ←
   ↪ filter)
3 {
4     std::vector<Student> result;
5     for (const Student& student : students) {
6         if (filter(student)) {
7             result.push_back(student);
8         }
9     }
10    return result;
11 }

```

两者没有绝对优劣。需要运行期多态和稳定 ABI 时，**std::function** 合理；需要极低开销和内联机会时，模板更合适。

18.6 回调接口不要制造隐藏所有权

如果回调捕获了对象指针，就要考虑对象生命周期：

Listing 18.11 回调捕获裸指针的风险

```

1 class Button {
2 public:

```

```

3   void on_click(std::function<void()> callback)
4   {
5       this->callback_ = std::move(callback);
6   }
7
8 private:
9     std::function<void()> callback_;
10 };

```

如果调用者写：

Listing 18.12 可能悬空的 this 捕获

```

1 button.on_click([this] {
2     this->save();
3 });

```

那么必须保证回调执行时当前对象还活着。GUI、异步任务、线程池中，这个保证经常很难。可以考虑捕获 `std::weak_ptr`，执行前尝试锁定；或者让拥有者负责在析构前注销回调。回调设计本质上是生命周期设计。

18.7 完整案例：可配置验证流水线

假设要验证注册用户：用户名合法、年龄达到要求、邮箱域名允许。每个规则都是一个谓词：

Listing 18.13 验证规则流水线

```

1 struct Registration {
2     std::string username;
3     int age = 0;
4     std::string email;
5 };
6
7 using Rule = std::function<bool(const Registration&);>;
8
9 bool validate_registration(const Registration& registration, std::span<const Rule> ↵
    ↵ rules)
10 {
11     for (const Rule& rule : rules) {
12         if (!rule(registration)) {
13             return false;
14         }
15     }
16     return true;
17 }

```

组装规则：

Listing 18.14 组装带状态规则

```

1 std::vector<Rule> rules;
2 rules.push_back([](const Registration& item) {
3     return is_valid_username(item.username);
4 });
5 rules.push_back([](const Registration& item) {

```

```

6     return item.age >= 18;
7 });
8
9 std::string allowed_domain = "@example.com";
10 rules.push_back([allowed_domain](const Registration& item) {
11     return item.email.ends_with(allowed_domain);
12 });

```

这里 `allowed_domain` 按值捕获，因为规则会保存在 `rules` 里，可能比局部变量活得更久。这个小案例把 lambda、捕获、`std::function` 和生命周期放到同一个场景里。

习题与作业

给验证流水线增加错误消息，不要只返回 `bool`。设计 `RuleResult`，让每条规则失败时返回原因。要求规则仍然可组合，验证函数返回第一条错误或全部错误列表。

18.8 lambda 背后的普通代码形状

只会写 lambda 还不够。要真正理解它，必须知道编译器大致会把它变成什么。先看一个不捕获任何变量的 lambda：

Listing 18.15 不捕获变量的 lambda

```

1 auto by_id = [](const User& lhs, const User& rhs) {
2     return lhs.id < rhs.id;
3 };

```

可以把它理解成一个匿名类对象，里面有一个调用运算符：

Listing 18.16 近似展开成函数对象

```

1 class ByIdComparator {
2 public:
3     [[nodiscard]] bool operator()(const User& lhs,
4                                   const User& rhs) const
5     {
6         return lhs.id < rhs.id;
7     }
8 };
9
10 ByIdComparator by_id;

```

`std::ranges::sort(users, by_id)` 并不关心 `by_id` 是普通函数、lambda 还是手写函数对象；它只关心这个对象能不能被调用，并返回排序所需的布尔结果。这就是“可调用对象”的统一模型。

18.9 按值捕获就是成员变量副本

再看按值捕获：

Listing 18.17 按值捕获阈值

```

1 int threshold = 60;
2 auto passed = [threshold](const Student& student) {
3     return student.score >= threshold;
4 };

```


近似展开：

Listing 18.18 按值捕获展开为成员副本

```

1 class PassedByValue {
2 public:
3     explicit PassedByValue(int threshold)
4         : threshold_{threshold}
5     {
6     }
7
8     [[nodiscard]] bool operator()(const Student& student) const
9     {
10         return student.score >= this->threshold_;
11     }
12
13 private:
14     int threshold_ = 0;
15 };
16
17 PassedByValue passed{threshold};

```

这解释了为什么外部 `threshold` 后续改变，不影响已经创建好的 `passed`。lambda 对象里保存的是一份成员副本，不是名字本身。

18.10 按引用捕获就是成员引用

按引用捕获：

Listing 18.19 按引用捕获阈值

```

1 int threshold = 60;
2 auto passed = [&threshold](const Student& student) {
3     return student.score >= threshold;
4 };

```

近似展开：

Listing 18.20 按引用捕获展开为引用成员

```

1 class PassedByReference {
2 public:
3     explicit PassedByReference(int& threshold)
4         : threshold_{threshold}
5     {
6     }
7
8     [[nodiscard]] bool operator()(const Student& student) const
9     {
10         return student.score >= this->threshold_;
11     }
12
13 private:
14     int& threshold_;
15 };

```

```

16
17 PassedByReference passed{threshold};

```

现在风险就清楚了：如果 `passed` 活得比 `threshold` 更久，它内部的引用成员就悬空。所谓“lambda 生命周期问题”，本质就是普通对象成员引用的生命周期问题。

18.11 mutable lambda 是非 const 调用运算符

默认情况下，lambda 的 `operator()` 是 `const`，不能修改按值捕获的成员副本。要修改副本，需要 `mutable`：

Listing 18.21 mutable lambda 修改自己的副本

```

1 int count = 0;
2 auto next = [count]() mutable {
3     ++count;
4     return count;
5 };

```

近似展开：

Listing 18.22 mutable 展开成非 const 调用

```

1 class Counter {
2 public:
3     explicit Counter(int count)
4         : count_{count}
5     {
6     }
7
8     int operator()()
9     {
10         ++this->count_;
11         return this->count_;
12     }
13
14 private:
15     int count_ = 0;
16 };

```

注意外部 `count` 不会变。变的是 lambda 对象自己的 `count_` 成员。把这个展开模型记住，就不会把 `mutable` 误解成“允许修改外部变量”。

18.12 泛型 lambda 背后是模板调用运算符

C++14 起，lambda 参数可以写 `auto`：

Listing 18.23 泛型 lambda

```

1 auto less_by_size = [](const auto& lhs, const auto& rhs) {
2     return lhs.size() < rhs.size();
3 };

```

近似展开：

Listing 18.24 泛型 lambda 展开成模板调用运算符

```

1 class LessBySize {
2 public:
3     template <typename Lhs, typename Rhs>
4     [[nodiscard]] bool operator()(const Lhs& lhs,
5                                   const Rhs& rhs) const
6     {
7         return lhs.size() < rhs.size();
8     }
9 };

```

这也解释了泛型 lambda 的报错：如果传入的类型没有 `size()`，错误会出现在模板实例化时。它不是神秘特性，只是编译器生成了一个带模板 `operator()` 的类。

18.13 捕获 this 的真实含义

成员函数里常写：

Listing 18.25 捕获 this

```

1 this->button_.on_click([this] {
2     this->save();
3 });

```

近似意思是 lambda 对象里保存了当前对象的指针：

Listing 18.26 this 捕获近似展开

```

1 class SaveCallback {
2 public:
3     explicit SaveCallback(Editor* self)
4         : self_{self}
5     {
6     }
7
8     void operator()() const
9     {
10         this->self_->save();
11     }
12
13 private:
14     Editor* self_ = nullptr;
15 };

```

如果回调执行时 `Editor` 已经销毁，`self_` 就是悬空指针。异步任务、GUI 回调、线程池任务里，捕获 `this` 必须非常谨慎。更稳的方式可能是捕获需要的数据副本，或捕获 `std::weak_ptr` 并在执行时检查对象是否还存在。

18.14 选择 lambda、函数对象还是普通函数

有了展开模型，选择就不再靠感觉：

形态	适合场景	关键判断
普通函数	无状态、可复用规则	名字能表达稳定概念

lambda	局部一次性规则	规则贴近使用点更清楚
函数对象类	有状态、要复用或单测	状态和行为值得命名
<code>std::function</code>	运行期保存不同 callable	接受类型擦除成本
模板 callable	热路径、类型编译期已知	追求内联和零额外抽象成本

lambda 不是“更高级的函数”，而是编译器帮你快速生成一个函数对象。知道这一点，捕获、生命周期、性能和错误信息都更容易推导。

第 19 章

optional、variant、expected 风格和标准库词汇类型

19.1 问题入口：返回值为什么不能只靠特殊值

查找用户时，找不到用户不是程序崩溃，也不是用户 ID 为 **-1**：

Listing 19.1 特殊值会污染语义

```
1 int find_user_id(std::string_view name)
2 {
3     if (name == "Ada") {
4         return 1;
5     }
6     return -1;
7 }
```

-1 是约定，不是类型。调用者可能忘记检查，也可能把 **-1** 当成普通 ID 继续传。标准库词汇类型的价值，是把常见语义放进类型：可能没有值、若干类型之一、共享文本视图、连续区间视图。

19.2 optional：可能没有值

`std::optional<T>` 表达“有一个 **T**，或没有”：

Listing 19.2 optional 表达查找结果

```
1 std::optional<User> find_user(std::string_view name)
2 {
3     if (name == "Ada") {
4         return User{.id = 1, .name = "Ada"};
5     }
6     return std::nullopt;
7 }
```

调用者必须面对没有值：

Listing 19.3 检查 optional

```
1 const std::optional<User> user = find_user("Grace");
2 if (!user.has_value()) {
3     std::cout << "not found\n";
4     return;
5 }
6 std::cout << user->name << '\n';
```

`optional` 不适合表达复杂错误原因。如果调用者需要知道“文件不存在、格式错误、权限不足”，就需要错误类型。

19.3 variant：若干类型之一

命令行参数可以是不同形态：

Listing 19.4 命令行值的 variant

```
1 struct Flag {
2     std::string name;
3 };
4
5 struct KeyValue {
6     std::string key;
7     std::string value;
8 };
9
10 using Argument = std::variant<Flag, KeyValue>;
```

处理时用 `std::visit`：

Listing 19.5 访问 variant 参数

```
1 void print_argument(const Argument& argument)
2 {
3     std::visit([](const auto& item) {
4         print_one(item);
5     }, argument);
6 }
```

`variant` 适合类型集合固定的情况。它让编译器知道所有可能形态，也能在新增形态时提醒处理点。

19.4 expected 风格：值或错误

C++23 有 `std::expected`，但很多 C++20 工程会自己定义简单结果类型，或使用成熟库。核心理想是一样的：成功时有值，失败时有错误。

Listing 19.6 expected 风格结果

```
1 template <typename T, typename E>
2 struct Result {
3     std::optional<T> value;
4     std::optional<E> error;
5 };
```

这个简化版本不能阻止同时有值和错误。更严谨可以用 `std::variant<T, E>`：

Listing 19.7 variant 表达二选一

```
1 template <typename T, typename E>
2 class Result2 {
3 public:
4     static Result2 success(T value)
5     {
6         return Result2{std::move(value)};
7     }
8 }
```

```

9     static Result2 failure(E error)
10    {
11        return Result2{std::move(error)};
12    }
13
14 private:
15     explicit Result2(std::variant<T, E> state)
16         : state_{std::move(state)}
17     {
18     }
19
20     std::variant<T, E> state_;
21 };

```

教材里经常用简单结构，是为了降低语法负担；工程里要根据风险选择更严谨的表示。

19.5 pair 和 tuple 不要滥用

`std::pair` 和 `std::tuple` 适合临时组合值，但不适合长期表达业务对象：

Listing 19.8 tuple 让含义消失

```
1 std::tuple<std::string, int, bool> user;
```

读者不知道三个字段分别是什么。更好的写法是结构体：

Listing 19.9 结构体保留字段含义

```

1 struct UserRow {
2     std::string name;
3     int age = 0;
4     bool active = false;
5 };

```

如果返回两个迭代器、两个坐标、键值对，`pair` 合理；如果字段有业务名，就写类型。

19.6 span 和 string_view 也是词汇类型

`std::span<const T>` 表达非拥有连续区间，`std::string_view` 表达非拥有文本视图。它们减少复制，但也把生命周期要求推给调用者。词汇类型不是越轻越好，而是要准确表达语义。

例如解析函数只观察文本：

Listing 19.10 解析只需要观察文本

```
1 std::optional<int> parse_port(std::string_view text);
```

配置对象要保存文本：

Listing 19.11 保存文本要拥有字符串

```

1 struct ServerConfig {
2     std::string host;
3 };

```

不要把 `string_view` 存进长期配置对象，除非你清楚底层字符串的生命周期。

19.7 完整案例：解析命令行参数

一个参数可以是标志、键值对或位置参数：

Listing 19.12 参数词汇类型

```

1 struct Positional {
2     std::string value;
3 };
4
5 using ParsedArgument = std::variant<Flag, KeyValue, Positional>;
6
7 ParsedArgument parse_argument(std::string_view text)
8 {
9     if (text.starts_with("--")) {
10         const std::size_t equal = text.find('=');
11         if (equal == std::string_view::npos) {
12             return Flag{.name = std::string{text.substr(2)}};
13         }
14         return KeyValue{
15             .key = std::string{text.substr(2, equal - 2)},
16             .value = std::string{text.substr(equal + 1)},
17         };
18     }
19     return Positional{.value = std::string{text}};
20 }
```

这个函数把不同形态显式化。后续处理参数时，编译器知道所有可能情况，而不是让一堆字符串约定散在代码里。

习题与作业

把前面的 `ParseResult` 改成 `std::variant<Options, ParseError>` 风格。写一个辅助函数判断成功或失败。比较这种写法和两个 `optional` 字段的优缺点。

19.8 optional 背后的状态模型

`std::optional<T>` 可以用普通代码近似理解成“一个是否有值的标志，加上一块能放 `T` 的存储”。它不是把 `T` 初始化成某个特殊值。

Listing 19.13 `optional` 的近似模型

```

1 template <typename T>
2 class OptionalLike {
3 public:
4     [[nodiscard]] bool has_value() const noexcept
5     {
6         return this->has_value_;
7     }
8
9 private:
10     bool has_value_ = false;
11     // 真实 optional 会用合适的未初始化存储保存 T
12 };
```

这解释了为什么 `optional<int>` 能区分“没有值”和“值为 0”。特殊值方案把状态塞进值域；

`optional` 把状态单独拿出来。

19.9 variant 背后的标签联合体

`std::variant<Flag, KeyValue, Positional>` 可以理解成“一个标签，加上一块足够大的存储”。标签记录当前里面是哪一种类型：

Listing 19.14 `variant` 的近似状态

```
1 enum class ArgumentKind {
2     flag,
3     key_value,
4     positional,
5 };
6
7 struct ArgumentLike {
8     ArgumentKind kind = ArgumentKind::flag;
9     // 真实 variant 会管理构造、析构、对齐和异常安全
10 };
```

手写这种结构很容易漏析构、拷贝和异常安全，所以标准库提供 `variant`。但理解“标签加存储”很重要：处理 `variant` 时必须看标签，也就是用 `std::visit` 或 `std::get_if` 分支处理当前形态。

19.10 visit 背后是按标签分发

下面的 `visit`：

Listing 19.15 `visit` 处理三种参数

```
1 std::visit([](const auto& item) {
2     print_one(item);
3 }, argument);
```

可以近似理解成：

Listing 19.16 按当前标签分发

```
1 switch (argument.kind()) {
2 case ArgumentKind::flag:
3     print_one(argument.as_flag());
4     break;
5 case ArgumentKind::key_value:
6     print_one(argument.as_key_value());
7     break;
8 case ArgumentKind::positional:
9     print_one(argument.as_positional());
10    break;
11 }
```

标准库帮你保证类型安全。新增一种参数形态时，访问器如果处理不了，编译器会在相应调用点报错。这比字符串里塞一个“`kind`”字段更早暴露遗漏。

19.11 expected 风格要保证二选一

两个 `optional` 字段的简化 `Result` 有一个设计漏洞：它可能同时有值和错误，也可能两者都没有。用 `variant<T, E>` 表达二选一更严谨，因为状态空间只允许成功或失败。

Listing 19.17 二选一结果的最小接口

```

1 template <typename T, typename E>
2 class Result {
3 public:
4     [[nodiscard]] bool has_value() const noexcept
5     {
6         return std::holds_alternative<T>(this->state_);
7     }
8
9     [[nodiscard]] const T& value() const
10    {
11        return std::get<T>(this->state_);
12    }
13
14    [[nodiscard]] const E& error() const
15    {
16        return std::get<E>(this->state_);
17    }
18
19 private:
20     std::variant<T, E> state_;
21 };

```

真实项目还要处理构造函数、移动、错误访问策略和易用性。本章重点是模型：如果业务语义是二选一，类型最好也只允许二选一。

19.12 词汇类型选择表

语义	类型	不要误用成
可能没有值	<code>std::optional<T></code>	特殊值 <code>-1</code> 、空字符串
固定几种形态之一	<code>std::variant<A, B></code>	手写 <code>kind</code> 加裸字段
值或错误	expected 风格结果	打印错误后返回默认值
非拥有连续区间	<code>std::span<T></code>	指针加长度分离传参
非拥有文本	<code>std::string_view</code>	长期保存的配置字段
业务字段集合	<code>struct</code>	长期使用的 <code>tuple</code>

词汇类型的价值是让读者在函数签名里看见语义。签名越诚实，调用者越不容易猜错。

第 20 章

编译期计算、type traits 和约束边界

20.1 问题入口：有些错误能不能提前到编译期

运行期检查很重要，但有些规则在编译期就知道。例如固定容量缓冲区的容量不能为 0。如果等运行到取模时才失败，错误出现得太晚。模板参数和 constexpr 让一部分规则可以提前：

Listing 20.1 编译期容量检查

```
1 template <typename T, std::size_t CAPACITY>
2     requires(CAPACITY > 0)
3 class RingBuffer {
4     // ...
5 };
```

这不是为了炫技，而是把“不可能正确”的类型直接挡在编译期。编译期计算的价值在于更早暴露错误、更清楚表达常量关系，以及减少运行期分支。

20.2 constexpr 函数

constexpr 函数可以在编译期求值，也可以在运行期调用：

Listing 20.2 端口范围检查

```
1 constexpr bool is_valid_port(int port) noexcept
2 {
3     return 1 <= port && port <= 65535;
4 }
5
6 static_assert(is_valid_port(80));
7 static_assert(!is_valid_port(70000));
```

static_assert 是编译期断言。它适合检查常量规则、模板约束和平台假设。不要把运行期输入塞进 static_assert，因为运行期输入编译时并不存在。

20.3 用 type traits 观察类型能力

type traits 是一组在编译期回答类型问题的工具。例如：

Listing 20.3 检查类型属性

```
1 static_assert(std::is_integral_v<int>);
2 static_assert(!std::is_integral_v<std::string>);
3 static_assert(std::is_move_constructible_v<std::vector<int>>);
```

这些工具常用于泛型库内部。普通业务代码不应该到处堆 traits；如果可以用 concept 表达接口，优先用 concept。traits 更像底层零件，concept 更像公开合同。

20.4 enable_if 到 concept 的演进

C++20 以前常用 std::enable_if 限制模板：

Listing 20.4 传统 SFINAE 写法

```

1 template <typename T, typename = std::enable_if_t<std::is_integral_v<T>>>
2 bool is_even(T value)
3 {
4     return value % 2 == 0;
5 }

```

C++20 可以写得更直接：

Listing 20.5 concept 写法更接近意图

```

1 template <std::integral T>
2 bool is_even(T value)
3 {
4     return value % 2 == 0;
5 }

```

两者本质都在限制类型，但 concept 把错误消息和接口意图都变清楚。学习 traits 是为了能读懂旧代码和写底层库，不是为了把所有新代码写复杂。

20.5 if constexpr：编译期分支

有些分支取决于类型，不取决于运行期值：

Listing 20.6 根据类型选择实现

```

1 template <typename T>
2 std::string to_debug_string(const T& value)
3 {
4     if constexpr (std::is_same_v<T, std::string>) {
5         return value;
6     } else if constexpr (std::is_arithmetic_v<T>) {
7         return std::to_string(value);
8     } else {
9         return "<object>";
10    }
11 }

```

if constexpr 未选择的分支不会被实例化。这和普通 if 不同。普通 if 的两个分支都必须语法和类型正确；if constexpr 可以根据类型裁掉不适用分支。

20.6 编译期表和运行期表

如果一组数据固定不变，可以用 constexpr std::array：

Listing 20.7 编译期关键字表

```

1 constexpr std::array<std::string_view, 4> KEYWORDS{
2     "class",
3     "const",
4     "return",
5     "while",
6 };
7
8 constexpr bool is_keyword(std::string_view word)

```

```

9 {
10     for (std::string_view keyword : KEYWORDS) {
11         if (keyword == word) {
12             return true;
13         }
14     }
15     return false;
16 }

```

这里的表很小，线性扫描足够。如果关键字很多，可以排序后二分，但不要为了“编译期”牺牲可读性。编译期优化也要有规模依据。

20.7 模板错误如何设计得更友好

一个泛型函数如果没有约束，用户传错类型时可能看到几十行错误。用 `concept` 或 `static_assert` 可以把错误变近：

Listing 20.8 用编译期断言给出明确错误

```

1 template <typename T>
2 T median(std::span<const T> values)
3 {
4     static_assert(std::totally_ordered<T>,
5                   "median requires an ordered value type");
6     // ...
7 }

```

更好的方式是直接写到模板头上：

Listing 20.9 约束排序能力

```

1 template <std::totally_ordered T>
2 T median(std::span<const T> values);

```

公开接口优先用约束，函数内部再用 `static_assert` 检查更细的不变量。

20.8 不要把运行期问题硬搬到编译期

编译期计算有成本：模板实例化更复杂，编译时间可能增加，错误信息可能变长。配置文件、用户输入、网络数据、数据库记录都天然不是运行期信息，不要为了“高级”硬做成模板参数。

一个实用判断：如果错误和类型或固定常量有关，考虑编译期；如果错误和外部输入有关，运行期处理更自然。

习题与作业

实现一个 `FixedString<N>`，保存固定长度字符串，并写一个 `constexpr` 函数判断它是否为空。然后回答：哪些检查发生在编译期，哪些仍然只能运行期处理？

20.9 constexpr 的普通代码理解

`constexpr` 函数不是只能在编译期运行的函数。它更像一段同时满足“可在编译期求值条件”的普通函数：

Listing 20.10 `constexpr` 可以编译期也可以运行期

```

1 constexpr bool is_valid_port(int port) noexcept
2 {

```

```

3     return 1 <= port && port <= 65535;
4 }
5
6 static_assert(is_valid_port(80));    // 编译期
7 bool ok = is_valid_port(user_input); // 运行期

```

当实参是常量表达式时，编译器可以在编译期计算；当实参来自用户输入时，只能运行期计算。不要把 `constexpr` 理解成“强制提前执行”。它提供的是提前执行的可能性。

20.10 `static_assert` 背后是编译期 `if`

`static_assert` 可以理解成编译期门禁：

Listing 20.11 平台假设检查

```

1 static_assert(sizeof(void*) == 8, "this example expects 64-bit pointers");

```

如果条件为假，编译停止，程序不会生成。它适合检查类型、模板参数、平台假设和固定常量。不适合检查配置文件端口，因为配置文件内容编译期并不存在。

20.11 `if constexpr` 和普通 `if` 的差别

普通 `if` 的两个分支都必须能通过类型检查：

Listing 20.12 普通 `if` 不能隐藏类型错误

```

1 if (condition) {
2     value.size();
3 } else {
4     value + 1;
5 }

```

哪怕运行时不会走某个分支，编译器也要确认它语法成立。`if constexpr` 则会在编译期选择分支，未选择分支不实例化：

Listing 20.13 `if constexpr` 按类型选择

```

1 template <typename T>
2 std::string describe_value(const T& value)
3 {
4     if constexpr (requires { value.size(); }) {
5         return "size=" + std::to_string(value.size());
6     } else {
7         return "scalar";
8     }
9 }

```

这就是它适合泛型代码的原因：不同类型可以走不同代码路径，而不适用的路径不会强行检查。

20.12 `type traits` 如何被 `concept` 包装

旧代码常写 traits：

Listing 20.14 traits 版本

```

1 template <typename T>

```

```

2 constexpr bool IS_SMALL_TRIVIAL_VALUE =
3     std::is_trivially_copyable_v<T> && sizeof(T) <= 16;

```

C++20 可以把它包装成 concept:

Listing 20.15 concept 包装类型条件

```

1 template <typename T>
2 concept SmallTrivialValue =
3     std::is_trivially_copyable_v<T> && sizeof(T) <= 16;

```

之后接口可以直接写:

Listing 20.16 接口用 concept 表达意图

```

1 template <SmallTrivialValue T>
2 void pass_by_value(T value);

```

traits 像螺丝和齿轮, concept 像对外写清楚的门牌。业务代码优先读门牌, 底层库才经常直接摆弄齿轮。

20.13 FixedString 案例

一个固定字符串类型可以保存长度进入类型:

Listing 20.17 固定字符串雏形

```

1 template <std::size_t N>
2 struct FixedString {
3     std::array<char, N> chars{};
4
5     constexpr explicit FixedString(const char (&text)[N])
6     {
7         std::copy_n(text, N, this->chars.begin());
8     }
9
10    [[nodiscard]] constexpr bool empty() const noexcept
11    {
12        return N <= 1;
13    }
14 };

```

如果写 `FixedString{"while"}`, 长度 `N` 来自字符串字面量, 编译器能知道它。这个例子说明哪些信息能进编译期: 字面量长度、模板参数、类型属性。用户运行时输入的一行文本, 仍然不能变成模板参数, 除非你重新编译程序。

20.14 编译期不是质量保证的全部

把错误提前到编译期很好, 但不能代替运行期防御。端口范围函数可以 `static_assert(is_valid_port(80))`, 但用户配置里的 `70000` 仍然要运行期报错。编译期工具负责固定世界, 运行期错误处理负责外部世界。两本账都要有。

第零部分

工程化：构建、测试、调试和性能

第 21 章

构建、测试和调试

21.1 问题入口：一份源文件走不远

单文件程序适合学习语法，但真实项目很快需要多个源文件、测试和构建脚本。假设项目有一个统计库：

Listing 21.1 最小项目结构

```
1 score_tool/  
2   CMakeLists.txt  
3   include/score/summary.hpp  
4   src/summary.cpp  
5   tests/summary_tests.cpp
```

目录结构表达所有权：公开头文件在 `include`，实现放 `src`，测试放 `tests`。不要把所有文件堆在一个目录里。

21.2 CMake 的基本边界

Listing 21.2 库和测试目标分开

```
1 cmake_minimum_required(VERSION 3.20)  
2 project(score_tool LANGUAGES CXX)  
3  
4 add_library(score_summary src/summary.cpp)  
5 target_compile_features(score_summary PUBLIC cxx_std_20)  
6 target_include_directories(score_summary PUBLIC include)  
7  
8 add_executable(summary_tests tests/summary_tests.cpp)  
9 target_link_libraries(summary_tests PRIVATE score_summary)  
10 enable_testing()  
11 add_test(NAME summary_tests COMMAND summary_tests)
```

目标是 CMake 的核心。不要把编译选项、include 路径和库依赖全局乱撒。谁需要依赖，谁声明依赖。

21.3 测试不是最后补

统计函数天然适合测试：

Listing 21.3 不用测试框架也能先写断言

```
1 #include <cassert>  
2 #include <vector>  
3  
4 int main()  
5 {  
6     const std::vector<int> scores{90, 80, 100};  
7     const auto summary = summarize_scores(scores);
```

```

8     assert(summary.has_value());
9     assert(summary->count == 3);
10    assert(summary->sum == 270);
11    assert(summary->min_score == 80);
12    assert(summary->max_score == 100);
13 }

```

真实项目可以使用 Catch2、GoogleTest 或 doctest。框架不是重点，重点是测试要覆盖边界：空输入、单元素、多元素、极值和错误路径。

21.4 调试先缩小问题

调试不等于到处打印。更可靠的流程是：

1. 复现问题，固定输入。
2. 写一个最小失败测试。
3. 确认失败位置：输入解析、核心逻辑、输出格式还是环境。
4. 修正后保留测试，防止回归。

调试器可以观察变量、调用栈和线程状态；日志可以保留运行历史；断言可以检查不变量。三者用途不同。

Listing 21.4 断言用于检查内部不变量

```

1 #include <cassert>
2
3 void push_value(std::vector<int>& values, int value)
4 {
5     const std::size_t old_size = values.size();
6     values.push_back(value);
7     assert(values.size() == old_size + 1);
8 }

```

断言不是用户错误处理。它适合检查“代码内部本应永远成立”的条件。

21.5 本章边界检查

一个能长期维护的 C++ 项目至少要有：

- 清晰目标：库、程序、测试分别是什么。
- 目标级 include 路径和依赖。
- 可重复运行的测试命令。
- Debug 和 Release 两种构建。
- 编译警告作为质量输入。

核心思想

工程化不是等项目变大后才需要。构建边界、测试入口和调试习惯越早建立，后面修改成本越低。

21.6 从一条编译命令演进到 CMake

单文件时可以直接编译：

Listing 21.5 单文件编译

```

1 g++ -std=c++20 -Wall -Wextra -pedantic main.cpp -o score_tool

```

当项目拆成头文件、源文件和测试后，手写命令会越来越长，而且容易漏依赖。CMake 的价值不是“多一个工具”，而是把目标关系记录下来：库由哪些源文件组成，公开哪些头文件路径，测试链接哪个库。

推荐的构建流程如下：

Listing 21.6 独立构建目录

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
2 cmake --build build
3 ctest --test-dir build --output-on-failure
```

-S 指源码目录，**-B** 指构建目录。不要把生成文件混进源码目录。Debug 构建适合调试，Release 构建适合性能测量。

21.7 头文件和源文件怎么拆

以成绩统计库为例，头文件只放接口和必要类型：

Listing 21.7 summary.hpp 暴露稳定接口

```
1 #pragma once
2
3 #include <optional>
4 #include <span>
5
6 struct ScoreSummary {
7     std::size_t count = 0;
8     int sum = 0;
9     int min_score = 0;
10    int max_score = 0;
11    double average = 0.0;
12 };
13
14 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores);
```

实现文件放算法细节：

Listing 21.8 summary.cpp 放实现

```
1 #include <score/summary.hpp>
2
3 #include <algorithm>
4 #include <numeric>
5
6 std::optional<ScoreSummary> summarize_scores(std::span<const int> scores)
7 {
8     if (scores.empty()) {
9         return std::nullopt;
10    }
11    const auto [min_it, max_it] = std::minmax_element(scores.begin(), scores.end());
12    const int sum = std::accumulate(scores.begin(), scores.end(), 0);
13    return ScoreSummary{scores.size(), sum, *min_it, *max_it,
14                        static_cast<double>(sum) / scores.size()};
15 }
```

调用者只需要包含头文件，不应该依赖实现文件里的私有细节。这个边界越干净，后面替换实现越容易。

21.8 测试要覆盖失败路径

很多测试只测普通输入，结果错误路径一直没人碰。成绩统计至少要有这些测试：

Listing 21.9 最小测试集合

```
1 void test_empty_scores()
2 {
3     const std::vector<int> scores;
4     assert(!summarize_scores(scores).has_value());
5 }
6
7 void test_single_score()
8 {
9     const std::vector<int> scores{90};
10    const auto summary = summarize_scores(scores);
11    assert(summary.has_value());
12    assert(summary->min_score == 90);
13    assert(summary->max_score == 90);
14    assert(summary->average == 90.0);
15 }
```

测试函数命名要说清行为，而不是叫 `test1`、`test2`。失败时，名字就是第一条诊断信息。

21.9 调试工具各自解决什么

编译警告帮你发现可疑代码；单元测试帮你固定行为；调试器帮你观察某次运行；消毒器帮助发现越界、悬空引用和数据竞争。可以给 Debug 构建加上地址消毒器：

Listing 21.10 为测试目标打开地址消毒器

```
1 target_compile_options(summary_tests PRIVATE -fsanitize=address ↵
    ↵ -fno-omit-frame-pointer)
2 target_link_options(summary_tests PRIVATE -fsanitize=address)
```

这不是替代测试，而是让测试失败得更早、更明确。比如越界访问可能平时“看起来能跑”，在消毒器下会直接报出位置。

习题与作业

把成绩统计项目拆成 `include/score/summary.hpp`、`src/summary.cpp`、`tests/summary_tests.cpp`。要求 `ctest` 能运行测试；再故意把空输入检查删掉，确认测试能失败。

第 22 章

性能、内存和测量

22.1 问题入口：字符串拼接为什么慢

性能优化不能靠感觉。先看一个常见例子：拼接很多小字符串。

Listing 22.1 反复拼接可能频繁分配

```
1 std::string build_line(const std::vector<std::string>& parts)
2 {
3     std::string line;
4     for (const std::string& part : parts) {
5         line += part;
6         line += ',';
7     }
8     return line;
9 }
```

这段代码可能多次扩容。每次扩容都可能分配新内存、复制旧内容。优化前先确认瓶颈，如果它在热路径，可以先预估长度并 `reserve`。

Listing 22.2 预留容量减少分配

```
1 std::string build_line_reserved(const std::vector<std::string>& parts)
2 {
3     std::size_t total_size = 0;
4     for (const std::string& part : parts) {
5         total_size += part.size() + 1;
6     }
7
8     std::string line;
9     line.reserve(total_size);
10    for (const std::string& part : parts) {
11        line += part;
12        line += ',';
13    }
14    return line;
15 }
```

22.2 复杂度和常数都要看

$O(n)$ 不等于一定快， $O(n \log n)$ 不等于一定慢。数据规模、内存局部性、分配次数和分支预测都会影响实际时间。比如 `std::vector` 线性查找小数组，可能比 `std::unordered_set` 更快，因为连续内存和低常数优势明显。

22.3 避免不必要复制

性能问题里最常见的是无意复制：

Listing 22.3 range-for 中复制字符串

```

1 for (auto name : names) {
2     use(name);
3 }

```

如果 `use` 只观察，应该写：

Listing 22.4 只读遍历用 `const` 引用

```

1 for (const auto& name : names) {
2     use(name);
3 }

```

函数参数同理。大的字符串、向量、路径、映射和业务对象，不要在只读场景按值传递。

22.4 测量要可重复

一个最小 benchmark 要避免三个错误：测量太短、把 I/O 算进去、编译器把结果优化掉。下面只是示意：

Listing 22.5 用稳定时钟做粗略测量

```

1 #include <chrono>
2 #include <iostream>
3
4 template <typename Func>
5 void measure(std::string_view name, Func func)
6 {
7     const auto start = std::chrono::steady_clock::now();
8     const auto result = func();
9     const auto finish = std::chrono::steady_clock::now();
10    const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
11        finish - start);
12    std::cout << name << ": " << elapsed.count()
13        << " us, result=" << result << '\n';
14 }

```

严肃 benchmark 应使用专门框架，并固定编译选项、输入规模、运行次数和机器状态。本章先建立测量意识。

22.5 优化前后的检查单

优化不是把代码写得更低级。每次优化前后都应该记录：

- 输入规模：小数据、中数据和目标大数据分别是多少。
- 编译模式：Debug 结果通常不能代表性能。
- 指标：延迟、吞吐、内存占用还是分配次数。
- 正确性：优化后是否跑过同一组测试。
- 可维护性：复杂度增加是否值得。

例如给字符串拼接加 `reserve`，复杂度没有改变，但减少了分配次数，代码仍然容易理解。反过来，如果为了省一个小对象复制引入手写内存池，就必须有测量证明它在目标场景中值得。

22.6 内存布局

对象布局会影响缓存。把频繁一起访问的数据放在连续内存里，通常比到处跳指针更友好。

Listing 22.6 连续数组适合批量处理

```

1 struct Point {
2     double x = 0.0;
3     double y = 0.0;
4 };
5
6 double sum_x(std::span<const Point> points)
7 {
8     double total = 0.0;
9     for (const Point& point : points) {
10         total += point.x;
11     }
12     return total;
13 }

```

更高级场景会讨论结构数组和数组结构。本书先记住：性能不仅是算法，也包括数据如何摆放。

常见误区

不要先优化再证明。先写清楚正确性和边界，再用测量找瓶颈。没有测量的优化，经常只是把代码变复杂。

22.7 从分配次数理解 reserve

字符串拼接慢，根本原因常常不是 `+=` 这个符号，而是容量不够时需要重新分配。假设目标字符串最终长度是 **100000**，如果容量从很小开始逐步增长，就会多次申请新内存，并把旧内容搬过去。每搬一次，历史内容又被读写一遍。

`reserve` 的推理链条是：我们能提前算出最终长度；最终长度给了容器容量上界；提前申请一次能减少后续扩容；扩容减少后，总复制量和分配次数下降。这个优化仍然保持同样的接口和同样的复杂度，维护成本很低，所以优先级高。

22.8 局部性案例：vector 与 list

很多人以为 `std::list` 插入删除快，所以应该更快。对大量顺序遍历来说，事实常常相反：

Listing 22.7 顺序求和更适合连续内存

```

1 int sum_vector(const std::vector<int>& values)
2 {
3     int total = 0;
4     for (int value : values) {
5         total += value;
6     }
7     return total;
8 }

```

`vector` 的元素连续，CPU 能更好地预取。链表每个节点可能分散在内存各处，遍历时不断追指针。算法复杂度同样是 $O(n)$ ，但常数和缓存局部性差很多。除非你真的需要稳定节点和频繁中间插入删除，否则不要默认选择链表。

22.9 避免复制要看语义

不是所有按值都是错。构造函数保存一个字符串时，按值接收再移动是合理的；只读函数按值接收大对象就不合理。判断标准是：函数是否需要拥有一份独立副本。

Listing 22.8 有意按值接收用于保存

```

1 class LogMessage {
2 public:
3     explicit LogMessage(std::string text)
4         : text_{std::move(text)}
5     {
6     }
7
8 private:
9     std::string text_;
10 };

```

如果只是打印：

Listing 22.9 只观察时不要复制

```

1 void print_message(std::string_view text)
2 {
3     std::cout << text << '\n';
4 }

```

`std::string_view` 不拥有字符内容，所以不能长期保存到对象里，除非你能证明底层字符串活得更久。性能优化和生命周期安全必须一起看。

22.10 benchmark 的最小可信做法

粗略测量时，至少让函数重复多次，并把结果用掉：

Listing 22.10 重复运行降低偶然波动

```

1 template <typename Func>
2 void measure_many(std::string_view name, Func func)
3 {
4     constexpr int RUNS = 50;
5     std::int64_t checksum = 0;
6     const auto start = std::chrono::steady_clock::now();
7     for (int i = 0; i < RUNS; ++i) {
8         checksum += func();
9     }
10    const auto finish = std::chrono::steady_clock::now();
11    const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
12        finish - start);
13    std::cout << name << ": " << elapsed.count()
14        << " us, checksum=" << checksum << '\n';
15 }

```

`checksum` 的作用是让计算结果参与可观察输出，降低编译器把整个计算优化掉的可能。真正严肃的性能分析还要用 profiler、benchmark 框架和固定环境；但这个最小版本已经比凭感觉强很多。

习题与作业

实现两个版本的字符串拼接：一个不 `reserve`，一个先计算总长度再 `reserve`。用 `Release` 构建分别测试 **100**、**10000**、**100000** 个片段。记录时间和输出长度，确认优化没有改变结果。

22.11 性能事故要先写观察记录

性能问题不能只说“感觉慢”。假设一个日志导出工具在测试数据上只跑一秒，到生产的一百万行日志上跑了两分钟。第一步不是马上改成多线程，而是写观察记录：

字段	记录
输入规模	行数、平均每行字段数、总字节数
构建模式	Debug 还是 Release，优化选项是什么
指标	总耗时、峰值内存、分配次数、输出大小
环境	机器型号、磁盘类型、是否写终端
正确性	优化前后输出是否字节级一致

这张表的作用是防止把问题想歪。很多“字符串慢”其实是终端输出慢，很多“算法慢”其实是 Debug 构建慢，很多“多线程能解决”其实被磁盘 I/O 限制。没有观察记录，优化就只是猜谜。

22.12 把字符串拼接拆成分配账本

再看 `build_line`。它的热路径不是 `for` 这个语法，而是容量变化。可以为它写分配账本：

- 初始容量可能很小。
- 每追加一个片段都检查容量。
- 容量不足时申请新缓冲区。
- 旧内容复制到新缓冲区。
- 旧缓冲区释放。

如果最终输出长度是十几 MB，这个账本里的“旧内容复制”可能重复发生多次。预先计算总长度后 `reserve`，就是把多次扩容尽量变成一次扩容。这个优化的好处是局部、可解释、容易验证；坏处是需要先扫一遍输入。对于内存中的字符串片段，额外一遍扫描通常很便宜；对于流式输入，可能就不能这样做。

22.13 Release 构建和 Debug 构建不是一回事

测性能时，Debug 构建经常完全不可信。标准库迭代器检查、未优化函数调用、断言和调试符号都会改变结果。至少用这样的命令做一个 Release 版本：

Listing 22.11 用优化构建测量样本

```
1 g++ -std=c++20 -O2 -DNDEBUG bench.cpp -o bench
2 ./bench
```

如果项目使用 CMake，则固定构建类型：

Listing 22.12 CMake Release 构建

```
1 cmake -S . -B build-release -DCMAKE_BUILD_TYPE=Release
2 cmake --build build-release
3 ./build-release/bench
```

测量报告里必须写清构建模式。否则“优化前后快了三倍”可能只是一次用 Debug、一次用 Release。

22.14 优化后也要防回归

性能优化不能只跑一次手工命令。至少要给正确性加测试，给性能留下可重复脚本。比如字符串拼接可以先验证输出完全一致：

Listing 22.13 优化前后结果必须一致

```
1 const std::vector<std::string> parts = make_parts(1000);
2 assert(build_line(parts) == build_line_reserved(parts));
```

然后性能脚本输出机器可读的记录：

Listing 22.14 性能记录示例

```
1 case=reserved parts=100000 bytes=1200000 micros=8300 checksum=1200000
2 case=plain parts=100000 bytes=1200000 micros=46200 checksum=1200000
```

有了这些字段，下次改代码时才能判断是输入变了、机器变了，还是性能真的回退。高质量优化的标准不是写了更底层的代码，而是改动有证据、结果可复现、正确性没变。

22.15 内存布局案例：对象数组还是指针数组

假设要处理一百万个点。两个表示都能工作：

Listing 22.15 连续对象数组

```
1 std::vector<Point> points;
```

Listing 22.16 指针数组

```
1 std::vector<std::unique_ptr<Point>> points;
```

如果只是批量求和坐标，第一种通常更合适：对象连续存放，遍历时内存访问可预测。第二种每个点单独分配，遍历时先读指针，再跳到对象地址，缓存局部性更差，还增加分配和释放成本。第二种不是永远错；如果对象需要多态、稳定地址、独立生命周期，它可能合理。但仅仅为了“对象很大”就把所有东西放到堆上，通常会让内存访问更差。

核心思想

性能推理要同时看算法复杂度、分配次数、内存局部性和接口语义。只看一个维度，容易把代码优化到错误方向。

第 23 章

异常安全、代码质量和工具链

23.1 问题入口：异常发生时对象还可信吗

异常安全不是“是否使用异常”的争论，而是异常发生后程序状态是否仍然可信。看一个转账函数：

Listing 23.1 中途失败会破坏状态

```
1 void transfer(Account& from, Account& to, int amount)
2 {
3     from.withdraw(amount);
4     to.deposit(amount);
5 }
```

如果 `withdraw` 成功，`deposit` 抛异常，钱从 `from` 扣了，却没有加到 `to`。这不是异常的问题，而是操作没有设计事务边界。异常安全要从不变量出发：函数失败后，哪些状态必须仍然成立？

23.2 先把失败路径写成账本

不要一上来背“基本保证、强保证”。先把转账函数的每一步列出来：

1. 检查 `from` 余额是否足够。
2. 从 `from` 扣钱。
3. 给 `to` 加钱。
4. 写审计日志或通知外部系统。

如果第 2 步之后第 3 步失败，账户总额被破坏；如果第 3 步之后第 4 步失败，账户余额已经正确，但审计缺失。两个失败点的性质不同：前者破坏核心不变量，后者是外部副作用缺失。异常安全分析必须把“不变量”和“附加动作”分开。

一种更稳的转账接口，是让单个账户的余额修改本身不抛异常，所有可能失败的检查都在修改前完成：

Listing 23.2 先验证再做不抛提交

```
1 class Account {
2 public:
3     [[nodiscard]] bool can_withdraw(int amount) const noexcept;
4     void withdraw_checked(int amount) noexcept;
5     void deposit_checked(int amount) noexcept;
6 };
7
8 bool transfer(Account& from, Account& to, int amount)
9 {
10     if (!from.can_withdraw(amount)) {
11         return false;
12     }
13
14     from.withdraw_checked(amount);
15     to.deposit_checked(amount);
16     return true;
```

17 }

这个版本不是靠异常恢复，而是改变设计：提交阶段只做不会失败的内存内状态修改。真实系统里如果涉及数据库、远程服务、审计日志，就需要更强的事务机制；普通 C++ 对象的异常安全不能自动覆盖外部世界。

23.3 三种常见保证

异常安全常用三个层级描述：

- 基本保证：异常后对象仍然有效，不泄漏资源，但值可能改变。
- 强保证：异常后状态像函数没被调用过。
- 不抛保证：函数承诺不抛异常，常用 `noexcept` 表达。

例如 `std::vector::push_back` 在很多情况下提供强保证：如果扩容或元素构造失败，原向量仍保持原样。要做到这一点，通常需要先准备新资源，成功后再提交状态。

23.4 先准备，后提交

假设要更新配置对象，解析新配置可能失败。不要边解析边修改当前配置：

Listing 23.3 边解析边修改会留下半更新状态

```
1 void reload_bad(Config& config, std::istream& input)
2 {
3     input >> config.host;
4     input >> config.port;
5     input >> config.thread_count;
6 }
```

更好的做法是先解析到临时对象，全部成功后再赋值：

Listing 23.4 先构造临时配置再提交

```
1 void reload(Config& config, std::istream& input)
2 {
3     Config next;
4     input >> next.host;
5     input >> next.port;
6     input >> next.thread_count;
7     validate(next);
8     config = std::move(next);
9 }
```

如果解析或校验失败，`config` 没有被修改。这就是强保证的基本形状。它通常会多一个临时对象，但换来更清楚的失败语义。

23.5 强保证背后的普通代码模型

强保证可以还原成三步普通代码：

1. 在临时状态里完成所有可能失败的工作。
2. 检查临时状态满足不变量。
3. 用一个尽量不抛的操作提交新状态。

配置热更新就是典型例子。坏版本直接改当前配置：

Listing 23.5 半更新配置的失败路径

```

1 void reload_bad(Config& config, std::istream& input)
2 {
3     input >> config.host;           // 可能已经改了 host
4     input >> config.port;           // 这里失败
5     input >> config.thread_count;   // 没执行
6 }

```

如果第二行失败，`config` 处在“新 host、旧 port、旧线程数”的混合状态。调用者不知道这个状态是否能继续使用。强保证版本把这种混合状态限制在局部变量里：

Listing 23.6 临时状态承担失败风险

```

1 Config parse_config(std::istream& input)
2 {
3     Config next;
4     input >> next.host;
5     input >> next.port;
6     input >> next.thread_count;
7     validate(next);
8     return next;
9 }
10
11 void reload(Config& config, std::istream& input)
12 {
13     Config next = parse_config(input);
14     config = std::move(next);
15 }

```

现在失败只会发生在 `parse_config` 内部，原 `config` 还没被碰。如果 `Config` 的移动赋值可能抛异常，最后一步仍然有风险；这时可以把 `Config` 设计成用 `std::shared_ptr<const ConfigData>` 持有不可变数据，提交时只交换指针，或者提供不抛的 `swap`：

Listing 23.7 用不抛 swap 提交

```

1 void reload_with_swap(Config& config, std::istream& input)
2 {
3     Config next = parse_config(input);
4     using std::swap;
5     swap(config, next); // 应当设计为 noexcept
6 }

```

这不是模板口号，而是一条能检查的实现规则：失败风险集中在临时对象，公开对象只在最后一步切换。

23.6 回滚对象适合局部状态，不适合所有副作用

有时无法做到“提交前不修改”。可以用一个小的回滚对象保存旧值，失败时恢复：

Listing 23.8 局部回滚对象

```

1 class BalanceRollback {
2 public:
3     explicit BalanceRollback(Account& account)
4         : account_{account}, old_balance_{account.balance()}

```

```

5     {
6     }
7
8     void commit() noexcept
9     {
10        this->committed_ = true;
11    }
12
13    ~BalanceRollback()
14    {
15        if (!this->committed_) {
16            this->account_.set_balance_for_rollback(this->old_balance_);
17        }
18    }
19
20 private:
21     Account& account_;
22     int old_balance_ = 0;
23     bool committed_ = false;
24 };

```

这个模型只适合恢复内存内、可逆、恢复过程不抛异常的状态。已经发出去的网络请求、已经打印的日志、已经写入另一个进程的数据，不能靠析构函数“撤回”。所以异常安全不是魔法回滚，它要求你先判断哪些动作可逆，哪些动作必须放到提交点之后，哪些动作需要外部事务系统。

23.7 析构函数不要抛异常

析构函数常在栈展开期间执行。如果析构函数再抛异常，程序可能直接终止。因此析构函数应该吞掉或记录清理失败，而不是让异常逃出：

Listing 23.9 析构中使用错误码清理

```

1 class TempFile {
2 public:
3     ~TempFile()
4     {
5         std::error_code ignored;
6         std::filesystem::remove(this->path_, ignored);
7     }
8
9 private:
10     std::filesystem::path path_;
11 };

```

如果清理失败必须报告，就提供显式 `close` 或 `commit` 函数，让调用者在可处理错误的位置调用。析构负责兜底清理，不负责复杂错误恢复。

23.8 noexcept 是接口承诺

`noexcept` 不是优化装饰，而是承诺。移动构造如果不抛，标准容器在扩容时更愿意移动元素：

Listing 23.10 移动构造通常应 `noexcept`

```

1 Buffer(Buffer&& other) noexcept
2     : data_{other.data_}, size_{other.size_}

```

```

3 {
4     other.data_ = nullptr;
5     other.size_ = 0;
6 }

```

不要随便给可能抛异常的函数标 `noexcept`。如果 `noexcept` 函数内部真的抛出异常，程序会调用 `std::terminate`。所以它必须和实现事实一致。

23.9 编译警告和静态检查

基础构建至少打开警告：

Listing 23.11 目标级警告选项

```

1 target_compile_options(my_target PRIVATE
2     -Wall
3     -Wextra
4     -Wpedantic
5     -Wconversion)

```

警告不是绝对真理，但它们是便宜的质量信号。比如未使用变量、隐式窄化转换、缺少返回值，经常能提前暴露真实 bug。不要把警告全局关掉，也不要为了“清零”写无意义代码。每个警告都要分类：真实问题、风格问题、工具误报，还是需要局部抑制。

23.10 消毒器：让未定义行为早点爆

地址消毒器可以发现越界、use-after-free 等问题：

Listing 23.12 用消毒器构建测试

```

1 cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=Debug \
2     -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer"
3 cmake --build build-asan
4 ctest --test-dir build-asan --output-on-failure

```

线程消毒器可以发现数据竞争，但通常不能和地址消毒器一起使用。并发代码要单独跑：

Listing 23.13 线程消毒器构建

```

1 cmake -S . -B build-tsan -DCMAKE_BUILD_TYPE=Debug \
2     -DCMAKE_CXX_FLAGS="-fsanitize=thread -fno-omit-frame-pointer"

```

工具不是证明程序正确，而是扩大测试能抓到的问题范围。没有测试输入，消毒器也无事可做。

23.11 代码审查清单

自查一段 C++ 代码时，可以按这个顺序看：

1. 所有权：谁拥有资源，谁只是观察？
2. 生命周期：引用、指针、视图是否可能悬空？
3. 错误路径：异常或错误返回后对象不变量是否仍成立？
4. 复制移动：是否有无意大对象复制？移动后对象是否被错误使用？
5. 复杂度：主要循环是否重复扫描历史数据？
6. 测试：空输入、单元素、边界值、失败路径是否覆盖？

这个顺序不是固定流程，但它能避免只看语法风格。真正影响可维护性的，通常是所有权、错误边界和数据结构选择。

习题与作业

选择前面 LRU 缓存项目，给每个公开函数写异常安全说明：容量为零、哈希表分配失败、Value 移动失败时，对象是否仍满足不变量？如果无法提供强保证，说明它至少能提供什么保证。

第 24 章

测试驱动的重构

24.1 问题入口：代码能跑，为什么还不敢改

很多项目真正的问题不是没有功能，而是不敢改。一个函数几百行，里面同时读文件、解析、计算、输出。你想修一个小 bug，却不知道会不会破坏其他路径。测试驱动的重构不是先写漂亮架构，而是先把行为固定住。

看一个混在一起的函数：

Listing 24.1 难以测试的混合函数

```
1 void process_file(const std::string& path)
2 {
3     std::ifstream input{path};
4     std::unordered_map<std::string, int> counts;
5     std::string word;
6     while (input >> word) {
7         normalize(word);
8         ++counts[word];
9     }
10    for (const auto& [word, count] : counts) {
11        std::cout << word << " " << count << '\n';
12    }
13 }
```

这段代码难测，因为输入来自文件，输出到终端，核心逻辑藏在中间。重构第一步不是改算法，而是找到可分离的纯逻辑。

24.2 先建立外部行为测试

如果暂时无法拆，可以先用端到端测试固定行为：准备小文件，运行程序，比较输出。这个测试可能慢，但它能保护你第一轮重构。然后逐步抽函数。

第一刀通常是把“清洗单词”抽出来：

Listing 24.2 先抽纯函数

```
1 std::string normalize_word(std::string_view word)
2 {
3     std::string result;
4     for (unsigned char ch : word) {
5         if (std::isalnum(ch)) {
6             result.push_back(static_cast<char>(std::tolower(ch)));
7         }
8     }
9     return result;
10 }
```

这个函数可以直接测试，不需要文件和终端。每抽出一个纯函数，系统就多一个稳定支点。

24.3 特征测试和单元测试配合

重构时常用两类测试：

- 特征测试：从外部固定现有行为，哪怕内部很乱。
- 单元测试：针对抽出来的小函数验证边界。

先用特征测试防止大方向跑偏，再用单元测试支撑细节。不要一开始就追求完美测试结构；面对旧代码，第一目标是建立安全网。

24.4 依赖注入让 I/O 可替换

把文件路径改成输入流，就能测试内存字符串：

Listing 24.3 把输入流作为依赖

```
1 WordCounts count_words_from_stream(std::istream& input)
2 {
3     WordCounts counts;
4     std::string word;
5     while (input >> word) {
6         const std::string normalized = normalize_word(word);
7         if (!normalized.empty()) {
8             ++counts[normalized];
9         }
10    }
11    return counts;
12 }
```

测试：

Listing 24.4 用字符串流测试

```
1 std::istringstream input{"Cpp cpp, memory"};
2 const WordCounts counts = count_words_from_stream(input);
3 assert(counts.at("cpp") == 2);
4 assert(counts.at("memory") == 1);
```

依赖注入不是一定要上框架。把 `std::istream&`、`std::ostream&`、接口引用或函数对象作为参数，就是最朴素的依赖注入。

24.5 重构步骤要小

一次可靠重构可以按这个节奏：

1. 写一个能失败的测试，或固定当前行为。
2. 抽出一个小函数，不改行为。
3. 运行测试。
4. 给小函数补边界测试。
5. 再抽下一步。

不要一边改结构，一边改需求，一边换算法。失败时你不知道是哪一步引入问题。高质量重构靠小步和反馈，不靠一次性大改。

24.6 命名是设计反馈

如果你抽出的函数很难命名，通常说明边界不清。比如 `handle_data`、`process_stuff`、`do_work` 都是危险信号。好名字应该能说明输入和输出之间的关系：

- `parse_config_line`: 解析一行配置。
- `count_words`: 统计单词。
- `top_words`: 取最高频单词。
- `validate_registration`: 校验注册信息。

命名不是表面美化，而是检验函数责任是否单一。

24.7 重构后的性能复查

重构可能改变性能。比如把 `std::string_view` 错改成 `std::string`，可能增加复制；把一次遍历拆成多次遍历，可能增加成本。每次重构后要看两件事：复杂度有没有变，热路径有没有新增大量分配。

如果重构为了可测试性多了一点常数成本，通常可以接受；如果把线性算法变成平方算法，就不能接受。工程判断要明确写出来。

习题与作业

把词频项目的 `read_words_from_file` 重构为两层：`count_words_from_stream` 和 `count_words_in_file`。要求核心统计测试不访问文件系统，文件层只测试打不开文件和正常打开两个路径。

24.8 从遗留函数建立安全网

重构最怕“我以为没改行为”。面对遗留函数，第一步可以写特征测试，即使输出格式不完美，也先固定下来。假设原函数对输入 `"Cpp cpp, Cpp!"` 输出：

Listing 24.5 当前词频输出

```
1 cpp 3
```

那就把这个行为写进测试。等安全网存在后，再重构内部。如果需求后来决定保留标点或区分大小写，那是需求变更，应单独改测试，而不是混在重构里。

24.9 测试命名要像规格

坏测试名：

Listing 24.6 弱测试名

```
1 void test_count_words();
```

好测试名应该说清场景和期望：

Listing 24.7 测试名表达规格

```
1 void count_words_merges_case_and_ignores_punctuation();
2 void count_words_ignores_empty_normalized_tokens();
3 void count_words_reports_two_distinct_words();
```

测试失败时，名字就是第一条诊断信息。不要让维护者打开测试文件后才知道它想保护什么。

24.10 先列测试矩阵再拆函数

以 `normalize_word` 为例，测试矩阵可以这样列：

场景	输入	期望
小写单词	<code>cpp</code>	<code>cpp</code>

大小写混合	Cpp	cpp
尾部标点	cpp,	cpp
全标点	!!!	空字符串
数字保留	c20	c20

矩阵能反推函数责任：它只负责单词规范化，不负责读取、不负责统计、不负责排序。函数责任越清楚，测试越小，重构越稳。

24.11 重构中的提交节奏

一次大重构可以拆成多个可验证提交：

1. 增加特征测试，不改生产代码。
2. 抽 `normalize_word`，行为不变。
3. 抽 `count_words_from_stream`，行为不变。
4. 抽输出函数，行为不变。
5. 在已有安全网下改需求或优化。

每一步都能构建和测试。这样如果第三步失败，你不用回看一千行 diff。高质量重构最重要的不是“最后结构好看”，而是过程可回退、可验证。

24.12 覆盖率不是数字游戏

覆盖率能帮助发现没测到的路径，但不能替代测试质量。一个只调用函数不检查结果的测试也能提高覆盖率，却没有保护价值。真正要关注的是高风险路径：

- 空输入。
- 边界值。
- 错误输入。
- 资源失败。
- 并发关闭。
- 排序和去重的相等情况。

如果项目有覆盖率工具，可以把未覆盖行和测试矩阵对照：未覆盖的是无意义防御代码，还是缺了真实场景？后者要补测试，前者可能要重构掉。

24.13 重构后做一次自审

每次重构结束，按四个问题自审：

1. 行为是否由测试固定？
2. 新函数是否有单一责任和清楚名字？
3. I/O 是否被推到边缘？
4. 性能复杂度和复制次数是否没有明显变差？

这四项都过，重构才算完成。否则只是把代码搬了位置。

第 25 章

CMake 依赖、安装边界和 CI

25.1 问题入口：项目在别人机器上构建不了

本机能构建不代表项目健康。常见问题包括：头文件路径靠 IDE 隐式配置，库路径写死在本机目录，测试命令只存在个人脚本里，依赖版本没有记录。工程化的目标是让项目在新机器上用少量命令复现。

25.2 目标级依赖

CMake 里应围绕 target 建模：

Listing 25.1 目标级依赖

```
1 add_library(config_loader src/config_loader.cpp)
2 target_compile_features(config_loader PUBLIC cxx_std_20)
3 target_include_directories(config_loader PUBLIC include)
4
5 add_executable(config_loader_tests tests/config_loader_tests.cpp)
6 target_link_libraries(config_loader_tests PRIVATE config_loader)
```

PUBLIC 表示使用者也需要这个属性，比如公开头文件路径。**PRIVATE** 表示只对当前目标有效。不要全局 **include_directories**，它会让模块意外依赖不该看到的路径。

25.3 库目标和可执行目标分开

如果所有逻辑都写在 **main.cpp**，测试很难复用。把核心逻辑做成库：

Listing 25.2 库和程序分开

```
1 add_library(wordfreq_core
2     src/split_words.cpp
3     src/count_words.cpp
4     src/top_words.cpp)
5 target_include_directories(wordfreq_core PUBLIC include)
6 target_compile_features(wordfreq_core PUBLIC cxx_std_20)
7
8 add_executable(wordfreq src/main.cpp)
9 target_link_libraries(wordfreq PRIVATE wordfreq_core)
10
11 add_executable(wordfreq_tests tests/wordfreq_tests.cpp)
12 target_link_libraries(wordfreq_tests PRIVATE wordfreq_core)
```

程序和测试都链接同一个核心库。这样测试覆盖的是生产代码，而不是复制出来的一份逻辑。

25.4 依赖版本要明确

如果项目需要第三方库，可以用系统包、子模块、包管理器或 CMake FetchContent。无论方式如何，都要记录版本。不要让“最新版本”成为隐式依赖。今天能构建，半年后最新版本变了，行为也可能变。

对小项目，尽量先用标准库。引入依赖要回答：

- 它解决的问题是否足够大？
- 是否有稳定版本和许可证？
- 构建和交叉平台成本是多少？
- 如果依赖停止维护，替换成本多高？

25.5 安装和导出不是第一天必需，但边界要清楚

库如果要给其他项目使用，需要安装头文件和库文件：

Listing 25.3 安装目标示意

```
1 install(TARGETS wordfreq_core
2     ARCHIVE DESTINATION lib
3     LIBRARY DESTINATION lib
4     RUNTIME DESTINATION bin)
5
6 install(DIRECTORY include/ DESTINATION include)
```

教学项目不一定要完整安装规则，但目录结构应该为未来留余地：公开头文件在 `include`，内部实现不暴露。

25.6 CI 最小流程

持续集成不是大公司专属。最小 CI 只需要在每次提交后运行：

1. 配置项目。
2. 编译。
3. 运行测试。
4. 可选：运行格式检查、静态分析、消毒器构建。

伪命令：

Listing 25.4 CI 核心命令

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
2 cmake --build build --parallel
3 ctest --test-dir build --output-on-failure
```

如果这些命令不能在干净环境运行，说明项目还有隐式依赖。

25.7 构建类型矩阵

常见矩阵：

- Debug：快速编译，适合单元测试。
- Release：发现优化下的问题，跑性能 smoke test。
- ASan/UBSan：发现内存和未定义行为。
- TSan：发现数据竞争，适合并发模块。

不一定每次提交都跑全部矩阵。可以把快速测试放在每次提交，慢测试放在夜间或发布前。关键是规则明确。

25.8 构建失败也要可诊断

CI 失败时，输出要足够定位问题。测试名字不要叫 `test1`；断言失败要能说明输入和期望；构建脚本不要吞掉错误。一个好的失败信息应该让维护者知道下一步看哪个模块。

习题与作业

给配置加载器项目写一个 CMake 结构：核心库、命令行程序、测试目标。要求测试只链接核心库，不运行命令行程序。再设计一个 CI 命令清单，包含 Debug 测试和 ASan 测试。

25.9 target 是构建系统里的对象

学习 CMake 时不要把它当成“命令脚本”。现代 CMake 的核心单位是 target。库、可执行文件、测试程序都是 target；include 路径、编译特性、链接库都挂在 target 上。这样依赖才能沿 target 传播。

反例：

Listing 25.5 全局 include 路径会污染依赖

```
1 include_directories(include)
2 add_executable(app src/main.cpp)
```

这个写法让后续所有目标都能看到 include，即使它们不应该依赖这个目录。目标级写法更清楚：

Listing 25.6 目标自己声明公开头文件路径

```
1 add_library(config_core src/config.cpp)
2 target_include_directories(config_core PUBLIC include)
3
4 add_executable(app src/main.cpp)
5 target_link_libraries(app PRIVATE config_core)
```

app 通过链接 config_core 获得它的公开 include 路径。这就是构建层面的接口传播。

25.10 PUBLIC、PRIVATE、INTERFACE 怎么判断

判断规则很具体：

可见性	含义	例子
PRIVATE	只当前 target 需要	.cpp 里用到的库或 include
PUBLIC	当前 target 和使用都需要	公开头文件里出现的类型所在 include
INTERFACE	当前 target 不编译，只给使用者传播	头文件库、编译选项集合

如果公开头文件里写了 `##include <span>` 并在函数签名使用 `std::span`，那么 C++20 特性是 PUBLIC；如果某个实现文件内部使用 `std::filesystem`，但公开接口没有暴露相关类型，它可能只是 PRIVATE。

25.11 ASan 构建不是普通 Debug

内存错误可以用 AddressSanitizer 辅助发现。一个简单配置：

Listing 25.7 ASan 构建命令

```
1 cmake -S . -B build-asan \
2   -DCMAKE_BUILD_TYPE=Debug \
3   -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer"
```

```
4 cmake --build build-asan --parallel
5 ctest --test-dir build-asan --output-on-failure
```

真实项目最好把 sanitizers 做成 CMake option，而不是长期手写 flags。教材这里先让你看到：测试不是只有一种构建。普通 Debug 能过，不代表没有越界、use after free 或未定义行为。

25.12 CI 日志也要可读

CI 失败有两类：构建失败和测试失败。构建失败要能看到编译命令、源文件和第一条错误；测试失败要能看到测试名、输入、期望和实际。不要让测试输出只有：

Listing 25.8 低质量失败信息

```
1 assertion failed
```

更好的失败信息：

Listing 25.9 可诊断失败信息

```
1 active_user_ids_handles_duplicate_ids:
2 expected [3, 7], actual [3, 3, 7]
```

这不是测试框架选择问题，而是工程习惯。失败信息越具体，修复越快。

25.13 依赖引入决策记录

每引入一个第三方库，至少写下四点：

- 用它解决什么问题。
- 锁定哪个版本。
- 许可证是否允许项目使用。
- 如果移除它，替代方案是什么。

很多小项目最后构建困难，不是代码难，而是依赖来源和版本没人记得。标准库能解决的先用标准库；依赖能显著降低复杂度时再引入，并把版本写进构建文件或文档。

25.14 本章验收：新机器三条命令能跑

一个健康小项目，换到新机器后至少能用三条命令跑起来：

Listing 25.10 新机器验收命令

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
2 cmake --build build --parallel
3 ctest --test-dir build --output-on-failure
```

如果还需要手动设置一堆 IDE 路径、复制本机库、猜依赖版本，说明构建边界还没完成。

第零部分

高级专题：并发、异步和大型代码组织

第 26 章

并发、线程和同步

26.1 问题入口：两个线程同时计数

并发最容易制造“偶尔错”的程序。看一个简单计数器：

Listing 26.1 数据竞争

```
1 #include <thread>
2
3 int counter = 0;
4
5 void increment_many()
6 {
7     for (int i = 0; i < 100000; ++i) {
8         ++counter;
9     }
10 }
```

如果两个线程同时执行，结果未必是 200000。`++counter` 不是不可分割动作，它通常包含读取、加一、写回。两个线程交错时会丢更新。这是数据竞争，行为不可靠。

26.2 mutex 保护共享状态

Listing 26.2 用 mutex 保护计数器

```
1 #include <mutex>
2
3 class Counter {
4 public:
5     void increment()
6     {
7         std::lock_guard<std::mutex> lock{this->mutex_};
8         ++this->value_;
9     }
10
11     [[nodiscard]] int value() const
12     {
13         std::lock_guard<std::mutex> lock{this->mutex_};
14         return this->value_;
15     }
16
17 private:
18     mutable std::mutex mutex_;
19     int value_ = 0;
20 };
```

`std::lock_guard` 是 RAII：构造时加锁，析构时解锁。即使中途抛异常，也会释放锁。

26.3 atomic 适合简单共享变量

简单计数可以用原子类型：

Listing 26.3 atomic 计数

```

1 #include <atomic>
2
3 std::atomic<int> counter{0};
4
5 void increment_many()
6 {
7     for (int i = 0; i < 100000; ++i) {
8         counter.fetch_add(1, std::memory_order_relaxed);
9     }
10 }
```

`memory_order_relaxed` 只保证计数本身原子，不建立额外同步关系。入门阶段不要随意选择内存序。默认顺序一致最容易推理；只有在测量证明需要时，才进入更细的内存序设计。

26.4 条件变量表达等待

线程间等待不要忙等。生产者消费者可以用 `std::condition_variable`。

Listing 26.4 条件变量等待队列非空

```

1 #include <condition_variable>
2 #include <mutex>
3 #include <queue>
4
5 class WorkQueue {
6 public:
7     void push(int value)
8     {
9         {
10             std::lock_guard<std::mutex> lock{this->mutex_};
11             this->queue_.push(value);
12         }
13         this->ready_.notify_one();
14     }
15
16     int pop()
17     {
18         std::unique_lock<std::mutex> lock{this->mutex_};
19         this->ready_.wait(lock, [this] {
20             return !this->queue_.empty();
21         });
22         const int value = this->queue_.front();
23         this->queue_.pop();
24         return value;
25     }
26
27 private:
28     std::mutex mutex_;
29     std::condition_variable ready_;
```

```

30     std::queue<int> queue_;
31 };

```

`wait` 必须带谓词，因为线程可能被虚假唤醒。谓词是等待条件的正式表达。

26.5 线程退出也是协议

生产者消费者队列如果没有退出协议，消费者可能永远阻塞。可以加入关闭标志：

Listing 26.5 带关闭协议的队列片段

```

1  class ClosableQueue {
2  public:
3      void close()
4      {
5          {
6              std::lock_guard<std::mutex> lock{this->mutex_};
7              this->closed_ = true;
8          }
9          this->ready_.notify_all();
10     }
11
12     std::optional<int> pop()
13     {
14         std::unique_lock<std::mutex> lock{this->mutex_};
15         this->ready_.wait(lock, [this] {
16             return this->closed_ || !this->queue_.empty();
17         });
18         if (this->queue_.empty()) {
19             return std::nullopt;
20         }
21         const int value = this->queue_.front();
22         this->queue_.pop();
23         return value;
24     }
25
26 private:
27     std::mutex mutex_;
28     std::condition_variable ready_;
29     std::queue<int> queue_;
30     bool closed_ = false;
31 };

```

关闭不是简单地杀线程，而是让等待方能观察到“不会再有新任务”的状态，并自然退出循环。

26.6 本章边界检查

写并发代码时至少检查：

- 哪些状态被多个线程共享？
- 每个共享状态由哪个锁或原子保护？
- 是否存在锁顺序反转导致死锁？
- 条件变量是否用谓词等待？
- 线程退出和资源释放路径是否明确？

核心思想

并发编程的第一原则是减少共享。能在线程内拥有的数据，不共享；必须共享的数据，用清晰同步边界保护。

26.7 从一次丢更新推导数据竞争

`++counter` 可以拆成三步：读取旧值、计算新值、写回新值。假设两个线程都看到旧值是 10：

Listing 26.6 一次丢更新的交错过程

```
1 线程 A: read counter -> 10
2 线程 B: read counter -> 10
3 线程 A: write 11
4 线程 B: write 11
```

执行了两次自增，结果却只增加一次。这不是“线程不够快”，而是没有同步保护共享状态。只要两个线程同时访问同一内存位置，至少一个写，并且没有同步，就形成数据竞争。

26.8 锁保护的是不变量，不是一行代码

给计数器加锁很简单，但真实对象往往有多个成员。锁要保护的是这些成员之间的不变量。比如银行账户转账，不能只给扣款和加款分别加锁后随意组合，还要考虑两个账户的锁顺序。

死锁的典型反例：

Listing 26.7 锁顺序反转可能死锁

```
1 void transfer(Account& from, Account& to, int amount)
2 {
3     std::lock_guard<std::mutex> first{from.mutex};
4     std::lock_guard<std::mutex> second{to.mutex};
5     from.balance -= amount;
6     to.balance += amount;
7 }
```

如果线程一执行 `transfer(a, b, 10)`，线程二同时执行 `transfer(b, a, 20)`，两个线程可能各自拿住一个锁，然后等待对方释放。可以用 `std::scoped_lock` 一次锁住多个互斥量：

Listing 26.8 作用域锁同时获取多个锁

```
1 void transfer(Account& from, Account& to, int amount)
2 {
3     std::scoped_lock lock{from.mutex, to.mutex};
4     from.balance -= amount;
5     to.balance += amount;
6 }
```

更完整的设计还要处理 `from` 和 `to` 是同一个账户、余额不足、异常安全等业务规则。并发代码不能脱离不变量讨论。

26.9 jthread 让线程生命周期更安全

C++20 提供 `std::jthread`，析构时会请求停止并自动 join。相比裸 `std::thread`，它更符合 RAII：

Listing 26.9 jthread 自动 join

```

1 #include <chrono>
2 #include <thread>
3
4 void worker(std::stop_token token)
5 {
6     while (!token.stop_requested()) {
7         do_one_unit_of_work();
8     }
9 }
10
11 void run_background()
12 {
13     std::jthread thread{worker};
14     std::this_thread::sleep_for(std::chrono::seconds{1});
15 } // thread 析构时请求停止并 join

```

这不等于可以随便启动后台线程。停止请求只是协作信号，工作函数必须定期检查它。如果线程阻塞在条件变量或 I/O 上，还需要设计唤醒和关闭协议。

26.10 条件变量为什么要谓词

条件变量可能虚假唤醒，也可能在通知前条件已经变化。谓词把“醒来后是否真的能继续”写成可重复检查的条件：

Listing 26.10 wait 等价于循环检查

```

1 while (queue.empty() && !closed) {
2     ready.wait(lock);
3 }

```

带谓词的 `wait` 帮你写出这个循环。不要用 `if` 替代 `while`，因为醒来不代表条件一定满足。

习题与作业

给 `ClosableQueue` 增加一个 `push` 规则：关闭后不允许再插入。设计返回值或异常策略，并测试三个场景：正常 `push/pop`、关闭空队列、关闭后 `push`。

26.11 并发事故从时间线开始

并发 bug 的难点是“代码顺序”和“实际交错顺序”不是一回事。两个线程丢更新的时间线可以写得更细：

时刻	线程 A	线程 B	共享值
1	读取 counter		10
2		读取 counter	10
3	计算 11		10
4		计算 11	10
5	写回 11		11
6		写回 11	11

这个表说明，问题不在循环次数，也不在某个线程“抢占了 CPU”。问题是两个线程同时读写同一个共享状态，且没有同步建立互斥或顺序。任何并发问题都先写时间线：谁读，谁写，哪个状态被共享，哪个同步原语保护它。

26.12 锁保护的是状态集合

只说“给变量加锁”太粗。锁真正保护的是一组状态和它们之间的不变量。以关闭队列为例，共享状态至少有两个：队列内容和关闭标志。

Listing 26.11 队列内容和关闭状态必须一起保护

```
1 std::queue<int> queue_;
2 bool closed_ = false;
3 std::mutex mutex_;
```

不变量包括：关闭后消费者如果发现队列空，应退出；关闭后是否允许生产者继续 push，要有明确规则；通知必须发生在状态改变之后。于是 `closed_` 和 `queue_` 应该由同一个互斥量保护。把队列用一个锁、关闭标志用另一个锁，或者把关闭标志改成 `atomic` 而队列仍用 `mutex`，都会让推理复杂很多。

26.13 push 也需要关闭协议

前面的 `ClosableQueue` 只展示了 `close` 和 `pop`。完整队列还要定义关闭后 `push` 怎么办。这里选择返回 `false`：

Listing 26.12 关闭后拒绝插入

```
1 bool push(int value)
2 {
3     {
4         std::lock_guard<std::mutex> lock{this->mutex_};
5         if (this->closed_) {
6             return false;
7         }
8         this->queue_.push(value);
9     }
10    this->ready_.notify_one();
11    return true;
12 }
```

为什么通知放在释放锁之后？这样等待线程被唤醒后，更可能直接拿到锁继续执行，减少无谓竞争。放在锁内通常也能正确，但要知道自己在做什么。并发代码里，“正确”和“减少等待”是两层问题，先保证正确，再谈性能。

26.14 等待谓词是协议文档

消费者等待的谓词：

Listing 26.13 等待条件写成谓词

```
1 this->ready_.wait(lock, [this] {
2     return this->closed_ || !this->queue_.empty();
3 });
```

这行代码是协议文档。它告诉读者：消费者醒来的合法原因有两个，要么有任务，要么队列关闭。醒来后还要分支：

Listing 26.14 醒来后区分任务和关闭

```
1 if (this->queue_.empty()) {
```

```

2     return std::nullopt;
3 }
4 const int value = this->queue_.front();
5 this->queue_.pop();
6 return value;

```

如果忘了把 `closed_` 放进谓词，关闭空队列时消费者可能永远睡下去。如果忘了在 `close` 里 `notify_all`，等待者也可能看不到关闭事件。条件变量要同时检查三件事：状态在哪里改，谓词怎么写，谁负责通知。

26.15 线程生命周期要有所有者

启动线程也属于资源管理。裸 `std::thread` 如果对象析构时仍然 `joinable`，会调用 `std::terminate`。所以线程所有者必须定义退出协议：

Listing 26.15 工作线程所有者的基本形状

```

1 class Worker {
2 public:
3     Worker()
4         : thread_{[this](std::stop_token token) {
5             this->run(token);
6         }}
7     {
8     }
9
10 private:
11     void run(std::stop_token token)
12     {
13         while (!token.stop_requested()) {
14             do_one_unit_of_work();
15         }
16     }
17
18     std::jthread thread_;
19 };

```

`std::jthread` 帮你做析构时请求停止和 `join`，但它不能替你设计 `run` 如何从阻塞等待中醒来。如果工作线程在条件变量上等待，析构或关闭函数必须修改共享状态并通知。线程生命周期同样要有账本：谁启动，谁请求停止，谁唤醒，谁 `join`。

26.16 本章验收：给每个共享状态标注保护者

审查并发代码时，在每个共享字段旁边写一句“由谁保护”。例如：`queue_` 和 `closed_` 由 `mutex_` 保护；`requests_` 是 relaxed atomic 统计计数，不保护其他数据；`stop_` 是停止信号，但阻塞等待还需要条件变量通知。标注不出来的共享状态，就是事故候选。

第 27 章

大型代码组织和演进

27.1 问题入口：一个 Manager 类吞掉项目

很多项目会出现一个 **AppManager**：它读配置、连数据库、处理请求、写日志、管理缓存、启动线程。短期看方便，长期看每个修改都要碰它，测试也越来越难。

架构不是画复杂图，而是控制依赖方向。先把系统拆成几个角色：

- **配置**：读取和验证启动参数。
- **领域逻辑**：处理核心业务，不知道文件和网络。
- **适配器**：把文件、数据库、HTTP 等外部世界接进来。
- **入口**：组装对象，建立异常和日志边界。

27.2 依赖接口而不是具体环境

假设业务逻辑需要读取用户信息。不要让它直接依赖文件路径：

Listing 27.1 用接口隔离外部数据源

```
1 #include <optional>
2 #include <string>
3
4 struct UserRecord {
5     int id = 0;
6     std::string name;
7 };
8
9 class UserRepository {
10 public:
11     virtual ~UserRepository() = default;
12     [[nodiscard]] virtual std::optional<UserRecord> find_by_id(int id) const = 0;
13 };
```

业务服务依赖这个接口：

Listing 27.2 业务逻辑依赖抽象

```
1 class GreetingService {
2 public:
3     explicit GreetingService(const UserRepository& repository)
4         : repository_{repository}
5     {
6     }
7
8     [[nodiscard]] std::string greeting_for(int id) const
9     {
10         const std::optional<UserRecord> user = this->repository_.find_by_id(id);
11         if (!user.has_value()) {
12             return "unknown user";
13         }
14         return "hello, " + user->name;
```

```

15     }
16
17 private:
18     const UserRepository& repository_;
19 };

```

测试时可以传入内存实现；生产环境可以传入文件或数据库实现。

27.3 继承要克制

上面的接口用了虚函数，是为了隔离外部数据源。不要把所有变化点都做成继承。很多时候，普通值对象、函数对象、模板参数或构造函数注入更简单。

Listing 27.3 小策略可以用函数对象

```

1 using PriceRule = std::function<int(int base_price)>;
2
3 int final_price(int base_price, const PriceRule& rule)
4 {
5     return rule(base_price);
6 }

```

如果策略在热路径，`std::function` 的类型擦除成本可能需要测量；也可以用模板参数。架构决策要和性能约束一起看。

27.4 模块边界和头文件

头文件是依赖传播的入口。一个头文件包含太多其他头文件，会让编译变慢，也会放大耦合。优先在头文件里暴露必要类型，在源文件里包含实现细节。

Listing 27.4 头文件保持小而稳定

```

1 #pragma once
2
3 #include <optional>
4 #include <string>
5
6 struct UserRecord;
7
8 class UserRepository {
9 public:
10     virtual ~UserRepository() = default;
11     [[nodiscard]] virtual std::optional<UserRecord> find_by_id(int id) const = 0;
12 };

```

不是所有类型都能前置声明，例如按值成员需要完整类型。知道这个边界，才能合理拆分。

27.5 演进优先级

当项目开始变大，优先处理这些问题：

1. 重复业务规则：抽成清晰函数或类型。
2. 隐式资源所有权：改成 RAII 和明确所有者。
3. 巨大函数：按阶段拆分。
4. 巨大类：按角色拆分。
5. 难测试外部依赖：通过接口或适配器隔离。

核心理想

高级 C++ 不是把模板、继承和并发全用上，而是在复杂系统里让依赖方向、资源生命周期和错误边界仍然清楚。

27.6 从难测试代码推导依赖方向

如果 `GreetingService` 自己打开文件、解析数据、拼接问候语，测试一次问候语就要准备文件路径和文件内容。难测试不是测试人员的问题，而是依赖方向暴露了设计问题：业务逻辑依赖了外部环境。

把依赖倒过来后，业务逻辑只依赖 `UserRepository` 抽象。测试可以写一个内存仓库：

Listing 27.5 测试用内存仓库

```
1 class InMemoryUserRepository final : public UserRepository {
2 public:
3     void add(UserRecord user)
4     {
5         this->users_[user.id] = std::move(user);
6     }
7
8     [[nodiscard]] std::optional<UserRecord> find_by_id(int id) const override
9     {
10         const auto found = this->users_.find(id);
11         if (found == this->users_.end()) {
12             return std::nullopt;
13         }
14         return found->second;
15     }
16
17 private:
18     std::unordered_map<int, UserRecord> users_;
19 };
```

这段测试实现不需要文件和数据库。它也逼着接口保持小：如果 `UserRepository` 暴露十几个无关方法，测试替身就会很重。

27.7 入口负责组装，不负责业务

程序入口可以理解成装配层：

Listing 27.6 入口组装真实依赖

```
1 int run_application(const Options& options)
2 {
3     FileUserRepository repository{options.user_file};
4     GreetingService service{repository};
5     std::cout << service.greeting_for(options.user_id) << '\n';
6     return 0;
7 }
```

这里可以出现文件路径、命令行选项、日志和异常边界，因为入口本来就在系统边缘。业务服务内部则不应该知道这些环境细节。这个边界能让核心代码在没有真实环境的情况下测试。

27.8 什么时候不用虚函数

虚函数适合运行期替换实现，例如生产环境和测试环境用不同仓库。如果策略在编译期就确定，模板或函数对象更轻：

Listing 27.7 编译期策略组合

```
1 template <typename DiscountRule>
2 int final_price(int base_price, DiscountRule rule)
3 {
4     return rule(base_price);
5 }
6
7 const int price = final_price(1000, [](int base) {
8     return base - base / 10;
9 });
```

选择虚函数、模板还是 `std::function`，要看变化发生在运行期还是编译期、是否需要类型擦除、是否在热路径、报错和编译时间是否可接受。高级设计不是某个模式永远正确，而是能说出取舍。

27.9 演进时不要大爆炸重写

大型代码最怕“一次性重写”。更稳的演进方式是先找最痛的边界，建立薄接口，然后逐步迁移调用者。例如一个巨大 `AppManager` 可以先抽出配置读取：

Listing 27.8 先抽出一个窄能力

```
1 class ConfigLoader {
2 public:
3     [[nodiscard]] Config load(const std::string& path) const;
4 };
```

下一步再抽日志、缓存、业务服务。每抽一步都要保留测试，让行为不漂移。架构改进不是为了目录好看，而是为了让下一次修改更小、更可验证。

习题与作业

选一个你写过的单文件程序，画出三类代码：输入输出、核心计算、格式化展示。先只抽出核心计算函数并写测试，不要急着引入类。等函数边界稳定后，再判断是否需要对象封装。

第 28 章

继承、多态和类型擦除

28.1 问题入口：多个数据源如何统一使用

假设一个服务需要读取用户信息。测试时从内存读取，生产环境从文件读取，未来可能从数据库读取。最差的写法是在业务逻辑里写分支：

Listing 28.1 业务逻辑直接判断数据源类型

```
1 if (mode == "memory") {  
2     // 从内存找用户  
3 } else if (mode == "file") {  
4     // 从文件找用户  
5 } else if (mode == "database") {  
6     // 从数据库找用户  
7 }
```

这个写法让业务逻辑知道太多环境细节。每新增一种数据源，都要修改业务代码。更好的方向是定义共同接口，让业务只依赖“能按 ID 查用户”这个能力。

28.2 虚函数接口

经典运行期多态用抽象基类：

Listing 28.2 用户仓库接口

```
1 class UserRepository {  
2 public:  
3     virtual ~UserRepository() = default;  
4     [[nodiscard]] virtual std::optional<UserRecord> find_by_id(int id) const = 0;  
5 };
```

内存实现：

Listing 28.3 内存仓库实现

```
1 class InMemoryUserRepository final : public UserRepository {  
2 public:  
3     void add(UserRecord user)  
4     {  
5         this->users_[user.id] = std::move(user);  
6     }  
7  
8     [[nodiscard]] std::optional<UserRecord> find_by_id(int id) const override  
9     {  
10         const auto found = this->users_.find(id);  
11         if (found == this->users_.end()) {  
12             return std::nullopt;  
13         }  
14         return found->second;  
15     }
```

```

16
17 private:
18     std::unordered_map<int, UserRecord> users_;
19 };

```

业务服务只依赖接口：

Listing 28.4 业务逻辑不关心数据源

```

1 class GreetingService {
2 public:
3     explicit GreetingService(const UserRepository& repository)
4         : repository_{repository}
5     {
6     }
7
8     [[nodiscard]] std::string greeting_for(int id) const
9     {
10         const std::optional<UserRecord> user = this->repository_.find_by_id(id);
11         if (!user.has_value()) {
12             return "unknown user";
13         }
14         return "hello, " + user->name;
15     }
16
17 private:
18     const UserRepository& repository_;
19 };

```

虚函数的价值是运行期替换实现。测试和生产可以传不同对象，业务代码不用重新编译成模板实例。

28.3 虚函数背后的函数表模型

虚函数不要只理解成“加一个 `virtual`”。它背后可以近似想成：每个多态对象能找到一张函数表，调用虚函数时先查表，再调用对应实现。标准不规定具体实现必须长这样，但主流实现很接近这个模型。

先看业务调用：

Listing 28.5 通过接口调用虚函数

```

1 std::optional<UserRecord> user = repository.find_by_id(id);

```

可以近似展开成：

Listing 28.6 虚调用的近似展开

```

1 struct UserRepositoryVTable {
2     std::optional<UserRecord> (*find_by_id)(const void* self, int id);
3 };
4
5 struct UserRepositoryObjectShape {
6     const UserRepositoryVTable* vtable = nullptr;
7     void* object = nullptr;
8 };

```

```

9
10 std::optional<UserRecord> call_find_by_id(
11     const UserRepositoryObjectShape& repository,
12     int id)
13 {
14     return repository.vtable->find_by_id(repository.object, id);
15 }

```

派生类会提供自己的表项：

Listing 28.7 派生实现进入函数表

```

1 std::optional<UserRecord> find_in_memory(const void* self, int id)
2 {
3     const auto* repository =
4         static_cast<const InMemoryUserRepository*>(self);
5     return repository->find_by_id(id);
6 }
7
8 const UserRepositoryVTable IN_MEMORY_REPOSITORY_TABLE{
9     .find_by_id = find_in_memory,
10 };

```

真实对象通常不是显式保存 `object` 指针，而是对象本体里带一个隐藏的 `vptr` 指向 `vtable`。这个模型解释了三个现象：

- 虚调用通常不能像普通非虚函数那样直接内联，因为运行期才知道表项。
- 多态基类需要虚析构，因为删除时也要通过表找到正确的派生析构逻辑。
- 不同派生类可以放进同一个接口引用或指针里，因为调用点只依赖表形状。

这也说明继承的核心不是“共享字段”，而是“共享一组可被运行期分发的操作”。

28.4 继承不是为了复用字段

很多继承层次出问题，是因为把继承当成“共享成员变量”的工具。比如 `Cat` 和 `Dog` 都有名字，就让它们继承 `AnimalBase`。如果没有真正的多态行为，这种继承往往只会增加耦合。

优先组合：

Listing 28.8 组合共享数据

```

1 struct NamedEntity {
2     std::string name;
3 };
4
5 class Cat {
6 public:
7     explicit Cat(std::string name)
8         : identity_{std::move(name)}
9     {
10    }
11
12 private:
13     NamedEntity identity_;
14 };

```

继承应当表达“可以通过基类接口使用派生对象”。如果没有这个需求，不要为了少写一个字段引入继承。

28.5 虚析构函数为什么必要

如果通过基类指针删除派生对象，基类析构函数必须是虚的：

Listing 28.9 多态基类需要虚析构

```
1 class Plugin {
2 public:
3     virtual ~Plugin() = default;
4     virtual void run() = 0;
5 };
```

否则删除 `Plugin*` 时，派生类析构可能不执行，资源泄漏。只要一个类有虚函数并打算作为基类使用，虚析构通常就是必要的。

28.6 override 和 final

`override` 让编译器检查你确实重写了基类虚函数：

Listing 28.10 `override` 防止签名写错

```
1 class FileUserRepository final : public UserRepository {
2 public:
3     [[nodiscard]] std::optional<UserRecord> find_by_id(int id) const override;
4 };
```

如果你不小心少写 `const`，编译器会报错，而不是悄悄定义一个新函数。`final` 表示这个类不再作为基类继续派生，能让设计意图更明确。

28.7 类型擦除：不暴露具体类型

`std::function` 是一种类型擦除：调用者只知道它能按某个签名调用，不知道里面是函数指针、lambda 还是函数对象。我们也可以用它替代简单接口：

Listing 28.11 用函数包装查找能力

```
1 using FindUser = std::function<std::optional<UserRecord>(int)>;
2
3 class GreetingService2 {
4 public:
5     explicit GreetingService2(FindUser find_user)
6         : find_user_{std::move(find_user)}
7     {
8     }
9
10    [[nodiscard]] std::string greeting_for(int id) const
11    {
12        const std::optional<UserRecord> user = this->find_user_(id);
13        if (!user.has_value()) {
14            return "unknown user";
15        }
16        return "hello, " + user->name;
17    }
```



```

18
19 private:
20     FindUser find_user_;
21 };

```

这种方式适合只有一两个操作的小接口。操作很多、需要共享状态和清晰生命周期时，抽象基类可能更合适。类型擦除不是继承的替代宗教，而是另一个工具。

28.8 std::function 背后的类型擦除

`std::function` 能保存普通函数、lambda、函数对象，是因为它把“具体类型”藏起来，只保留“如何调用、如何复制、如何销毁”这组操作。可以用一个简化版模型理解：

Listing 28.12 类型擦除的近似形状

```

1 class AnyFindUser {
2 public:
3     template <typename Callable>
4     explicit AnyFindUser(Callable callable)
5         : object_{new Callable{std::move(callable)}},
6           table_{&table_for<Callable>()}
7     {
8     }
9
10    ~AnyFindUser()
11    {
12        this->table_->destroy(this->object_);
13    }
14
15    [[nodiscard]] std::optional<UserRecord> operator()(int id) const
16    {
17        return this->table_->call(this->object_, id);
18    }
19
20 private:
21     struct Table {
22         std::optional<UserRecord> (*call)(const void* object, int id);
23         void (*destroy)(void* object);
24     };
25
26     template <typename Callable>
27     static const Table& table_for()
28     {
29         static const Table table{
30             .call = [](const void* object, int id) {
31                 const auto* callable = static_cast<const Callable*>(object);
32                 return (*callable)(id);
33             },
34             .destroy = [](void* object) {
35                 delete static_cast<Callable*>(object);
36             },
37         };
38         return table;
39     }

```

```

40
41     void* object_ = nullptr;
42     const Table* table_ = nullptr;
43 };

```

真实 `std::function` 还要处理复制、移动、小对象优化、空状态和异常安全，但骨架就是这两部分：一块不知道真实类型的对象存储，一张知道怎么操作它的表。

这个模型能推导出选择边界：如果调用发生在热循环里，而且 callable 类型编译期已知，模板参数常常更便宜；如果需要把不同 callable 放进同一个容器、跨模块保存、运行期替换，那么类型擦除的成本换来了接口稳定。

28.9 variant：封闭集合的多态

如果可能类型是封闭的，比如输入可以是文件、标准输入或内存字符串，可以用 `std::variant`：

Listing 28.13 封闭类型集合用 `variant`

```

1 struct FileInput {
2     std::filesystem::path path;
3 };
4
5 struct StdinInput {
6 };
7
8 struct MemoryInput {
9     std::string text;
10 };
11
12 using InputSource = std::variant<FileInput, StdinInput, MemoryInput>;

```

处理时用 `std::visit`：

Listing 28.14 访问 `variant`

```

1 std::string load_input(const InputSource& source)
2 {
3     return std::visit([](const auto& item) -> std::string {
4         return load_from(item);
5     }, source);
6 }

```

新增一种输入类型时，编译器会帮助你找到需要处理的地方。虚函数适合开放集合，`variant` 适合封闭集合。先判断变化方向，再选机制。

28.10 variant 背后是标签加存储

`std::variant<FileInput, StdinInput, MemoryInput>` 可以近似理解成两部分：一个标签记录当前是哪种类型，一块足够大的存储放当前对象：

Listing 28.15 `variant` 的近似状态

```

1 enum class InputTag {
2     file,
3     stdin_input,
4     memory,

```

```

5 };
6
7 class InputSourceShape {
8 public:
9     InputTag tag = InputTag::stdin_input;
10    // storage must be large and aligned enough for every alternative
11    std::aligned_storage_t<64, 8> storage;
12 };

```

`std::visit` 的近似逻辑就是根据标签分发：

Listing 28.16 `visit` 的近似分发

```

1 std::string load_input_shape(const InputSourceShape& source)
2 {
3     switch (source.tag) {
4     case InputTag::file:
5         return load_from(read_as<FileInput>(source.storage));
6     case InputTag::stdin_input:
7         return load_from(read_as<StdinInput>(source.storage));
8     case InputTag::memory:
9         return load_from(read_as<MemoryInput>(source.storage));
10    }
11    std::terminate();
12 }

```

这个展开模型解释了 `variant` 和虚函数的根本差异。虚函数把“新增类型”变容易：新增派生类，旧调用点不动；`variant` 把“穷尽处理”变明确：所有类型写在一个类型列表里，访问点必须考虑每种情况。如果输入来源由你自己的程序封闭控制，`variant` 很合适；如果插件或外部模块未来会不断新增实现，虚函数接口通常更合适。

28.11 多态选择表

场景	机制	取舍
运行期替换多个实现	虚函数接口	稳定边界，有间接调用成本
小型回调或策略	<code>std::function</code>	简单灵活，可能类型擦除开销
编译期已知策略	模板参数	可内联，编译膨胀和报错更重
封闭类型集合	<code>std::variant</code>	编译期穷尽检查，新增类型需改访问点
共享少量字段	组合	耦合低，不提供运行期多态

习题与作业

把问候服务分别用虚函数接口、`std::function` 和 `std::variant` 写三版。比较测试代码、生产组装代码和新增一种数据源时要改哪些地方。不要只看代码行数，要看依赖方向和错误暴露位置。

第 29 章

异步边界、取消和背压

29.1 问题入口：任务提交得比处理快怎么办

任务队列项目里，生产者可以无限 **push**。如果生产速度长期超过消费速度，内存会一直增长。并发系统不能只问“怎么让线程跑起来”，还要问“过载时怎么办”。这就是背压：系统用某种方式让上游知道下游处理不过来。

最简单的背压是有界队列：

Listing 29.1 有界队列状态

```
1 class BoundedQueue {
2 public:
3     explicit BoundedQueue(std::size_t capacity)
4         : capacity_{capacity}
5     {
6     }
7
8 private:
9     std::size_t capacity_ = 0;
10    std::queue<Task> tasks_;
11    bool closed_ = false;
12    std::mutex mutex_;
13    std::condition_variable not_empty_;
14    std::condition_variable not_full_;
15 };
```

有界队列必须定义队列满时的行为：阻塞等待、返回失败、丢弃旧任务还是丢弃新任务。不同业务答案不同。

29.2 有界队列背后的状态机

队列满不只是一个 **if**，它是一组状态转换。一个有界队列至少有四类状态：

状态	条件	允许的动作
空且未关闭	<code>tasks_.empty()</code>	消费者等待，生产者可提交
非空且未滿	<code>0 < size < capacity</code>	生产者和消费者都可继续
满且未关闭	<code>size == capacity</code>	生产者按策略阻塞、失败或丢弃
已关闭	<code>closed_ == true</code>	唤醒等待者，拒绝新提交，消费剩余任务

写并发代码时，先把状态表列出来，再写条件变量谓词。否则很容易漏掉“关闭时唤醒等待者”这种边界。比如生产者等待“未滿”，但关闭也应该让它醒来：

Listing 29.2 等待条件包含关闭状态

```
1 this->not_full_.wait(lock, [this] {
2     return this->closed_ || this->tasks_.size() < this->capacity_;
3 });
```

消费者等待“非空”，但关闭也应该让它醒来：

Listing 29.3 消费者等待条件也包含关闭状态

```

1 this->not_empty_.wait(lock, [this] {
2     return this->closed_ || !this->tasks_.empty();
3 });

```

这两个谓词不是模板，而是由状态表推出来的结果：任何可能让等待者继续判断下一步的状态变化，都必须通知等待者。

29.3 阻塞提交

阻塞提交会等待队列有空间：

Listing 29.4 队列满时等待

```

1 bool push(Task task)
2 {
3     std::unique_lock<std::mutex> lock{this->mutex_};
4     this->not_full_.wait(lock, [this] {
5         return this->closed_ || this->tasks_.size() < this->capacity_;
6     });
7     if (this->closed_) {
8         return false;
9     }
10    this->tasks_.push(std::move(task));
11    this->not_empty_.notify_one();
12    return true;
13 }

```

这里返回 `false` 表示队列已关闭。等待谓词同时检查关闭和容量，否则关闭后生产者可能永久阻塞。

29.4 消费后通知空间

消费者取走任务后，要通知可能等待的生产者：

Listing 29.5 取任务后通知未满足条件

```

1 std::optional<Task> pop()
2 {
3     std::unique_lock<std::mutex> lock{this->mutex_};
4     this->not_empty_.wait(lock, [this] {
5         return this->closed_ || !this->tasks_.empty();
6     });
7     if (this->tasks_.empty()) {
8         return std::nullopt;
9     }
10    Task task = std::move(this->tasks_.front());
11    this->tasks_.pop();
12    this->not_full_.notify_one();
13    return task;
14 }

```

两个条件变量分别表达“非空”和“未满足”。用一个条件变量也能做，但两个名字更清楚。并发代码里的名字是文档，能减少误用。

29.5 取消是协作协议

C++20 的 `std::stop_token` 表达协作取消。它不会强行杀线程，工作代码必须检查：

Listing 29.6 检查停止请求

```
1 void worker(std::stop_token token, BoundedQueue& queue)
2 {
3     while (!token.stop_requested()) {
4         std::optional<Task> task = queue.pop();
5         if (!task.has_value()) {
6             return;
7         }
8         (*task)();
9     }
10 }
```

如果 `pop` 会长期阻塞，还需要在停止时关闭队列或通知条件变量。取消信号和阻塞等待必须配套设计。

29.6 future 的异常传播

异步任务如果抛异常，调用者应该能拿到。可以用 `std::promise` 和 `std::future`：

Listing 29.7 用 `promise` 传递异常

```
1 std::future<void> submit(TaskQueue& queue, std::function<void()> task)
2 {
3     auto promise = std::make_shared<std::promise<void>>();
4     std::future<void> future = promise->get_future();
5     queue.push([promise, task = std::move(task)] {
6         try {
7             task();
8             promise->set_value();
9         } catch (...) {
10             promise->set_exception(std::current_exception());
11         }
12     });
13     return future;
14 }
```

调用者执行 `future.get()` 时，如果任务失败，会重新抛出异常。这个边界比在工作线程里打印错误更可组合。

29.7 flush 也是一种消息协议

异步系统里，“等前面的任务都处理完”不能靠睡眠。睡眠只是猜测后台线程可能处理完了。更可靠的做法是把 `flush` 变成队列里的一个特殊消息：当工作线程处理到这条消息时，说明它之前的消息都已经处理过。

Listing 29.8 `flush` 消息的队列协议

```
1 struct FlushRequest {
2     std::promise<void> done;
3 };
4
```

```
5 using QueueItem = std::variant<Task, FlushRequest>;
```

提交 `flush`:

Listing 29.9 提交 `flush` 并等待完成

```
1 void flush(BoundedQueue& queue)
2 {
3     auto request = std::make_shared<std::promise<void>>>();
4     std::future<void> done = request->get_future();
5
6     queue.push([request] {
7         request->set_value();
8     });
9
10    done.get();
11 }
```

这个简化版本把 `flush` 做成一个任务。只要队列保持 FIFO, `flush` 任务执行时, 它前面的任务已经执行完。真实日志器里还要在设置 `promise` 前刷新文件缓冲。这里的关键推导是: 异步等待点要通过明确的同步对象建立因果关系, 而不是靠时间估计。

29.8 异步接口要说明生命周期

如果异步任务捕获引用, 就要保证引用对象活到任务执行结束:

Listing 29.10 异步引用捕获风险

```
1 std::string message = "hello";
2 executor.submit([&message] {
3     write_log(message);
4 });
```

如果 `message` 在任务执行前销毁, `lambda` 就悬空。更安全的是按值捕获:

Listing 29.11 异步任务按值捕获数据

```
1 std::string message = "hello";
2 executor.submit([message = std::move(message)] {
3     write_log(message);
4 });
```

异步边界的保守规则: 任务跨线程、跨时间保存时, 捕获自己需要的数据; 需要共享对象时, 用明确的共享所有权或生命周期管理协议。

29.9 过载策略表

策略	行为	适用场景
阻塞生产者	队列满时等待	不能丢任务, 允许上游变慢
返回失败	队列满时提交失败	调用者能降级或重试
丢弃新任务	满时拒绝新任务	实时刷新类任务
丢弃旧任务	保留最新任务	状态采样、监控刷新
扩容队列	临时吸收峰值	峰值短、内存可控

不要默认无限队列。无限队列只是把过载问题延后成内存问题。

习题与作业

把任务队列改成有界队列，容量为 2。写测试验证：队列满时 `try_push` 返回失败；关闭后等待中的生产者被唤醒；消费者取走任务后生产者可以继续提交。

第 30 章

原子操作和内存模型入门

30.1 问题入口：atomic 不是万能锁

原子计数器很简单：

Listing 30.1 原子计数器

```
1 std::atomic<int> counter{0};
2 counter.fetch_add(1);
```

但如果有两个相关字段，单独把它们都做成 atomic 并不自动保护不变量：

Listing 30.2 多个 atomic 仍可能不一致

```
1 std::atomic<int> used{0};
2 std::atomic<int> capacity{100};
```

如果业务不变量是 `used <= capacity`，你仍然要设计同步方式。atomic 保证单个原子对象的访问规则，不自动把多个字段组成事务。

30.2 数据竞争和逻辑竞争

atomic 可以消除数据竞争，但不一定消除逻辑竞争。比如库存扣减：

Listing 30.3 检查再扣减不是一个原子事务

```
1 if (stock.load() > 0) {
2     stock.fetch_sub(1);
3 }
```

两个线程可能都看到库存为 1，然后都扣减。结果变成 -1。需要用比较交换循环：

Listing 30.4 用比较交换做条件更新

```
1 bool try_take_one(std::atomic<int>& stock)
2 {
3     int current = stock.load();
4     while (current > 0) {
5         if (stock.compare_exchange_weak(current, current - 1)) {
6             return true;
7         }
8     }
9     return false;
10 }
```

`compare_exchange_weak` 失败时会把当前值写回 `current`，循环继续判断。这个例子说明：atomic 代码也要围绕不变量设计。

30.3 memory_order 先保守

默认原子操作使用顺序一致模型，最容易推理。你可以把它理解成所有线程看到一个全局一致的原子操作顺序。它可能不是最快，但通常是正确起点。

`memory_order_relaxed` 只保证这个原子变量本身的读写原子，不建立跨变量同步关系。适合统计计数这类不需要用计数值保护其他数据的场景：

Listing 30.5 relaxed 适合统计计数

```
1 requests.fetch_add(1, std::memory_order_relaxed);
```

如果一个线程写数据，再用 `atomic` 标志通知另一个线程读数据，就不能随便用 `relaxed`。你需要 `release/acquire` 关系。

30.4 release/acquire 的直觉

生产者写数据，然后发布 `ready`：

Listing 30.6 发布数据

```
1 data = make_data();
2 ready.store(true, std::memory_order_release);
```

消费者等待 `ready`，再读数据：

Listing 30.7 获取数据

```
1 while (!ready.load(std::memory_order_acquire)) {
2 }
3 use(data);
```

`release/acquire` 建立同步：消费者看到 `ready == true` 后，也能看到生产者在 `release` 之前完成的数据写入。这个例子只是入门直觉；真实代码里，忙等通常不如条件变量或 `latch`。

30.5 不要用 `atomic` 拼复杂协议

一旦协议涉及队列、多个字段、等待和关闭，互斥量往往更清楚：

Listing 30.8 复杂共享状态优先 `mutex`

```
1 std::mutex mutex;
2 std::queue<Task> tasks;
3 bool closed = false;
```

锁不是低级，`atomic` 也不是高级。锁保护一组不变量；`atomic` 适合简单共享状态或精心设计的无锁结构。无锁代码要有非常强的测试、审查和测量理由。

30.6 伪共享

两个不同 `atomic` 变量如果落在同一个缓存行，两个线程频繁写它们，也可能互相拖慢。这叫伪共享。解决方法可能是分离数据布局或填充到缓存行大小。但不要一上来手写填充，先用 `profiler` 或基准确认问题。

30.7 完整案例：停止标志

一个后台线程只需要知道是否停止，可以用 `atomic bool`：

Listing 30.9 停止标志

```

1 class StopFlag {
2 public:
3     void request_stop() noexcept
4     {
5         this->stopped_.store(true);
6     }
7
8     [[nodiscard]] bool stop_requested() const noexcept
9     {
10        return this->stopped_.load();
11    }
12
13 private:
14     std::atomic<bool> stopped_{false};
15 };

```

如果线程可能阻塞在条件变量上，仅设置标志还不够，还要通知条件变量。停止协议要覆盖“正在运行”和“正在等待”两种状态。

习题与作业

实现一个线程安全库存计数器，提供 `try_take(int count)`。要求不能扣成负数。先用 `mutex` 写一版，再用 `compare_exchange` 写一版。比较哪一版更容易证明正确。

30.8 库存扣减从互斥版本开始

学习 `atomic` 之前，先写互斥版本，因为它更容易证明不变量：

Listing 30.10 互斥量版本库存扣减

```

1 class StockCounter {
2 public:
3     explicit StockCounter(int initial)
4         : stock_{initial}
5     {
6     }
7
8     bool try_take(int count)
9     {
10        std::lock_guard<std::mutex> lock{this->mutex_};
11        if (count <= 0 || this->stock_ < count) {
12            return false;
13        }
14        this->stock_ -= count;
15        return true;
16    }
17
18 private:
19     std::mutex mutex_;
20     int stock_ = 0;
21 };

```

锁内的临界区很短，但不变量清楚：检查和扣减在同一个互斥区域里完成，其他线程看不到中间状态。这个版本也更容易扩展错误消息、审计日志或多字段库存。先能写对这个版本，再考虑 `atomic`。

30.9 比较交换版本的推理

atomic 版本要把“检查旧值”和“写入新值”合成一次条件更新：

Listing 30.11 支持扣减多个库存的比较交换

```

1 class AtomicStockCounter {
2 public:
3     explicit AtomicStockCounter(int initial)
4         : stock_{initial}
5     {
6     }
7
8     bool try_take(int count)
9     {
10        if (count <= 0) {
11            return false;
12        }
13
14        int current = this->stock_.load();
15        while (current >= count) {
16            const int desired = current - count;
17            if (this->stock_.compare_exchange_weak(current, desired)) {
18                return true;
19            }
20        }
21        return false;
22    }
23
24 private:
25     std::atomic<int> stock_;
26 };

```

关键点在 `compare_exchange_weak` 失败后会更新 `current`。所以循环不需要手动再 `load` 一次；它拿到最新观察值后重新判断 `current >= count`。如果另一个线程已经把库存扣到不足，本线程自然返回失败。

这个版本只适合单个整数不变量。如果扣减还要同时写订单记录、限制每个用户购买数量、更新多个仓库，互斥量会更清楚。

30.10 relaxed 统计和发布数据不是一回事

请求计数可以用 relaxed：

Listing 30.12 统计计数不发布其他数据

```

1 requests.fetch_add(1, std::memory_order_relaxed);

```

因为它只回答“发生了多少次”，不要求看到其他内存写入。但下面这个模式不能用 relaxed 当同步：

Listing 30.13 标志发布数据需要同步关系

```

1 payload = build_payload();
2 ready.store(true, std::memory_order_release);

```

消费者：

Listing 30.14 读取标志后使用数据

```

1 if (ready.load(std::memory_order_acquire)) {
2     use(payload);
3 }

```

这里 `ready` 不只是布尔值，它还是“payload 已经写好”的发布证据。release/acquire 的意义就是让消费者看到 `ready` 为真时，也能看到发布前对 `payload` 的写入。把这类标志写成 relaxed，就切断了证据链。

30.11 atomic 不能让复合对象自动安全

两个 atomic 字段仍然可能产生不一致快照：

Listing 30.15 两个 atomic 字段不能自动组成事务

```

1 std::atomic<int> x{0};
2 std::atomic<int> y{0};

```

线程 A 写 `x=1` 再写 `y=1`，线程 B 可能读到 `x=1`，`y=0`。这不是数据竞争，但可能违反业务语义。如果业务要求两个字段一起变化，就需要锁、版本号协议，或把状态编码进一个原子对象。不要把“没有数据竞争”误认为“业务状态一致”。

30.12 内存序选择流程

入门阶段可以按这个流程：

1. 能不用共享就不用共享。
2. 共享的是一组状态和不变量时，优先用 mutex。
3. 共享的是单个计数、标志或指针，并且协议很小，再考虑 atomic。
4. 默认先用顺序一致。
5. 只有测量证明需要，并且能写出 release/acquire 或 relaxed 的证据链，才降低内存序。

这个流程保守，但能让项目活下来。无锁代码的价值在极少数热点路径里很大，但它的证明成本、测试成本和维护成本也很高。

第 31 章

协程入门：异步流程不是线程

31.1 问题入口：回调嵌套为什么难维护

异步 I/O 常见写法是回调：

Listing 31.1 回调嵌套示意

```
1 read_request(socket, [socket](Request request) {  
2     query_database(request.user_id, [socket](User user) {  
3         write_response(socket, make_response(user));  
4     });  
5 });
```

逻辑顺序是“读请求、查数据库、写响应”，但代码结构变成嵌套。错误处理、取消和资源释放会更复杂。协程提供一种把异步流程写得像顺序代码的机制。

31.2 协程不是自动多线程

C++20 协程是语言机制，不等于线程池，也不等于事件循环。协程可以暂停和恢复，但谁来恢复它，取决于 awaiter、调度器或库。没有运行时库，语言本身不会替你完成异步 I/O。

伪代码形态：

Listing 31.2 协程式异步流程示意

```
1 Task<Response> handle(Socket socket)  
2 {  
3     Request request = co_await read_request(socket);  
4     User user = co_await query_database(request.user_id);  
5     co_return make_response(user);  
6 }
```

这段代码表达的是控制流形态，不是本书要实现完整协程库。学习重点是理解 `co_await`、`co_return` 和生命周期边界。

31.3 `co_await` 的直觉

`co_await something` 大致表示：如果结果还没准备好，当前协程可以暂停，把控制权还给调度器；等结果准备好，再从这里继续。它不是阻塞当前线程等待。阻塞会占住线程，暂停协程则可以让线程去做别的工作。

这也是协程适合 I/O 密集任务的原因：大量请求都在等待网络或磁盘时，不需要为每个请求占一个线程。

31.4 协程帧和生命周期

协程暂停时，局部变量必须保存起来。编译器会把协程的状态放进协程帧。于是协程里的引用、指针和视图仍然要遵守生命周期规则：

Listing 31.3 协程中捕获引用的风险示意

```
1 Task<void> bad(std::string_view text)  
2 {
```

```

3     co_await delay();
4     use(text); // text 指向的字符串可能已经销毁
5 }

```

如果协程会跨越调用者作用域继续运行，就不要保存指向调用者临时数据的视图。需要长期使用的数据应该复制或转移所有权。

31.5 取消和异常

协程流程看起来像同步代码，但错误边界仍然要设计。异步操作可能失败，`co_await` 处可能抛异常或返回错误。取消也不是强制销毁一切，而是协作协议：正在等待的操作要能收到取消请求，并让协程走到清理路径。

伪代码：

Listing 31.4 协程中的错误边界示意

```

1 Task<void> serve_one(Socket socket)
2 {
3     try {
4         Request request = co_await read_request(socket);
5         Response response = co_await build_response(request);
6         co_await write_response(socket, response);
7     } catch (const std::exception& error) {
8         log_error(error.what());
9     }
10 }

```

捕获异常必须有目的。这里是在请求边界记录错误，防止一个请求失败影响整个服务循环。

31.6 什么时候不需要协程

协程不是所有异步问题的默认答案。下面场景普通代码更合适：

- 简单后台任务，用 `std::jthread` 足够。
- CPU 密集并行，用线程池和任务分解更直接。
- 项目没有成熟协程库或调度器。
- 团队还无法调试协程栈和生命周期问题。

协程适合大量异步等待、流程嵌套严重、已有可靠异步运行时的场景。

31.7 从回调到状态机

即使不用协程，也可以把异步流程看成状态机：

Listing 31.5 请求处理状态

```

1 ReadRequest -> QueryUser -> WriteResponse -> Done
2                                     -> Failed

```

协程的一个价值，是让编译器帮你保存状态机的局部状态。理解这一点，就不会把协程误解成神秘语法。它是在帮你表达“可暂停的函数”。

习题与作业

把“读文件、解析配置、启动服务”写成三步伪异步流程。先用回调写，再用协程伪代码写。比较错误处理放在哪里更清楚。重点不是编译运行，而是理解控制流形状。

31.8 协程背后的手写状态机

协程最容易被误解成魔法。先把这段伪代码拆开：

Listing 31.6 协程顺序写法

```
1 Task<Response> handle(Socket socket)
2 {
3     Request request = co_await read_request(socket);
4     User user = co_await query_database(request.user_id);
5     co_return make_response(user);
6 }
```

如果不用协程，可以手写一个状态机：

Listing 31.7 手写状态机形态

```
1 state = ReadRequest;
2 保存 socket;
3
4 当 read_request 完成:
5     保存 request;
6     state = QueryDatabase;
7     发起 query_database;
8
9 当 query_database 完成:
10    保存 user;
11    state = Done;
12    生成 response;
```

协程的作用，是让编译器帮你生成和维护这套状态机。局部变量 `request`、`user`、`socket` 如果跨越暂停点还要使用，就会被保存到协程帧里。所谓协程帧，可以理解成“堆上或运行时管理的一块状态机存储”。

31.9 co await 近似展开

`co_await read_request(socket)` 大致做三件事：

1. 问 `awaiter`：结果是否已经准备好。
2. 如果没准备好，保存当前协程状态并把控制权还给调度器。
3. 等外部事件完成后，调度器恢复协程，从暂停点继续。

这和阻塞调用不同。阻塞调用会让当前线程卡住；协程暂停通常让线程回到事件循环或调度器，去处理别的任务。具体行为由库实现决定，语言只提供暂停和恢复的协议。

31.10 生命周期问题为什么更隐蔽

普通函数返回后，局部变量销毁，调用栈消失。协程暂停后，局部状态可能还活在协程帧里。于是下面的函数看起来只是接收一个 `string_view`，但暂停后风险变大：

Listing 31.8 跨暂停点保存视图

```
1 Task<void> log_later(std::string_view message)
2 {
3     co_await delay();
4     write_log(message);
5 }
```


如果调用者传入临时字符串，暂停期间临时对象早就销毁。更稳的接口是拥有数据：

Listing 31.9 协程跨暂停点拥有字符串

```
1 Task<void> log_later(std::string message)
2 {
3     co_await delay();
4     write_log(message);
5 }
```

这不是说协程里不能用视图，而是要看视图是否跨越暂停点。跨暂停点的数据，生命周期要求要按异步任务处理。

31.11 协程库边界

C++20 只给语言机制，不给完整任务类型。教材里的 `Task<T>` 是伪类型，真实项目需要库来定义：

- 协程如何启动。
- 暂停后由谁保存句柄。
- I/O 完成后由谁恢复协程。
- 异常如何存储和传播。
- 取消如何通知等待中的操作。

所以学习协程时要分清两层：语言层的 `co_await`、`co_return`、协程帧；库层的 `Task`、调度器、事件循环和 I/O awaiter。没有库层，语言层不能独立完成服务器。

31.12 把回调错误处理改成协程错误边界

回调版本里，错误处理容易散落：

Listing 31.10 回调错误散落

```
1 read_request(socket,
2     [socket](Request request) {
3         query_database(request.user_id,
4             [socket](User user) {
5                 write_response(socket, make_response(user));
6             },
7             [](Error error) {
8                 log_error(error);
9             });
10    },
11    [](Error error) {
12        log_error(error);
13    });
```

协程版本可以把一个请求的错误边界集中起来：

Listing 31.11 协程请求边界

```
1 Task<void> serve_one(Socket socket)
2 {
3     try {
4         Request request = co_await read_request(socket);
5         User user = co_await query_database(request.user_id);
```

```
6      co_await write_response(socket, make_response(user));
7  } catch (const RequestError& error) {
8      log_request_error(error);
9  }
10 }
```

这就是协程的实际价值：不是自动更快，而是把异步流程恢复成接近顺序代码的形状，让错误、取消和资源释放更容易放在同一个边界里。

第零部分

综合项目：把语言能力落到工程

第 32 章

项目一：命令行词频统计器

32.1 项目目标

这一章把前面学过的内容放进一个小工具：读取文本文件，统计单词出现次数，输出出现次数最高的前 *k* 个单词。这个项目覆盖文件 RAII、字符串处理、哈希表、排序、错误处理和命令行参数。目标命令如下：

Listing 32.1 词频工具目标用法

```
1 wordfreq --input article.txt --top 20
```

输出示例：

Listing 32.2 词频输出示例

```
1 the 128
2 system 42
3 memory 31
```

32.2 第一版：把文件读进字符串

最简单的做法是逐词读取：

Listing 32.3 逐词读取文件

```
1 #include <fstream>
2 #include <string>
3 #include <vector>
4
5 std::vector<std::string> read_words(const std::string& path)
6 {
7     std::ifstream input{path};
8     std::vector<std::string> words;
9     std::string word;
10    while (input >> word) {
11        words.push_back(word);
12    }
13    return words;
14 }
```

这段代码利用 `std::ifstream` 的 RAII 自动关闭文件。但它还不够：文件打不开时返回空数组，调用者无法区分“空文件”和“文件不存在”；标点也没有处理，“`system`”和“`system,`”会被当成两个词。

32.3 定义错误和清洗规则

先定义读取结果：

Listing 32.4 读取结果类型

```

1 struct WordReadError {
2     std::string message;
3 };
4
5 struct WordReadResult {
6     std::vector<std::string> words;
7     std::optional<WordReadError> error;
8 };

```

单词清洗规则先保持简单：保留字母和数字，统一转小写，其他字符当作分隔。

Listing 32.5 清洗一行文本

```

1 #include <cctype>
2
3 std::vector<std::string> split_words(std::string_view line)
4 {
5     std::vector<std::string> words;
6     std::string current;
7     for (unsigned char ch : line) {
8         if (std::isalnum(ch)) {
9             current.push_back(static_cast<char>(std::tolower(ch)));
10            continue;
11        }
12        if (!current.empty()) {
13            words.push_back(current);
14            current.clear();
15        }
16    }
17    if (!current.empty()) {
18        words.push_back(current);
19    }
20    return words;
21 }

```

这里用 `unsigned char` 是细节。很多 `cctype` 函数要求参数能表示为 `unsigned char` 或 EOF，直接传负的 `char` 可能出问题。

32.4 统计和排序

统计用 `unordered_map`，排序用 `vector`。

Listing 32.6 统计词频

```

1 #include <unordered_map>
2
3 using WordCounts = std::unordered_map<std::string, int>;
4
5 WordCounts count_words(std::span<const std::string> words)
6 {
7     WordCounts counts;
8     for (const std::string& word : words) {
9         ++counts[word];
10    }

```

```

11     return counts;
12 }

```

为了排序，把哈希表转成数组：

Listing 32.7 取 top k 高频词

```

1  #include <algorithm>
2
3  struct WordCount {
4      std::string word;
5      int count = 0;
6  };
7
8  std::vector<WordCount> top_words(const WordCounts& counts, std::size_t limit)
9  {
10     std::vector<WordCount> items;
11     items.reserve(counts.size());
12     for (const auto& [word, count] : counts) {
13         items.push_back(WordCount{.word = word, .count = count});
14     }
15
16     std::ranges::sort(items, [])(const WordCount& lhs, const WordCount& rhs) {
17         if (lhs.count != rhs.count) {
18             return lhs.count > rhs.count;
19         }
20         return lhs.word < rhs.word;
21     });
22
23     if (items.size() > limit) {
24         items.resize(limit);
25     }
26     return items;
27 }

```

排序规则必须完整：次数高的排前面；次数相同按字典序。这让输出稳定，测试也稳定。

32.5 复杂度和替代方案

设单词总数为 n ，不同单词数为 m 。统计平均复杂度是 $O(n)$ ，排序是 $O(m \log m)$ 。如果 k 很小而 m 很大，可以用小顶堆把排序降到 $O(m \log k)$ 。但第一版项目更重视正确性和可读性，全排序完全可以接受。

32.6 测试清单

- 空文件：输出为空。
- 大小写：C++ 和 c++ 合并。
- 标点：memory, memory. 计为同一个词。
- 次数相同：按字典序输出。
- 文件不存在：返回明确错误。

核心思想

一个小工具也要有结构：输入和清洗、统计、排序、输出分开。这样每一步都能单独测试，后续替换算法也不会牵动整段程序。

32.7 把需求拆成可测试流水线

词频工具看起来是一个命令，其实可以拆成五步：

1. 解析命令行，得到输入路径和 `top` 数量。
2. 读取文件，每次拿到一行文本。
3. 把一行文本清洗成单词。
4. 统计所有单词出现次数。
5. 排序、截断并输出。

拆分的标准是：每一步都有清楚输入输出，可以单独测试。比如 `split_words("Cpp, CPP!")` 应该得到两个 `cpp`；这个测试不需要真实文件。等清洗规则可靠后，再测试文件读取。

32.8 读取文件时保留错误

前面的 `read_words` 会把文件不存在和空文件都变成空数组。改进版应该显式检查文件打开：

Listing 32.8 逐行读取并返回错误

```
1 WordReadResult read_words_from_file(const std::string& path)
2 {
3     std::ifstream input{path};
4     if (!input) {
5         return {error = WordReadError{"cannot open input file"}};
6     }
7
8     WordReadResult result;
9     std::string line;
10    while (std::getline(input, line)) {
11        std::vector<std::string> words = split_words(line);
12        result.words.insert(result.words.end(),
13                            std::make_move_iterator(words.begin()),
14                            std::make_move_iterator(words.end()));
15    }
16    if (input.bad()) {
17        return {error = WordReadError{"failed while reading input file"}};
18    }
19    return result;
20 }
```

这里用 `std::getline` 是为了按行清洗标点；用 `make_move_iterator` 是因为临时 `words` 里的字符串已经不再需要，可以移动到结果里。即便如此，第一版仍然把所有单词存进内存。若文件很大，下一步可以改成边读边统计。

32.9 流式版本减少内存

更好的结构是读取一行就更新计数，不必保存所有单词：

Listing 32.9 边读边统计

```
1 struct CountResult {
2     WordCounts counts;
```

```

3     std::optional<WordReadError> error;
4 };
5
6 CountResult count_words_in_file(const std::string& path)
7 {
8     std::ifstream input{path};
9     if (!input) {
10         return {.error = WordReadError{"cannot open input file"}};
11     }
12
13     CountResult result;
14     std::string line;
15     while (std::getline(input, line)) {
16         for (std::string& word : split_words(line)) {
17             ++result.counts[word];
18         }
19     }
20     if (input.bad()) {
21         return {.error = WordReadError{"failed while reading input file"}};
22     }
23     return result;
24 }

```

这个版本的峰值内存主要由不同单词数决定，而不是所有单词总数。我们不是为了炫耀优化，而是从数据规模推出结构：如果文本很大，保存全部中间单词没有必要。

32.10 主程序如何连接模块

入口函数把前面步骤串起来：

Listing 32.10 词频工具主流程

```

1 int run_wordfreq(const Options& options)
2 {
3     const CountResult counted = count_words_in_file(options.input_path);
4     if (counted.error.has_value()) {
5         std::cerr << counted.error->message << '\n';
6         return 2;
7     }
8
9     const std::vector<WordCount> top = top_words(counted.counts, options.top);
10    for (const WordCount& item : top) {
11        std::cout << item.word << " " << item.count << '\n';
12    }
13    return 0;
14 }

```

这个函数仍然很短，因为细节已经在可测试函数里。入口处理错误消息和输出，核心函数处理数据规则。以后如果要输出 JSON，只需要替换输出层；如果要支持停用词，只需要在清洗或统计之间加一步过滤。

32.11 项目验收标准

一个可交付的小工具不只是“我本机能跑”。至少要满足：

- 命令行缺少 `-input` 或 `-top` 时给出错误。
- 文件打不开时返回非零退出码。
- 标点、大小写、空行处理符合文档规则。
- 次数相同时输出顺序稳定。
- 核心函数有测试，测试不依赖真实工作目录。

习题与作业

给词频工具增加 `-min-length N` 参数，只统计长度至少为 `N` 的单词。先写清楚 `N = 0`、缺值、非数字、负数分别怎么处理，再改参数解析和统计流程。

第 33 章

项目二：实现一个 LRU 缓存

33.1 项目目标

LRU 缓存是面试题，也是工程中常见结构。需求是：缓存最多保存 `capacity` 个键值对；每次访问一个键，这个键变成最新；插入新键时如果容量满，删除最久未使用的键。
一个可用接口如下：

Listing 33.1 LRU 缓存接口

```
1 template <typename Key, typename Value>
2 class LruCache {
3 public:
4     explicit LruCache(std::size_t capacity);
5     [[nodiscard]] std::optional<Value> get(const Key& key);
6     void put(Key key, Value value);
7     [[nodiscard]] std::size_t size() const noexcept;
8 };
```

33.2 为什么不能只用 `unordered_map`

`unordered_map` 能快速按键查找，但不知道谁最久没用。只用 `vector` 能维护顺序，但查找是线性的。LRU 需要两个结构组合：

- `std::list` 维护新旧顺序，表头表示最新。
- `std::unordered_map` 从 `key` 定位到链表节点。

33.3 节点和成员

Listing 33.2 LRU 的内部结构

```
1 #include <list>
2 #include <optional>
3 #include <unordered_map>
4
5 template <typename Key, typename Value>
6 class LruCache {
7 private:
8     struct Entry {
9         Key key;
10        Value value;
11    };
12
13    using EntryList = std::list<Entry>;
14    using EntryIterator = typename EntryList::iterator;
15
16 public:
17     explicit LruCache(std::size_t capacity)
18         : capacity_{capacity}
```

```

19     {
20     }
21
22 private:
23     std::size_t capacity_ = 0;
24     EntryList entries_;
25     std::unordered_map<Key, EntryIterator> index_;
26 };

```

链表迭代器在移动节点时保持有效，这是选择 `std::list` 的关键原因。

33.4 访问时移动到表头

Listing 33.3 get 操作

```

1 template <typename Key, typename Value>
2 std::optional<Value> LruCache<Key, Value>::get(const Key& key)
3 {
4     const auto found = this->index_.find(key);
5     if (found == this->index_.end()) {
6         return std::nullopt;
7     }
8
9     this->entries_.splice(this->entries_.begin(), this->entries_, found->second);
10    return found->second->value;
11 }

```

`splice` 不复制节点，只把已有节点移动到新位置。移动后迭代器仍指向同一个节点。

33.5 插入和淘汰

Listing 33.4 put 操作

```

1 template <typename Key, typename Value>
2 void LruCache<Key, Value>::put(Key key, Value value)
3 {
4     if (this->capacity_ == 0) {
5         return;
6     }
7
8     const auto found = this->index_.find(key);
9     if (found != this->index_.end()) {
10        found->second->value = std::move(value);
11        this->entries_.splice(this->entries_.begin(), this->entries_, found->second);
12        return;
13    }
14
15    this->entries_.push_front(Entry{.key = std::move(key), .value = std::move(value)});
16    this->index_[this->entries_.front().key] = this->entries_.begin();
17
18    if (this->entries_.size() > this->capacity_) {
19        const Key evicted_key = this->entries_.back().key;

```

```

20     this->index_.erase(evicted_key);
21     this->entries_.pop_back();
22 }
23 }

```

这个版本为清晰牺牲了一点细节：淘汰时复制了 **Key**。如果 key 很大，可以先移动或用节点引用优化，但要小心 **pop_back** 后引用失效。教材第一版先让所有权和顺序正确。

33.6 不变量

LRU 的核心不变量：

- `entries_.size() == index_.size()`。
- `index_` 中每个迭代器都指向 `entries_` 中的节点。
- 链表表头是最新访问项，表尾是最久未使用项。
- 容量为 0 时，缓存永远为空。

测试必须围绕这些不变量设计。

33.7 测试用例

Listing 33.5 LRU 行为测试思路

```

1 LruCache<int, std::string> cache{2};
2 cache.put(1, "one");
3 cache.put(2, "two");
4 assert(cache.get(1) == "one"); // 1 变成最新
5 cache.put(3, "three");         // 淘汰 2
6 assert(!cache.get(2).has_value());
7 assert(cache.get(1) == "one");
8 assert(cache.get(3) == "three");

```

常见误区

LRU 最容易错在“查找表和链表不同步”。每次插入、访问、淘汰后，都要能解释两者如何保持一致。

33.8 从样例手推状态变化

先不用代码，手推一个容量为 2 的缓存。链表从左到右表示“最新到最旧”。

Listing 33.6 LRU 状态推演

```

1 put(1, one)      [1]
2 put(2, two)      [2, 1]
3 get(1) -> one    [1, 2]
4 put(3, three)    [3, 1] 淘汰 2
5 get(2) -> miss   [3, 1]
6 put(1, uno)      [1, 3] 更新 1 并移动到最新

```

这段推演给出实现规则：新增节点放表头；命中节点移动到表头；容量超过时删除表尾；更新已有键时不能增加大小。任何实现只要违背其中一条，都会在样例里暴露。

33.9 put 的细节拆解

put 有三条路径。第一，容量为零，任何插入都无效。第二，键已存在，只更新值并移动节点。第三，键不存在，插入新节点，必要时淘汰旧节点。把这三条路径分清，代码就不会乱。

为了减少淘汰时复制 **key**，可以先读取表尾 **key**，再擦除映射，最后弹出节点。注意顺序不能反：

Listing 33.7 淘汰顺序必须保证 **key** 仍可读

```
1 const Key& evicted_key = this->entries_.back().key;
2 this->index_.erase(evicted_key);
3 this->entries_.pop_back();
```

这里在 **pop_back** 之前使用引用是安全的；**pop_back** 之后引用立刻失效，所以不能再访问。教材前面的复制版本更保守，适合先学正确性；这个版本展示如何在清楚生命周期的前提下减少复制。

33.10 完整类实现

把接口、成员和操作合在一起：

Listing 33.8 可用的 LRU 缓存实现

```
1 template <typename Key, typename Value>
2 class LruCache {
3 private:
4     struct Entry {
5         Key key;
6         Value value;
7     };
8
9     using EntryList = std::list<Entry>;
10    using EntryIterator = typename EntryList::iterator;
11
12 public:
13     explicit LruCache(std::size_t capacity)
14         : capacity_{capacity}
15     {
16     }
17
18     [[nodiscard]] std::optional<Value> get(const Key& key)
19     {
20         const auto found = this->index_.find(key);
21         if (found == this->index_.end()) {
22             return std::nullopt;
23         }
24         this->entries_.splice(this->entries_.begin(), this->entries_, found->second);
25         return found->second->value;
26     }
27
28     void put(Key key, Value value)
29     {
30         if (this->capacity_ == 0) {
31             return;
32         }
33
34         const auto found = this->index_.find(key);
```

```

35     if (found != this->index_.end()) {
36         found->second->value = std::move(value);
37         this->entries_.splice(this->entries_.begin(), this->entries_, found->
↵ second);
38         return;
39     }
40
41     this->entries_.push_front(Entry{std::move(key), std::move(value)});
42     this->index_[this->entries_.front().key] = this->entries_.begin();
43     if (this->entries_.size() > this->capacity_) {
44         const Key& evicted_key = this->entries_.back().key;
45         this->index_.erase(evicted_key);
46         this->entries_.pop_back();
47     }
48 }
49
50 [[nodiscard]] std::size_t size() const noexcept
51 {
52     return this->entries_.size();
53 }
54
55 private:
56     std::size_t capacity_ = 0;
57     EntryList entries_;
58     std::unordered_map<Key, EntryIterator> index_;
59 };

```

33.11 为什么返回 `optional<Value>` 而不是引用

`get` 返回 `std::optional<Value>` 会复制值。对于大对象，这可能有成本。为什么教材第一版仍然这么写？因为它简单、安全，调用者拿到的是独立值，不会因为后续缓存淘汰变成悬空引用。工程里可以设计两个接口：

Listing 33.9 按需提供引用接口

```

1 [[nodiscard]] Value* find(const Key& key);
2 [[nodiscard]] const Value* find(const Key& key) const;

```

指针返回 `nullptr` 表示未命中，命中时不复制。但文档必须说明：返回指针只在缓存下一次可能修改前有效。接口越高效，生命周期要求通常越严格。

33.12 测试不变量

除了行为测试，还可以在测试版本里暴露一个检查函数，验证映射和链表同步：

Listing 33.10 测试辅助不变量检查

```

1 [[nodiscard]] bool invariant_holds() const
2 {
3     if (this->entries_.size() != this->index_.size()) {
4         return false;
5     }
6     for (auto it = this->entries_.begin(); it != this->entries_.end(); ++it) {
7         const auto found = this->index_.find(it->key);

```

```
8     if (found == this->index_.end() || found->second != it) {
9         return false;
10    }
11    }
12    return true;
13 }
```

生产代码不一定要公开这个函数，但测试思路必须围绕它：每次 `put`、`get`、淘汰和更新后，链表与哈希表都应该描述同一组条目。

33.13 复杂度和替代设计

`get` 和 `put` 平均是 $O(1)$ ，前提是哈希表表现正常，链表移动是常数时间。替代方案包括：

- 只用 `vector`：顺序清楚，但查找和移动都是线性。
- 只用 `unordered_map`：查找快，但不知道谁最旧。
- `unordered_map` 加自定义双向链表：可控性更高，但实现和测试成本更高。

标准库组合已经能满足大多数场景。只有在性能测量证明 `std::list` 节点分配成为瓶颈时，才值得考虑自定义存储。

习题与作业

给 LRU 增加 `contains` 和 `erase`。要求 `contains` 不改变新旧顺序，`erase` 同时删除链表节点和哈希表项。写测试覆盖：删除不存在的 key、删除最新 key、删除最旧 key、删除后再次插入。

第 34 章

项目三：配置加载器

34.1 项目目标

这个项目实现一个小型配置加载器。输入是 `key=value` 文件，支持注释、空行、重复 key 覆盖、类型转换和必填项校验。输出是一个强类型配置对象。

示例配置：

Listing 34.1 配置文件示例

```
1 # server config
2 host = 127.0.0.1
3 port = 8080
4 workers = 4
5 log_level = info
```

目标不是写一个通用配置库，而是把字符串处理、错误接口、类型建模、文件 I/O 和测试组织合在一起。配置加载器是非常适合练基本功的项目，因为它看起来简单，但边界很多。

34.2 先定义业务配置

不要让整个程序到处传 `unordered_map<string, string>`。读取阶段可以用字符串映射，业务阶段应该使用强类型：

Listing 34.2 强类型服务器配置

```
1 enum class LogLevel {
2     debug,
3     info,
4     warning,
5     error,
6 };
7
8 struct ServerConfig {
9     std::string host;
10    int port = 0;
11    int workers = 0;
12    LogLevel log_level = LogLevel::info;
13 };
```

这一步把业务要求显式化：`port` 是整数，`workers` 是整数，`log_level` 只能是枚举值。后续代码不需要反复解析字符串。

34.3 错误类型要能定位问题

配置错误需要带上下文：

Listing 34.3 配置错误

```
1 enum class ConfigErrorKind {
2     file_not_found,
```



```

3     invalid_line,
4     missing_key,
5     invalid_value,
6 };
7
8 struct ConfigError {
9     ConfigErrorKind kind = ConfigErrorKind::invalid_line;
10    int line = 0;
11    std::string key;
12    std::string message;
13 };
14
15 struct ConfigResult {
16     std::optional<ServerConfig> config;
17     std::optional<ConfigError> error;
18 };

```

为什么要有 `line` 和 `key`？因为用户修配置时需要知道哪里错。错误结构不是为了程序员自娱自乐，而是为了让调用者能做出具体动作。

34.4 解析阶段：从行到映射

先把文件解析成键值映射：

Listing 34.4 解析配置条目

```

1 using RawConfig = std::unordered_map<std::string, std::string>;
2
3 struct RawConfigResult {
4     RawConfig values;
5     std::optional<ConfigError> error;
6 };
7
8 RawConfigResult parse_raw_config(std::istream& input)
9 {
10     RawConfigResult result;
11     std::string line;
12     int line_number = 0;
13     while (std::getline(input, line)) {
14         ++line_number;
15         try {
16             std::optional<ConfigEntry> entry = parse_config_line(line);
17             if (entry.has_value()) {
18                 result.values[entry->key] = entry->value;
19             }
20         } catch (const std::exception& error) {
21             return {.error = ConfigError{
22                 .kind = ConfigErrorKind::invalid_line,
23                 .line = line_number,
24                 .message = error.what(),
25             }};
26         }
27     }
28     return result;

```

```
29 }
```

重复 key 的规则在这里定义：后者覆盖前者。真实项目也可以记录警告，但规则必须写清楚，不能让行为由容器偶然决定。

34.5 类型转换阶段

字符串转整数要检查完整消费和范围：

Listing 34.5 解析整数配置

```
1 std::optional<int> parse_int_value(std::string_view text)
2 {
3     int value = 0;
4     const char* begin = text.data();
5     const char* end = text.data() + text.size();
6     const auto [ptr, ec] = std::from_chars(begin, end, value);
7     if (ec != std::errc{} || ptr != end) {
8         return std::nullopt;
9     }
10    return value;
11 }
```

日志级别是有限集合：

Listing 34.6 解析日志级别

```
1 std::optional<LogLevel> parse_log_level(std::string_view text)
2 {
3     if (text == "debug") {
4         return LogLevel::debug;
5     }
6     if (text == "info") {
7         return LogLevel::info;
8     }
9     if (text == "warning") {
10        return LogLevel::warning;
11    }
12    if (text == "error") {
13        return LogLevel::error;
14    }
15    return std::nullopt;
16 }
```

用 `enum class` 的好处是后续 `switch` 不会把日志级别和普通整数混用。

34.6 从映射构造强类型配置

构造 `ServerConfig` 时处理必填项、默认值和范围：

Listing 34.7 构造强类型配置

```
1 std::optional<std::string_view> find_raw(const RawConfig& raw, std::string_view key)
2 {
3     const auto found = raw.find(std::string{key});
4     if (found == raw.end()) {
```

```

5     return std::nullopt;
6 }
7     return std::string_view{found->second};
8 }
9
10 ConfigResult build_server_config(const RawConfig& raw)
11 {
12     ServerConfig config;
13
14     const std::optional<std::string_view> host = find_raw(raw, "host");
15     if (!host.has_value()) {
16         return {error = ConfigError{kind = ConfigErrorKind::missing_key,
17                                     .key = "host",
18                                     .message = "missing host"}};
19     }
20     config.host = *host;
21
22     const std::optional<std::string_view> port_text = find_raw(raw, "port");
23     const std::optional<int> port = port_text.has_value()
24         ? parse_int_value(*port_text)
25         : std::nullopt;
26     if (!port.has_value() || *port < 1 || *port > 65535) {
27         return {error = ConfigError{kind = ConfigErrorKind::invalid_value,
28                                     .key = "port",
29                                     .message = "port must be 1..65535"}};
30     }
31     config.port = *port;
32
33     config.workers = 1;
34     if (const std::optional<std::string_view> workers = find_raw(raw, "workers")) {
35         const std::optional<int> parsed = parse_int_value(*workers);
36         if (!parsed.has_value() || *parsed <= 0) {
37             return {error = ConfigError{kind = ConfigErrorKind::invalid_value,
38                                         .key = "workers",
39                                         .message = "workers must be positive"}};
40         }
41         config.workers = *parsed;
42     }
43
44     if (const std::optional<std::string_view> level = find_raw(raw, "log_level")) {
45         const std::optional<LogLevel> parsed = parse_log_level(*level);
46         if (!parsed.has_value()) {
47             return {error = ConfigError{kind = ConfigErrorKind::invalid_value,
48                                         .key = "log_level",
49                                         .message = "invalid log level"}};
50         }
51         config.log_level = *parsed;
52     }
53
54     return {.config = std::move(config)};
55 }

```

这段代码刻意分阶段：先把文本读成原始映射，再把映射转换成强类型对象。这样解析行格式

和业务校验可以分别测试。

34.7 文件入口

最终文件入口很薄：

Listing 34.8 加载配置文件

```
1 ConfigResult load_config_file(const std::filesystem::path& path)
2 {
3     std::ifstream input{path};
4     if (!input) {
5         return {error = ConfigError{.kind = ConfigErrorKind::file_not_found,
6                                     .message = "cannot open config file"}};
7     }
8
9     RawConfigResult raw = parse_raw_config(input);
10    if (raw.error.has_value()) {
11        return {error = raw.error};
12    }
13    return build_server_config(raw.values);
14 }
```

这里没有把所有逻辑塞进一个函数。文件打开、行解析、类型转换各有边界，错误也能保留上下文。

34.8 测试计划

这个项目至少要覆盖：

- 空行和注释行被忽略。
- 缺少等号返回行号错误。
- 重复 key 使用后者。
- 缺少必填 `host` 或 `port` 报错。
- `port=abc`、`port=0`、`port=65536` 报错。
- `workers` 缺失时使用默认值 `1`。
- 非法 `log_level` 报错。

测试尽量用 `std::istringstream`，不要每个单元测试都创建真实文件。只有 `load_config_file` 这一层需要文件系统测试。

习题与作业

给配置加载器增加 `include=other.conf`。要求避免递归实现，使用显式队列或栈处理待加载文件，并检测重复包含。写清楚相对路径以哪个目录为基准。

34.9 从一行配置推导解析边界

配置加载器看起来只是字符串切分，但边界很多。以这一行为例：

Listing 34.9 带空白的配置行

```
1 port = 8080
```

解析步骤应该明确：

1. 去掉整行首尾空白。

2. 如果为空或以 `##` 开头，返回无条目。
3. 查找第一个等号。
4. 等号左边 trim 成 key。
5. 等号右边 trim 成 value。
6. key 不能为空。

这六步就是 `parse_config_line` 的合同。不要在同一个函数里顺手解析端口范围、打开 include 文件、构造业务对象。单行解析只负责行格式，业务校验交给下一层。

34.10 重复 key 规则要可见

本项目选择“后者覆盖前者”。这条规则不能只靠 `unordered_map` 的赋值行为隐藏起来。可以显式记录警告：

Listing 34.10 合并配置条目并记录覆盖

```
1 struct MergeWarning {
2     std::string key;
3     int previous_line = 0;
4     int new_line = 0;
5 };
6
7 struct RawConfigValue {
8     std::string value;
9     int line = 0;
10 };
11
12 using RawConfig = std::unordered_map<std::string, RawConfigValue>;
```

合并时：

Listing 34.11 重复 key 后者覆盖

```
1 void merge_entry(RawConfig& config,
2                 std::vector<MergeWarning>& warnings,
3                 ConfigEntry entry,
4                 int line)
5 {
6     if (const auto found = config.find(entry.key); found != config.end()) {
7         warnings.push_back(MergeWarning{.key = entry.key,
8                                           .previous_line = found->second.line,
9                                           .new_line = line});
10    }
11    config[entry.key] = RawConfigValue{.value = std::move(entry.value),
12                                        .line = line};
13 }
```

现在覆盖行为是项目规则，不是容器偶然效果。用户也能看到“第几行覆盖了第几行”。

34.11 错误对象要服务调用者

配置错误要能被 CLI 打印，也能被测试断言。错误对象至少包含 kind、line、key、message。测试不要只断言“失败了”，而要断言失败类型：

Listing 34.12 配置错误测试关注类型和上下文

```

1 const ConfigResult result = load_config_text("port=abc\n");
2 assert(result.error.has_value());
3 assert(result.error->kind == ConfigErrorKind::invalid_value);
4 assert(result.error->key == "port");

```

这样未来修改错误文本时，测试不会因为文案微调全部失败；但错误类型和 key 改错时，测试能挡住。

34.12 include 支持的显式栈

扩展 `include=other.conf` 时，不要用递归函数无限调用自己。用显式栈或队列更容易控制深度和重复包含：

Listing 34.13 用显式栈处理 include

```

1 std::vector<std::filesystem::path> pending;
2 std::unordered_set<std::string> visited;
3 pending.push_back(root_path);
4
5 while (!pending.empty()) {
6     const std::filesystem::path current = pending.back();
7     pending.pop_back();
8
9     const std::string canonical = canonical_key(current);
10    if (!visited.insert(canonical).second) {
11        continue;
12    }
13
14    RawConfigResult parsed = parse_one_file(current);
15    if (parsed.error.has_value()) {
16        return {.error = parsed.error};
17    }
18    for (const std::filesystem::path& include : parsed.includes) {
19        pending.push_back(resolve_include(current.parent_path(), include));
20    }
21 }

```

这里的关键规则是：相对 include 以当前配置文件所在目录为基准；已访问文件不再重复加载；错误带着当前文件和行号。显式栈让这些规则都能被测试。

34.13 配置加载器验收矩阵

项目验收不要只跑一个正常配置。至少覆盖：

场景	期望
空文件	缺少必填项错误
注释和空行	被忽略
缺少等号	invalid line, 带行号
空 key	invalid line, 带行号
重复 key	后者覆盖, 产生警告
port 不是数字	invalid value, key 为 <code>port</code>
port 越界	invalid value
workers 缺失	使用默认值 1

非法 log level
include 循环

invalid value, key 为 `log_level`
不死循环，规则明确

有了这个矩阵，配置加载器才像一个可以交给别人用的小工具，而不是演示字符串切分的片段。

第 35 章

项目四：一个小型任务队列

35.1 项目目标

本项目实现一个小型任务队列：多个生产者提交任务，一个或多个工作线程取任务执行，关闭时不丢任务、不让线程永久阻塞。它不是完整线程池库，但足够训练并发代码最重要的几个问题：共享状态、条件变量、关闭协议、异常边界和生命周期。

目标接口：

Listing 35.1 任务队列接口

```
1 class TaskQueue {
2 public:
3     void push(std::function<void()> task);
4     std::optional<std::function<void()>> pop();
5     void close();
6 };
```

`pop` 返回 `std::nullopt` 表示队列已经关闭且没有剩余任务。这个返回值就是线程退出协议。

35.2 共享状态和不变量

队列内部状态：

Listing 35.2 任务队列状态

```
1 class TaskQueue {
2 public:
3     void push(std::function<void()> task);
4     std::optional<std::function<void()>> pop();
5     void close();
6
7 private:
8     std::mutex mutex_;
9     std::condition_variable ready_;
10    std::queue<std::function<void()>> tasks_;
11    bool closed_ = false;
12 };
```

不变量是：`tasks_` 和 `closed_` 只能在持有 `mutex_` 时访问。条件变量等待的条件是“队列非空，或已经关闭”。只要这个不变量破了，并发错误就会变得很难复现。

35.3 push：关闭后不能继续提交

关闭后继续提交任务，语义必须明确。本项目选择抛异常：

Listing 35.3 提交任务

```
1 void TaskQueue::push(std::function<void()> task)
2 {
3     {
```



```

4     std::lock_guard<std::mutex> lock{this->mutex_};
5     if (this->closed_) {
6         throw std::runtime_error{"cannot push to closed queue"};
7     }
8     this->tasks_.push(std::move(task));
9 }
10 this->ready_.notify_one();
11 }

```

通知放在释放锁之后，通常能减少被唤醒线程马上又阻塞在同一把锁上的机会。放在锁内也常常正确，但要理解取舍。

35.4 pop：谓词等待

Listing 35.4 取任务

```

1 std::optional<std::function<void()>> TaskQueue::pop()
2 {
3     std::unique_lock<std::mutex> lock{this->mutex_};
4     this->ready_.wait(lock, [this] {
5         return this->closed_ || !this->tasks_.empty();
6     });
7
8     if (this->tasks_.empty()) {
9         return std::nullopt;
10    }
11
12    std::function<void()> task = std::move(this->tasks_.front());
13    this->tasks_.pop();
14    return task;
15 }

```

醒来后队列可能为空，只有一种合理解释：队列关闭且任务已经取完。因此返回 `std::nullopt`。如果不用谓词，虚假唤醒会让线程在队列为空时继续执行错误逻辑。

35.5 close：唤醒所有等待线程

Listing 35.5 关闭队列

```

1 void TaskQueue::close()
2 {
3     {
4         std::lock_guard<std::mutex> lock{this->mutex_};
5         this->closed_ = true;
6     }
7     this->ready_.notify_all();
8 }

```

为什么是 `notify_all`？因为可能有多个工作线程都阻塞在 `pop`。关闭时所有线程都应该醒来检查条件，然后在任务耗尽后退出。

35.6 Worker：线程生命周期

工作线程循环取任务：

Listing 35.6 工作线程循环

```

1 void worker_loop(TaskQueue& queue)
2 {
3     while (true) {
4         std::optional<std::function<void()>> task = queue.pop();
5         if (!task.has_value()) {
6             return;
7         }
8         try {
9             (*task)();
10        } catch (const std::exception& error) {
11            std::cerr << "task failed: " << error.what() << '\n';
12        }
13    }
14 }

```

任务异常不能让工作线程悄悄结束，至少要在任务边界捕获并记录。真实系统可能把错误发送到监控或结果队列。

35.7 用 `jthread` 管理工作线程

Listing 35.7 小型任务执行器

```

1 class TaskExecutor {
2 public:
3     explicit TaskExecutor(std::size_t worker_count)
4     {
5         for (std::size_t i = 0; i < worker_count; ++i) {
6             this->workers_.emplace_back([this] {
7                 worker_loop(this->queue_);
8             });
9         }
10    }
11
12    ~TaskExecutor()
13    {
14        this->queue_.close();
15    }
16
17    void submit(std::function<void()> task)
18    {
19        this->queue_.push(std::move(task));
20    }
21
22 private:
23     TaskQueue queue_;
24     std::vector<std::jthread> workers_;
25 };

```

`std::jthread` 析构会 `join`。析构函数先关闭队列，让所有工作线程最终退出。成员析构顺序也重要：成员按声明逆序析构。这里 `workers_` 在 `queue_` 之后声明，所以析构时先销毁 `workers_`，等待线程结束，再销毁 `queue_`。工作线程不会访问已经销毁的队列。

35.8 析构顺序的隐藏坑

如果把成员顺序写反：

Listing 35.8 危险的成员顺序

```
1 std::vector<std::jthread> workers_;
2 TaskQueue queue_;
```

析构时会先销毁 `queue_`，再销毁 `workers_`。工作线程可能还在访问队列，造成悬空引用。并发类的成员顺序不是排版问题，而是生命周期协议的一部分。

35.9 测试并发代码

并发测试要尽量验证确定性结果：

Listing 35.9 任务执行器测试思路

```
1 TaskExecutor executor{4};
2 std::atomic<int> counter{0};
3 for (int i = 0; i < 1000; ++i) {
4     executor.submit([&counter] {
5         counter.fetch_add(1, std::memory_order_relaxed);
6     });
7 }
```

这个测试还需要等待任务完成，否则析构时才关闭队列，可能很难在中途断言。可以给执行器增加 `drain`，或者在测试里使用 `std::promise`、`std::latch` 等同步工具。测试并发代码的重点是避免靠睡眠猜时间。

35.10 扩展方向

完整线程池还需要：

- 返回任务结果，例如 `std::future<T>`。
- 限制队列长度，避免生产者无限堆积任务。
- 支持取消和停止令牌。
- 记录任务异常。
- 避免析构时在工作线程内部 `join` 自己。

每个扩展都牵涉接口语义。比如队列满了时，`submit` 是阻塞、返回失败还是丢弃任务？这些规则必须先写清楚，再写代码。

习题与作业

给 `TaskExecutor` 增加 `submit_counting`，提交任务后返回一个 `std::future<void>`，调用者可以等待任务完成并接收异常。要求任务异常传给 `future`，而不是只打印日志。

35.11 为什么这个项目先叫任务队列

很多人一上来想写“线程池”，结果很快被 `future`、取消、优先级、动态扩缩容、任务窃取压垮。本项目先叫任务队列，是为了把最核心的协议讲清：生产者如何提交，消费者如何等待，关闭时如何退出，异常如何越过任务边界。线程池只是建立在这些协议之上的更大结构。

35.12 背压：队列不能无限长

当前 `TaskQueue` 没有限制长度。生产者比消费者快时，任务会无限堆积，内存可能被耗尽。一个工程版本通常需要背压。先定义策略：

- 队列未滿时，`push` 成功。
- 队列已滿时，`push` 阻塞等待空间。
- 队列关闭时，`push` 失败或抛异常。

状态增加一个容量和一个条件变量：

Listing 35.10 有界任务队列状态

```
1 std::mutex mutex_;
2 std::condition_variable not_empty_;
3 std::condition_variable not_full_;
4 std::queue<std::function<void()>> tasks_;
5 std::size_t capacity_ = 0;
6 bool closed_ = false;
```

`push` 等待“不关闭且未滿”，`pop` 取走任务后通知 `not_full_`。这里有两个等待条件，所以用两个条件变量能让意图更清楚。

35.13 有界 `push` 的协议

可以让 `push` 返回布尔值而不是抛异常：

Listing 35.11 有界队列提交任务

```
1 bool push(std::function<void()> task)
2 {
3     std::unique_lock<std::mutex> lock{this->mutex_};
4     this->not_full_.wait(lock, [this] {
5         return this->closed_ || this->tasks_.size() < this->capacity_;
6     });
7
8     if (this->closed_) {
9         return false;
10    }
11
12    this->tasks_.push(std::move(task));
13    lock.unlock();
14    this->not_empty_.notify_one();
15    return true;
16 }
```

等待谓词里必须包含 `closed_`。否则队列滿时，如果另一个线程关闭队列，生产者可能永远睡在“等待空间”上。关闭时也必须同时 `notify_all` 两个条件变量，让等待任务和等待空间的线程都醒来。

35.14 `drain` 不是 `sleep`

测试里最常见坏味道是：

Listing 35.12 不要靠睡眠等待任务完成

```
1 std::this_thread::sleep_for(std::chrono::milliseconds{100});
```

```
2 assert(counter == 1000);
```

这在快机器上可能过，在慢机器或 CI 上可能失败。更好的测试使用同步原语。例如提交一批任务，每个任务倒数一个 latch：

Listing 35.13 用 latch 等待任务完成

```
1 std::latch done{1000};
2 for (int i = 0; i < 1000; ++i) {
3     executor.submit([&done] {
4         done.count_down();
5     });
6 }
7 done.wait();
```

并发测试要等待明确事件，而不是等待一段时间。时间只能作为超时保护，不应作为正确性依据。

35.15 future 版本的任务提交

如果调用者需要接收异常，可以用 `std::packaged_task`：

Listing 35.14 提交任务并返回 future

```
1 std::future<void> submit_future(std::function<void()> task)
2 {
3     auto packaged = std::make_shared<std::packaged_task<void()>>(
4         [task = std::move(task)] {
5             task();
6         });
7     std::future<void> result = packaged->get_future();
8     this->queue_.push([packaged] {
9         (*packaged)();
10    });
11    return result;
12 }
```

如果任务抛异常，`packaged_task` 会把异常存进 future，调用者在 `get()` 时重新收到。这比工作线程直接打印日志更适合库接口，因为错误回到了提交者手里。

35.16 项目验收：关闭路径最重要

任务队列的验收重点不是“能跑几个任务”，而是关闭路径：

- 空队列关闭后，所有 worker 退出。
- 有剩余任务时关闭，已提交任务是否执行完，规则要明确。
- 队列满且生产者阻塞时关闭，生产者能醒来。
- 任务抛异常时，worker 不应无声死亡。
- 析构时不会访问已经销毁的队列。

并发项目最贵的 bug 往往在退出路径。把退出协议测透，比多加一个功能更重要。

第 36 章

项目五：异步日志器

36.1 项目目标

日志器是一个综合项目：它接收多线程提交的日志消息，后台线程写入文件，关闭时刷新剩余消息。这个项目覆盖字符串格式化、时间、互斥量、条件变量、文件 RAII、异常边界和背压策略。目标接口：

Listing 36.1 日志器接口

```
1 class AsyncLogger {
2 public:
3     explicit AsyncLogger(std::filesystem::path path);
4     ~AsyncLogger();
5
6     void info(std::string message);
7     void error(std::string message);
8 };
```

接口按值接收 `std::string`，因为日志器要保存消息到后台队列。调用者传临时字符串时可以移动，传左值时复制一份，语义清楚。

36.2 日志消息类型

Listing 36.2 日志消息

```
1 enum class LogLevel {
2     info,
3     error,
4 };
5
6 struct LogMessage {
7     LogLevel level = LogLevel::info;
8     std::chrono::system_clock::time_point time;
9     std::string text;
10 };
```

不要把日志级别保存成裸字符串。枚举让内部逻辑更稳定，输出时再转换成文本。

36.3 从同步日志推到异步日志

先写最直接的同步版本：

Listing 36.3 同步日志器

```
1 class SyncLogger {
2 public:
3     explicit SyncLogger(std::filesystem::path path)
4         : output_{path}
5     {
```

```

6         if (!this->output_) {
7             throw std::runtime_error{"cannot open log file"};
8         }
9     }
10
11     void info(std::string_view message)
12     {
13         std::lock_guard<std::mutex> lock{this->mutex_};
14         this->output_ << "INFO " << message << '\n';
15     }
16
17 private:
18     std::ofstream output_;
19     std::mutex mutex_;
20 };

```

这个版本容易验证：调用 `info` 时，当前线程直接写文件。缺点也明显：文件 I/O 慢时，业务线程会卡在 `info` 里。异步日志器不是为了让代码更花，而是把慢 I/O 从调用线程移到后台线程。

从同步到异步，本质上多了三件事：

1. 调用线程只构造消息并放入队列。
2. 后台线程从队列取消息并写文件。
3. 关闭时要让后台线程写完剩余消息再退出。

所以异步日志器首先是一个队列协议项目，其次才是格式化项目。

36.4 队列和关闭协议

日志器内部和任务队列类似：

Listing 36.4 日志器内部状态

```

1 class AsyncLogger {
2 public:
3     explicit AsyncLogger(std::filesystem::path path);
4     ~AsyncLogger();
5
6     void info(std::string message);
7     void error(std::string message);
8
9 private:
10    void push(LogLevel level, std::string message);
11    void run();
12
13    std::ofstream output_;
14    std::mutex mutex_;
15    std::condition_variable ready_;
16    std::queue<LogMessage> messages_;
17    bool closed_ = false;
18    std::jthread worker_;
19 };

```

成员顺序要注意：`worker_` 最后声明，析构时最先析构并 `join`。但析构函数会先设置关闭状态并通知线程，让后台线程自然退出。

36.5 关闭协议的状态推导

关闭不是简单把 `closed_` 设成 `true`。要先定义规则：

- 关闭前已经进入队列的消息必须写完。
- 关闭后新的 `info` 和 `error` 不再进入队列。
- 后台线程在队列为空且已关闭时退出。
- 析构函数不能抛异常。

由这四条规则推出后台线程的退出条件：

Listing 36.5 退出条件由关闭规则推出

```
1 this->ready_.wait(lock, [this] {
2     return this->closed_ || !this->messages_.empty();
3 });
4 if (this->messages_.empty() && this->closed_) {
5     return;
6 }
```

如果只写 `if (this->closed_) return;`，会丢掉关闭前已经排队的消息。正确判断是“队列空并且关闭”才退出。这个边界是异步日志器最容易写错的地方。

36.6 构造和析构

Listing 36.6 构造和析构日志器

```
1 AsyncLogger::AsyncLogger(std::filesystem::path path)
2     : output_{path}, worker_{[this] { this->run(); }}
3 {
4     if (!this->output_) {
5         throw std::runtime_error{"cannot open log file"};
6     }
7 }
8
9 AsyncLogger::~AsyncLogger()
10 {
11     {
12         std::lock_guard<std::mutex> lock{this->mutex_};
13         this->closed_ = true;
14     }
15     this->ready_.notify_all();
16 }
```

这里有一个细节：构造函数先初始化 `output_`，再启动 `worker_`。成员按声明顺序初始化，而不是按初始化列表顺序。构造期间启动线程要非常小心，确保线程访问的成员已经构造完成。

36.7 构造函数里启动线程的风险

更稳的构造方式是先打开文件，确认成功后再启动线程。因为 `std::jthread` 一旦构造，后台函数就可能立刻运行：

Listing 36.7 显式 start 降低构造风险

```
1 class AsyncLogger {
2 public:
```



```

3     explicit AsyncLogger(std::filesystem::path path)
4         : output_{path}
5     {
6         if (!this->output_) {
7             throw std::runtime_error{"cannot open log file"};
8         }
9         this->worker_ = std::jthread{[this] { this->run(); }};
10    }
11
12 private:
13     std::ofstream output_;
14     std::mutex mutex_;
15     std::condition_variable ready_;
16     std::queue<LogMessage> messages_;
17     bool closed_ = false;
18     std::jthread worker_;
19 };

```

这种写法要求 `worker_` 可以默认构造，然后在构造函数体里赋值启动。好处是失败路径更清楚：文件打不开时对象构造失败，后台线程根本没有启动。教学项目可以先用这种形态，降低半构造对象被线程访问的风险。

36.8 提交日志

Listing 36.8 提交日志消息

```

1 void AsyncLogger::info(std::string message)
2 {
3     this->push(LogLevel::info, std::move(message));
4 }
5
6 void AsyncLogger::error(std::string message)
7 {
8     this->push(LogLevel::error, std::move(message));
9 }
10
11 void AsyncLogger::push(LogLevel level, std::string message)
12 {
13     {
14         std::lock_guard<std::mutex> lock{this->mutex_};
15         if (this->closed_) {
16             return;
17         }
18         this->messages_.push(LogMessage{
19             .level = level,
20             .time = std::chrono::system_clock::now(),
21             .text = std::move(message),
22         });
23     }
24     this->ready_.notify_one();
25 }

```

关闭后提交日志，本项目选择静默忽略。也可以选择抛异常或返回 `bool`，但规则必须写进接口

文档。析构期间再抛异常通常不是好主意。

36.9 有界队列和丢弃计数

无限队列把压力藏进内存。教学版可以增加容量，并定义简单策略：队列满时丢弃 **info**，尽量保留 **error**。为避免静默丢失，还要统计丢弃数量：

Listing 36.9 带容量和丢弃计数的状态

```
1 class AsyncLogger {
2 private:
3     static constexpr std::size_t LOGGER_MAX_QUEUE_SIZE = 8192;
4
5     std::queue<LogMessage> messages_;
6     std::uint64_t dropped_info_count_ = 0;
7 };
```

push 的策略可以写成明确分支：

Listing 36.10 队列满时丢弃 **info**

```
1 void AsyncLogger::push(LogLevel level, std::string message)
2 {
3     {
4         std::lock_guard<std::mutex> lock{this->mutex_};
5         if (this->closed_) {
6             return;
7         }
8         if (this->messages_.size() >= LOGGER_MAX_QUEUE_SIZE &&
9             level == LogLevel::info) {
10             ++this->dropped_info_count_;
11             return;
12         }
13         this->messages_.push(LogMessage{
14             .level = level,
15             .time = std::chrono::system_clock::now(),
16             .text = std::move(message),
17         });
18     }
19     this->ready_.notify_one();
20 }
```

这段代码还没有解决“队列满但来了 **error**”时怎么办。可以选择阻塞、同步写入、或者丢掉最旧的 **info** 给 **error** 腾位置。教材里故意把这个问题暴露出来：背压策略必须按业务定义，不能靠容器自动决定。

36.10 后台写入

Listing 36.11 后台写日志

```
1 void AsyncLogger::run()
2 {
3     while (true) {
4         std::optional<LogMessage> message;
5         {
```

```

6         std::unique_lock<std::mutex> lock{this->mutex_};
7         this->ready_.wait(lock, [this] {
8             return this->closed_ || !this->messages_.empty();
9         });
10        if (this->messages_.empty()) {
11            return;
12        }
13        message = std::move(this->messages_.front());
14        this->messages_.pop();
15    }
16    this->write_message(*message);
17 }
18 }

```

取出消息后立刻释放锁，再写文件。文件 I/O 可能慢，如果持锁写文件，提交日志的线程也会被阻塞。锁只保护队列，不保护所有操作。

36.11 flush 的完整实现思路

为了让测试和调用者知道“当前已经提交的日志写完了”，可以把队列项从单一 `LogMessage` 扩展成命令：

Listing 36.12 日志队列命令

```

1 struct FlushCommand {
2     std::shared_ptr<std::promise<void>> done;
3 };
4
5 using LoggerCommand = std::variant<LogMessage, FlushCommand>;

```

队列从 `std::queue<LogMessage>` 改成 `std::queue<LoggerCommand>`。提交 `flush`：

Listing 36.13 提交 `flush` 命令

```

1 void AsyncLogger::flush()
2 {
3     auto done = std::make_shared<std::promise<void>>();
4     std::future<void> future = done->get_future();
5     {
6         std::lock_guard<std::mutex> lock{this->mutex_};
7         if (this->closed_) {
8             return;
9         }
10        this->messages_.push(FlushCommand{.done = done});
11    }
12    this->ready_.notify_one();
13    future.get();
14 }

```

后台线程处理命令：

Listing 36.14 后台处理 `flush` 命令

```

1 void AsyncLogger::handle_command(LoggerCommand command)
2 {

```

```

3     std::visit([this](auto& item) {
4         using Item = std::decay_t<decltype(item)>;
5         if constexpr (std::is_same_v<Item, LogMessage>) {
6             this->write_message(item);
7         } else {
8             this->output_.flush();
9             item.done->set_value();
10        }
11    }, command);
12 }

```

这里的顺序很关键：**flush** 命令在队列中的位置保证它前面的日志消息已经被处理；**output_.flush()** 保证文件缓冲被刷新；**promise** 保证调用线程有明确等待点。测试就可以调用 **flush()**，再读取文件断言内容，而不是睡一段时间碰运气。

36.12 格式化输出

Listing 36.15 格式化日志级别

```

1 std::string_view level_name(LogLevel level)
2 {
3     switch (level) {
4     case LogLevel::info:
5         return "INFO";
6     case LogLevel::error:
7         return "ERROR";
8     }
9     return "UNKNOWN";
10 }

```

写消息：

Listing 36.16 写日志消息

```

1 void AsyncLogger::write_message(const LogMessage& message)
2 {
3     this->output_ << level_name(message.level)
4                 << " "
5                 << message.text
6                 << '\n';
7 }

```

时间格式化可以继续扩展。第一版先保证线程和关闭协议正确，不要把所有功能一次塞进去。

36.13 背压和丢日志策略

当前日志器队列无限增长。如果日志产生速度超过写盘速度，会占用越来越多内存。可以增加最大队列长度：

Listing 36.17 日志队列背压策略

```

1 constexpr std::size_t LOGGER_MAX_QUEUE_SIZE = 8192;

```

队列满时常见策略：

- 阻塞调用者：日志不能丢，但业务线程可能变慢。
- 丢弃新日志：业务不阻塞，但可能缺关键信息。
- 丢弃低级别日志：保留错误，丢掉 info。
- 同步写入：队列满时调用者自己写文件，复杂度更高。

日志系统没有完美策略，只有适合业务的取舍。教学版可以先选“队列满时丢弃 info，保留 error”，并统计丢弃数量。

36.14 最终验收标准

这个项目完成后，不应该只看“能打印几行日志”。要按协议验收：

1. 构造失败时不启动后台线程。
2. 多线程提交后，调用 `flush()` 能稳定看到已提交日志。
3. 析构时队列中已有消息被写完。
4. 关闭后提交不会抛出异常，也不会写入新消息。
5. 队列满时的行为和文档一致，丢弃计数可观察。
6. 后台写文件时不持有队列锁。

这些标准都是从异步对象的状态机推出来的。项目代码再小，只要跨线程、跨生命周期，就要用协议而不是运气来设计。

36.15 测试计划

日志器测试不应该依赖睡眠。可以把后台写入逻辑拆成可测试的 `LogSink`，或者给日志器提供 `flush`：

- 单线程写入一条 info，文件中出现一行。
- 多线程提交固定数量日志，最终行数正确。
- 析构后剩余消息被写完。
- 文件打不开时构造失败。
- 队列满时策略符合文档。

并发测试要避免“睡 100ms 应该写完”。更可靠的是用 `std::promise`、`std::latch` 或显式 `flush` 建立同步点。

习题与作业

给异步日志器增加 `flush()`：调用后等待当前已经提交的日志全部写完。不要用睡眠。
提示：可以向队列提交一个特殊消息，后台线程处理到它时通知 `promise`。

第 37 章

项目六：命令行文件索引器

37.1 项目目标

文件索引器扫描一个目录，统计每种扩展名的文件数量和总字节数，输出排序后的报表。这个项目覆盖文件系统遍历、错误处理、路径处理、容器聚合、排序、命令行参数和性能边界。

目标命令：

Listing 37.1 文件索引器用法

```
1 fileindex --root ./src --top 20
```

输出示例：

Listing 37.2 文件索引器输出

```
1 .cpp 128 files 240000 bytes
2 .hpp 96 files 120000 bytes
3 .md 12 files 34000 bytes
```

37.2 数据模型

Listing 37.3 扩展名统计

```
1 struct ExtensionStats {
2     std::uint64_t file_count = 0;
3     std::uint64_t total_bytes = 0;
4 };
5
6 using ExtensionIndex = std::unordered_map<std::string, ExtensionStats>;
```

这里用 `std::uint64_t`，因为文件数量和字节数可能超过 `int` 范围。类型选择来自值域，不是随手写。

37.3 路径和扩展名规则

扩展名规则要先写清楚：

- 没有扩展名的文件归为 `<none>`。
- 扩展名保留点号，例如 `.cpp`。
- 大小写是否合并由项目决定；本项目统一转小写。

实现：

Listing 37.4 提取扩展名

```
1 std::string normalized_extension(const std::filesystem::path& path)
2 {
3     std::string extension = path.extension().string();
4     if (extension.empty()) {
5         return "<none>";
6     }
7 }
```

```

6     }
7     for (char& ch : extension) {
8         ch = static_cast<char>(std::tolower(static_cast<unsigned char>(ch)));
9     }
10    return extension;
11 }

```

这个函数可以单独测试，不需要扫描真实目录。

37.4 遍历目录

Listing 37.5 扫描目录

```

1 ExtensionIndex build_index(const std::filesystem::path& root)
2 {
3     ExtensionIndex index;
4     std::error_code error;
5     std::filesystem::recursive_directory_iterator it{
6         root,
7         std::filesystem::directory_options::skip_permission_denied,
8         error};
9     std::filesystem::recursive_directory_iterator end;
10
11    for (; it != end; it.increment(error)) {
12        if (error) {
13            error.clear();
14            continue;
15        }
16        if (!it->is_regular_file(error) || error) {
17            error.clear();
18            continue;
19        }
20
21        const std::uint64_t size = it->file_size(error);
22        if (error) {
23            error.clear();
24            continue;
25        }
26        ExtensionStats& stats = index[normalized_extension(it->path())];
27        ++stats.file_count;
28        stats.total_bytes += size;
29    }
30    return index;
31 }

```

这里用 `std::error_code` 避免权限问题直接抛异常终止扫描。项目选择是：跳过无法访问的文件。生产工具应记录警告；教材版本先保留核心流程。

37.5 排序报表

Listing 37.6 排序扩展名统计

```

1 struct ExtensionRow {

```

```

2     std::string extension;
3     ExtensionStats stats;
4 };
5
6 std::vector<ExtensionRow> sort_index(const ExtensionIndex& index)
7 {
8     std::vector<ExtensionRow> rows;
9     rows.reserve(index.size());
10    for (const auto& [extension, stats] : index) {
11        rows.push_back(ExtensionRow{extension, stats});
12    }
13
14    std::ranges::sort(rows, [])(const ExtensionRow& lhs, const ExtensionRow& rhs) {
15        if (lhs.stats.total_bytes != rhs.stats.total_bytes) {
16            return lhs.stats.total_bytes > rhs.stats.total_bytes;
17        }
18        return lhs.extension < rhs.extension;
19    });
20    return rows;
21 }

```

排序规则必须稳定可解释：总字节数大的排前面，字节数相同按扩展名字典序。这样测试输出不会因为哈希表顺序变化而随机变化。

37.6 输出层

Listing 37.7 写报表

```

1 void write_report(std::ostream& output, std::span<const ExtensionRow> rows, std::↵
    ↵ size_t limit)
2 {
3     const std::size_t count = std::min(rows.size(), limit);
4     for (std::size_t i = 0; i < count; ++i) {
5         const ExtensionRow& row = rows[i];
6         output << row.extension << " "
7             << row.stats.file_count << " files "
8             << row.stats.total_bytes << " bytes\n";
9     }
10 }

```

输出函数接受 `std::ostream&`，因此可以用字符串流测试，不必真正打印到终端。

37.7 性能边界

目录遍历是 I/O 密集型。优化前先明确瓶颈：

- 文件数量很大时，遍历系统调用是主要成本。
- 扩展名种类通常不多，哈希表不是瓶颈。
- 输出 top k 可以全排序，也可以用堆；当扩展名种类少时全排序足够。
- 并行遍历可能受磁盘和文件系统限制，不一定更快。

第一版不要急着并行。先让错误处理和结果稳定。等测量证明遍历慢，再考虑多线程扫描。

37.8 测试计划

测试可以创建临时目录：

- 空目录输出为空。
- 没有扩展名的文件归为 `<none>`。
- `.CPP` 和 `.cpp` 合并。
- 多个扩展名按总字节数排序。
- 权限错误不让整个扫描崩溃。

临时目录要用 RAII 管理，测试结束自动删除。这样测试不会污染工作区。

习题与作业

给文件索引器增加 `-min-size N`，只统计至少 `N` 字节的文件。要求参数解析拒绝负数和非数字；扫描逻辑不关心命令行字符串，只接收已经解析好的整数。

37.9 扫描策略要先定义错误政策

文件系统遍历会遇到权限不足、符号链接循环、文件在扫描中被删除、路径太长等问题。如果不先定义错误政策，代码就会在异常和忽略之间摇摆。本项目采用：

- 根目录打不开：命令失败。
- 子目录权限不足：跳过并记录警告。
- 单个文件读取大小失败：跳过并记录警告。
- 符号链接：第一版不跟随。

这些政策会影响接口。只返回 `ExtensionIndex` 不够，还应返回警告：

Listing 37.8 索引结果带警告

```
1 struct IndexWarning {
2     std::filesystem::path path;
3     std::string message;
4 };
5
6 struct IndexResult {
7     ExtensionIndex index;
8     std::vector<IndexWarning> warnings;
9 };
```

这样 CLI 可以把警告写到错误流，测试也能断言权限问题没有让整个扫描失败。

37.10 遍历循环的证据字段

`recursive_directory_iterator` 使用 `error_code` 时，每一步都要检查。可以把循环理解成三段：

1. 迭代器推进是否成功。
2. 当前条目是否是普通文件。
3. 文件大小是否可读。

每段失败都产生不同警告。不要把所有失败都简单 `continue`，否则用户不知道为什么统计少了文件。教材里可以先简化，但工程版本必须留下错误证据。

37.11 top k 是否需要堆

输出前 `k` 个扩展名时有两种做法。第一种是全部排序，复杂度 $O(m \log m)$ ，其中 `m` 是扩展名种类数。第二种是维护大小为 `k` 的堆，复杂度 $O(m \log k)$ 。听起来堆更高级，但文件扩展名种类通常

很少，全部排序简单、稳定、可读，完全足够。

这个例子训练算法选择：不要因为知道更复杂的数据结构就立刻使用。先估计 m 的规模。如果 m 只有几十或几百，排序成本远小于文件系统遍历成本。真正热点在系统调用和磁盘 I/O。

37.12 min-size 参数贯穿三层

增加 `-min-size N` 时，要分三层：

层	输入	责任
参数解析	字符串 <code>N</code>	完整解析非负整数
扫描配置	<code>std::uint64_t min_size</code>	保存已经验证的阈值
扫描逻辑	文件大小	小于阈值则跳过

扫描函数不应该知道 `"-min-size"` 这个字符串。它只接收配置对象：

Listing 37.9 扫描配置对象

```
1 struct IndexOptions {
2     std::filesystem::path root;
3     std::uint64_t min_size = 0;
4     std::size_t top = 20;
5 };
```

这样命令行、配置文件或图形界面都可以复用同一个扫描逻辑。

37.13 临时目录测试

文件索引器需要真实文件系统测试，但要控制范围。测试可以创建临时目录：

Listing 37.10 临时目录测试思路

```
1 TemporaryDirectory temp;
2 write_file(temp.path() / "a.cpp", "1234");
3 write_file(temp.path() / "b.HPP", "12");
4 write_file(temp.path() / "README", "x");
5
6 const IndexResult result = build_index(IndexOptions{.root = temp.path()});
7 assert(result.index.at(".cpp").file_count == 1);
8 assert(result.index.at(".hpp").file_count == 1);
9 assert(result.index.at("<none>").file_count == 1);
```

`TemporaryDirectory` 应使用 RAII：构造时创建目录，析构时删除。这样测试失败时也尽量清理现场。不要把测试文件散落到仓库根目录。

37.14 输出稳定性

因为 `unordered_map` 遍历顺序不稳定，任何面向用户或测试的输出都必须先排序。排序规则要覆盖平局情况：字节数相同按扩展名升序，扩展名相同不可能出现。只要平局规则不完整，测试就可能在不同平台或不同运行中随机失败。

37.15 项目验收清单

文件索引器完成时至少验收：

- 空目录输出为空。
- 大小写扩展名合并。

- 无扩展名归入固定名称。
- top 限制生效。
- min-size 拒绝非法参数。
- 单文件错误产生警告而不是崩溃。
- 输出顺序稳定。

这些验收项覆盖了文件系统项目最容易出错的边界：路径、错误、排序、参数和可重复测试。

第 38 章

项目七：插件式命令行工具

38.1 项目目标

本项目实现一个插件式命令行工具骨架。它支持多个子命令，例如 `wordfreq`、`fileindex`、`configcheck`。每个子命令有自己的参数解析和执行逻辑，主程序只负责分发。

目标用法：

Listing 38.1 插件式 CLI 用法

```
1 tool wordfreq --input article.txt --top 20
2 tool fileindex --root ./src --top 10
3 tool configcheck --path server.conf
```

这个项目训练大型代码组织：接口、注册表、命令边界、错误码和测试替身。

38.2 命令接口

每个命令都实现同一个接口：

Listing 38.2 命令接口

```
1 struct CommandContext {
2     std::ostream& output;
3     std::ostream& error;
4 };
5
6 class Command {
7 public:
8     virtual ~Command() = default;
9     [[nodiscard]] virtual std::string_view name() const noexcept = 0;
10    [[nodiscard]] virtual int run(std::span<const std::string_view> args,
11                                CommandContext context) const = 0;
12 };

```

为什么 `CommandContext` 里放输出流？这样测试命令时可以传 `std::ostringstream`，不用真的写终端。接口从一开始就为测试留出边界。

38.3 命令注册表

Listing 38.3 命令注册表

```
1 class CommandRegistry {
2 public:
3     void add(std::unique_ptr<Command> command)
4     {
5         const std::string key{command->name()};
6         this->commands_[key] = std::move(command);
7     }
8

```

```

9      [[nodiscard]] const Command* find(std::string_view name) const
10     {
11         const auto found = this->commands_.find(std::string{name});
12         if (found == this->commands_.end()) {
13             return nullptr;
14         }
15         return found->second.get();
16     }
17
18 private:
19     std::unordered_map<std::string, std::unique_ptr<Command>> commands_;
20 };

```

注册表拥有命令对象，所以用 `std::unique_ptr<Command>`。查找返回裸指针，只表示观察，不转移所有权。这个接口把拥有和观察区分开了。

38.4 主分发逻辑

Listing 38.4 命令分发

```

1 int dispatch(const CommandRegistry& registry,
2             std::span<const std::string_view> args,
3             CommandContext context)
4 {
5     if (args.empty()) {
6         context.error << "missing command\n";
7         return 2;
8     }
9
10    const Command* command = registry.find(args.front());
11    if (command == nullptr) {
12        context.error << "unknown command: " << args.front() << '\n';
13        return 2;
14    }
15
16    return command->run(args.subspan(1), context);
17 }

```

分发器不关心 `wordfreq` 怎么解析参数，也不关心 `fileindex` 怎么扫描文件。这就是边界。新增命令时，主分发逻辑不需要修改。

38.5 实现一个 configcheck 命令

Listing 38.5 配置检查命令

```

1 class ConfigCheckCommand final : public Command {
2 public:
3     [[nodiscard]] std::string_view name() const noexcept override
4     {
5         return "configcheck";
6     }
7
8     [[nodiscard]] int run(std::span<const std::string_view> args,

```

```

9             CommandContext context) const override
10     {
11         if (args.size() != 2 || args[0] != "--path") {
12             context.error << "usage: configcheck --path FILE\n";
13             return 2;
14         }
15
16         const ConfigResult result = load_config_file(std::filesystem::path{args[1]});
17         if (result.error.has_value()) {
18             context.error << result.error->message << '\n';
19             return 1;
20         }
21         context.output << "ok\n";
22         return 0;
23     }
24 };

```

命令内部可以调用配置加载器项目。这样前面的项目不是孤立练习，而是能组合进更大的工具。

38.6 注册命令

Listing 38.6 组装命令

```

1 CommandRegistry build_registry()
2 {
3     CommandRegistry registry;
4     registry.add(std::make_unique<ConfigCheckCommand>());
5     registry.add(std::make_unique<WordFreqCommand>());
6     registry.add(std::make_unique<FileIndexCommand>());
7     return registry;
8 }

```

这里的 `build_registry` 是装配层。它知道具体命令类型，业务分发器不知道。装配层集中依赖具体实现，是大型项目常见组织方式。

38.7 测试分发器

可以写一个假命令：

Listing 38.7 测试用假命令

```

1 class FakeCommand final : public Command {
2 public:
3     [[nodiscard]] std::string_view name() const noexcept override
4     {
5         return "fake";
6     }
7
8     [[nodiscard]] int run(std::span<const std::string_view> args,
9                           CommandContext context) const override
10    {
11        context.output << "args=" << args.size() << '\n';
12        return 0;
13    }

```

```
14 };
```

测试分发器时，不需要真实扫描文件或读取配置。只验证：缺命令报错，未知命令报错，已知命令收到剩余参数。这就是依赖抽象的测试价值。

38.8 插件化的下一步

本章的“插件式”还是编译期注册。真正动态插件还涉及动态库加载、ABI 稳定、版本协商和安全边界。不要一开始就做动态插件。先把命令接口、注册表和测试边界做好，已经能支撑大多数内部工具。

习题与作业

给 CLI 增加 `help` 命令。要求每个命令提供 `usage()`，`help` 能列出所有命令。思考注册表是否应该保留命令顺序；如果需要稳定顺序，应该增加什么数据结构？

38.9 项目从目录结构开始

插件式 CLI 如果只停留在一个文件里，很快会变成大杂烩。一个可维护的最小目录可以这样安排：

Listing 38.8 命令行工具目录结构

```
1 src/
2   main.cpp
3   cli/
4     command.hpp
5     command_registry.hpp
6     dispatch.cpp
7   commands/
8     config_check_command.cpp
9     word_freq_command.cpp
10    file_index_command.cpp
11  app/
12    build_registry.cpp
13 tests/
14   cli/
15     dispatch_tests.cpp
16   commands/
17     config_check_command_tests.cpp
```

`cli` 目录只放抽象接口和分发逻辑；`commands` 放具体命令；`app` 是装配层，知道要注册哪些命令；`main.cpp` 只连接系统入口。目录结构不是形式，它让依赖方向清楚：核心分发不依赖具体命令，具体命令依赖各自项目能力，装配层把它们拼起来。

38.10 注册表还需要顺序

前面的 `CommandRegistry` 用哈希表查找命令，适合分发。但 `help` 命令需要稳定列出命令。如果直接遍历 `unordered_map`，输出顺序不稳定。可以同时维护一个顺序数组：

Listing 38.9 同时支持查找和稳定顺序

```
1 class CommandRegistry {
2 public:
3     void add(std::unique_ptr<Command> command)
4     {
```

```

5      const std::string key{command->name()};
6      this->ordered_.push_back(command.get());
7      this->commands_[key] = std::move(command);
8  }
9
10     [[nodiscard]] std::span<Command* const> ordered() const noexcept
11     {
12         return this->ordered_;
13     }
14
15 private:
16     std::unordered_map<std::string, std::unique_ptr<Command>> commands_;
17     std::vector<Command*> ordered_;
18 };

```

这里 `commands_` 拥有对象，`ordered_` 只保存观察指针。这个设计成立的前提是：命令对象由 `unique_ptr` 管理，移动 `unique_ptr` 本身不会移动被拥有的命令对象；注册表析构时，两个成员一起销毁，外部不能长期保存 `ordered` 返回的指针。如果你觉得这个生命周期证明太绕，可以改成 `std::vector<std::string>` 保存命令名，用名字再查找，牺牲一点效率换更简单的所有权。

38.11 usage 是接口的一部分

为了支持 `help`，命令接口应扩展：

Listing 38.10 命令提供用法说明

```

1 class Command {
2 public:
3     virtual ~Command() = default;
4     [[nodiscard]] virtual std::string_view name() const noexcept = 0;
5     [[nodiscard]] virtual std::string_view usage() const noexcept = 0;
6     [[nodiscard]] virtual int run(std::span<const std::string_view> args,
7                                 CommandContext context) const = 0;
8 };

```

然后 `HelpCommand` 可以依赖注册表的只读视图：

Listing 38.11 `help` 命令列出所有命令

```

1 class HelpCommand final : public Command {
2 public:
3     explicit HelpCommand(const CommandRegistry& registry)
4         : registry_{registry}
5     {
6     }
7
8     [[nodiscard]] std::string_view name() const noexcept override
9     {
10         return "help";
11     }
12
13     [[nodiscard]] std::string_view usage() const noexcept override
14     {
15         return "help";
16     }

```



```
17
18 [[nodiscard]] int run(std::span<const std::string_view>,
19                       CommandContext context) const override
20 {
21     for (const Command* command : this->registry_.ordered()) {
22         context.output << command->usage() << '\n';
23     }
24     return 0;
25 }
26
27 private:
28     const CommandRegistry& registry_;
29 };
```

这个版本引入了一个新生命周期要求：`HelpCommand` 引用的注册表必须比它活得久。如果 `HelpCommand` 本身也被放进注册表，就会形成自引用结构，设计要更谨慎。更简单的方案是让 `help` 不是普通命令，而是分发器内置逻辑。工程设计没有唯一答案，关键是把所有权和生命周期写清楚。

38.12 错误码合同

命令行工具的返回码也要统一：

返回码	含义
0	成功
1	命令执行失败，例如文件不存在、配置非法
2	用法错误，例如未知命令、参数缺失、参数格式错误

有了这个合同，测试就能写得具体：

Listing 38.12 分发器错误码测试

```
1 std::ostringstream output;
2 std::ostringstream error;
3 const int code = dispatch(registry, {}, CommandContext{output, error});
4 assert(code == 2);
5 assert(error.str().find("missing command") != std::string::npos);
```

返回码不是小事。脚本、CI 和其他程序都会依赖它判断工具是否成功。

38.13 从编译期插件到动态插件

本章的插件是编译期插件：所有命令在编译时链接进程序，只是在运行时通过注册表分发。它已经足够训练抽象边界。动态插件则是另一个项目：要考虑动态库加载、符号导出、ABI 稳定、版本协商、崩溃隔离和安全策略。不要把“可扩展”一上来理解成“必须加载动态库”。大多数内部 CLI 先做到编译期可扩展、测试清楚、错误码稳定，就已经很有价值。

38.14 项目验收清单

完成这个项目时，至少验收：

- 缺命令返回 2，错误输出可读。
- 未知命令返回 2，包含原始命令名。
- 已知命令只收到自己的剩余参数。
- 命令可以用假输出流测试，不依赖真实终端。
- `help` 输出顺序稳定。

- 新增命令只需要改装配层和测试，不需要改分发器。

如果这些都成立，这个 CLI 骨架就不只是“能跑”，而是能承受后续项目继续接入。

附录 A

学习检查清单

A.1 入门阶段

- 能解释编译、链接和运行的区别。
- 能区分对象、值、名字、引用和指针。
- 能写出不越界的 `vector` 遍历。
- 能把读输入、处理数据、输出结果拆成不同函数。
- 能用 `const&` 避免只读场景的复制。

A.2 中级阶段

- 能用构造函数建立类不变量。
- 能说明 RAII 如何处理提前返回和异常。
- 能根据需求选择 `vector`、`unordered_map`、`map` 等容器。
- 能把错误放进接口，而不是用魔法返回值。
- 能写出基本单元测试覆盖边界输入。
- 能区分引用、指针、`span` 和 `string_view` 的生命周期要求。
- 能说明对象是否应该复制、移动，还是禁止复制。
- 能解释表达式里的副作用、作用域和常量边界。
- 能用小实验复现编译器诊断、容器扩容和对象布局差异。
- 能把公开头文件、源文件和测试文件拆开，并解释 `include` 边界。

A.3 高级阶段

- 能用 `concept` 约束模板参数。
- 能判断 `ranges` 视图是否存在悬空风险。
- 能用测量定位性能瓶颈，而不是凭直觉优化。
- 能用 `mutex`、`atomic` 和 `condition variable` 解释同步边界。
- 能拆分大型类，控制依赖方向。
- 能判断虚函数、模板、`std::function` 和 `variant` 分别适合什么变化方向。
- 能用 `constexpr`、`concept` 和 `type traits` 把类型错误前移到编译期。
- 能说明一个函数提供基本保证、强保证还是不抛保证。
- 能为异步队列设计关闭、取消和背压策略。
- 能重构混合 I/O 和业务逻辑的函数，并用测试固定行为。
- 能区分数据竞争和逻辑竞争，知道 `atomic` 不能自动保护多个字段的不变量。
- 能把 CMake 目标、测试目标和 CI 命令组织成可复现构建。
- 能解释协程解决的控制流问题，以及它不等于自动多线程。

A.4 项目验收清单

- 每个公开函数都能说清楚所有权：观察、修改、保存副本还是接管资源。
- 每个错误路径都有测试或明确的人工验证步骤。
- 每个容器选择都能说出主要操作和复杂度理由。
- 每个类至少有一个围绕不变量的测试，而不只是 `getter` 测试。
- 每次性能优化都有优化前后的输入规模、构建模式和测量结果。

- 每个后台线程都有退出协议，程序关闭时不会依赖强制杀线程。

A.5 读代码自检问题

1. 这段代码有没有隐藏的资源释放规则？
2. 有没有返回局部对象的引用、保存短生命周期视图、或长期持有可能失效的迭代器？
3. 错误是被吞掉、打印后继续，还是进入了接口？
4. 测试是否覆盖空输入、单元素、边界值和失败路径？
5. 如果数据规模扩大一百倍，主要成本会落在哪里？

附录 B

标准库能力地图

领域	常用设施	学习重点
字符串	<code>std::string</code> , <code>std::string_view</code>	拥有和观察的区别
连续容器	<code>std::vector</code> , <code>std::array</code> , <code>std::span</code>	生命周期、扩容、边界
关联容器	<code>std::map</code> , <code>std::unordered_map</code>	有序性、哈希、复杂度
资源管理	<code>unique_ptr</code> , <code>shared_ptr</code> , RAII 类型	所有权表达
错误表达	<code>optional</code> , <code>error_code</code>	异常, 错误是否进入接口
算法	<code>find</code> , <code>sort</code> , <code>copy_if</code> , <code>accumulate</code>	用标准词汇表达意图
泛型 并发	<code>template</code> , <code>concept</code> , <code>type traits</code> <code>thread</code> , <code>mutex</code> , <code>atomic</code> , <code>condition_variable</code>	能力约束和实例化成本 共享状态和同步边界

这张表不是背诵列表。每学一个设施，都要问三件事：它是否拥有资源，它的复杂度合同是什么，它失效或出错时会怎样。

B.1 按问题选择设施

你遇到的问题	优先考虑	先问清楚
函数可能没有结果 函数要返回成功或错误	<code>std::optional</code> 结果结构、 <code>expected</code> 风格类型	没有结果是否还需要原因 调用者是否要按错误类型分支
只观察一段连续数据 只观察字符串内容 独占动态对象 共享生命周期对象	<code>std::span<const T></code> <code>std::string_view</code> <code>std::unique_ptr</code> <code>std::shared_ptr</code> , <code>std::weak_ptr</code>	底层数据是否活得足够久 是否会保存到调用之后 所有权在哪里转移 是否存在循环引用
按 key 快速查找	<code>std::unordered_map</code>	是否需要稳定顺序和自定义 hash
表达筛选转换 保护共享状态	<code>std::ranges</code> , <code>views</code> <code>std::mutex</code> , RAII 锁	视图是否可能悬空 锁保护的不变量是什么

B.2 容易误用的设施

- `std::string_view`: 它不拥有字符，不能替代需要长期保存的 `std::string`。
- `std::shared_ptr`: 它不是“不想思考所有权”的默认答案。
- `std::vector::reserve`: 它只预留容量，不创建元素。
- `std::remove`: 它不删除容器元素，需要配合 `erase`。
- ranges 视图: 它常常惰性且非拥有，返回视图前要检查底层生命周期。

- `std::atomic`: 它只解决特定共享变量的原子访问，不自动保护多个字段的不变量。

练习路线图

C.1 第一阶段：写小函数但不跳过边界

第一阶段不要急着写大型项目。目标是把函数输入、输出和错误路径写清楚。建议练习：

- 字符串修剪：实现 `trim`、`split`、`starts_with`。
- 数字解析：用 `std::from_chars` 解析整数，拒绝半截成功。
- 成绩统计：空输入、单元素、多元素都要测试。
- 用户名校验：长度、字符集合、空字符串都要覆盖。

每个练习都要写至少五个测试。不要只测普通输入；普通输入通常最不容易暴露问题。

C.2 第二阶段：写小类型并保护不变量

这一阶段练习类设计：

- **Money**：用整数分表示金额，禁止把数量和金额混加。
- **Date**：构造时校验年月日。
- **TemperatureRange**：保证最低温不高于最高温。
- **Percent**：范围限制在 `0` 到 `100`。

每个类型都要回答：对象什么时候有效？哪些状态不允许出现？接口是业务动作还是裸 `setter`？对象是否应该复制？

C.3 第三阶段：写资源和容器项目

这一阶段练习 RAII 和容器选择：

- 临时目录：构造创建目录，析构删除目录，禁止复制，允许移动。
- LRU 缓存：哈希表加链表，验证映射和链表同步。
- 配置加载器：解析、校验、强类型配置分层。
- 词频统计器：清洗、统计、排序、输出分开。

项目不要一口气写完。先写纯函数，再接 I/O；先测核心逻辑，再测文件入口。

C.4 第四阶段：写并发和工程化练习

并发练习必须小心，不要用睡眠掩盖同步问题：

- 线程安全计数器：分别用 `mutex` 和 `atomic` 实现。
- 可关闭队列：支持 `push`、`pop`、`close`。
- 任务执行器：使用 `jthread` 管理生命周期。
- 配置热加载：先解析到临时对象，再原子替换共享配置。

每个并发练习都要写出共享状态、不变量、锁保护范围和退出协议。写不出来这些，就先不要写代码。

C.5 第五阶段：复盘自己的代码

每完成一个练习，按下面问题复盘：

1. 哪个函数最难测试？为什么？
2. 哪个类型的不变量最重要？测试有没有覆盖？
3. 有没有无意复制大对象？

4. 有没有保存非拥有视图到过长生命周期?
5. 错误消息能不能帮助用户定位问题?
6. 如果输入扩大一百倍, 瓶颈可能在哪里?

复盘比刷更多语法更重要。高级 C++ 能力不是知道更多特性, 而是能在复杂约束下写出可解释、可测试、可维护的代码。

术语表

RAII Resource Acquisition Is Initialization。把资源生命周期绑定到对象生命周期。

不变量 对象在对外可见状态下必须始终成立的条件。

值语义 对象可以像普通值一样复制、赋值和组合，副本之间互不影响。

所有权 谁负责释放资源、谁能延长资源生命周期的规则。

悬空引用 引用或指针指向的对象已经销毁。

迭代器失效 容器操作后，原迭代器不再安全可用。

concept C++20 中用于约束模板参数能力的语言机制。

数据竞争 多个线程并发访问同一对象，至少一个写入，且没有同步。

视图 观察已有数据的轻量对象，通常不拥有底层数据。

严格弱序 排序比较器需要满足的关系，相等元素之间不能互相小于对方。

死锁 多个执行单元互相等待对方释放资源，导致都无法继续。

RAII 锁 用对象构造和析构管理加锁、解锁的同步写法。

lambda 可以在局部定义的匿名可调用对象，常用于谓词、回调和小策略。

类型擦除 隐藏具体可调用对象或实现类型，只暴露统一接口的技术。

异常安全 异常发生后对象和程序状态仍满足约定保证的能力。

强保证 函数失败后状态像没有调用过一样。

虚函数 通过基类接口在运行期分派到派生类实现的机制。

variant 表示若干已知类型之一的标准库类型，适合封闭类型集合。

背压 下游处理不过来时，让上游减速、失败或丢弃任务的过载控制机制。

对齐 对象在内存中按平台要求放置到特定地址边界的规则。

填充字节 编译器为满足对齐或对象布局要求插入的无业务含义字节。

特征测试 在重构前固定现有外部行为的测试，用于防止行为漂移。

重载 多个函数共享同一名字，但参数列表不同，由编译器根据调用选择。

命名空间 控制名字归属和范围的语言机制，用于组织模块并减少冲突。

词汇类型 标准库中表达常见语义的类型，例如 `optional`、`variant`、`span`。

顺序一致 默认原子内存序，提供最容易推理的全局一致原子操作顺序。

逻辑竞争 没有数据竞争但业务检查和更新不是一个原子事务，导致结果违反业务规则。

constexpr 表示表达式或函数可用于编译期求值的语言机制。

type traits 在编译期查询类型属性的一组模板工具。

协程 可以暂停和恢复的函数形态，用于表达异步控制流。

协程帧 协程暂停时保存局部状态和恢复位置的存储区域。

索引

invariant, 36

不变量, 36